

系统架构师历年论文真题解析及精选范文实战汇编

目录

2021 年系统架构师论文.....	10
一、论面向方面的编程技术及其应用 (AOP)	10
1、真题原文:	10
2、参考理论:	10
3、参考范文:	11
二、论系统安全架构设计及其应用	12
1、真题原文:	12
2、参考理论:	12
3、参考范文:	13
三、论企业集成平台的理解与应用	19
1、真题原文:	19
2、参考理论:	19
3、参考范文:	20
四、论微服务架构及其应用	26
1、真题原文:	26
2、参考理论:	26
3、参考范文:	28
2020 年系统架构师论文.....	36
一、论数据分片技术及其应用	36
1、真题原文:	36
2、参考理论:	36
3、参考范文:	40
二、论云原生架构及其应用	40
1、真题原文:	40
2、参考理论:	40
3、参考范文:	41
三、论软件测试中缺陷管理及其应用	44
1、真题原文:	44

2、参考理论：	45
3、参考范文：	47
四、论企业集成架构设计及应用	50
1、真题原文：	50
2、参考理论：	50
3、参考范文：	51
2019 年系统架构师论文	53
一、论软件设计方法及其应用	53
1、真题原文：	53
2、参考理论：	54
3、参考范文：	55
二、论软件系统架构评估及其应用	58
1、真题原文：	58
2、参考理论：	59
3、参考范文：	59
三、论数据湖技术及其应用	62
1、真题原文：	62
2、参考理论：	63
3、参考范文：	63
四、论负载均衡技术在 Web 系统中的应用	70
1、真题原文：	70
2、参考理论：	71
3、参考范文：	71
2018 年系统架构师论文	74
一、论软件开发过程 RUP 及其应用	74
1、真题原文：	74
2、参考理论：	75
3、参考范文：	76
二、论软件体系结构的演化	76

1、真题原文：	76
2、参考理论：	77
3、参考范文：	78
三、论面向服务架构设计及其应用	78
1、真题原文：	78
2、参考理论：	78
3、参考范文：	79
四、论 NoSQL 数据库技术及其应用	82
1、真题原文：	82
2、参考理论：	83
3、参考范文：	86
2017 年系统架构师论文	86
一、论软件系统建模方法及其应用	86
1、真题原文：	86
2、参考理论：	87
3、参考范文：	88
二、论软件架构风格	91
1、真题原文：	91
2、参考理论：	91
3、参考范文：	95
三、论无服务器架构及其应用	101
1、真题原文：	101
2、参考理论：	101
3、参考范文：	103
四、论软件质量保证及其应用	104
1、真题原文：	104
2、参考理论：	104
3、参考范文：	105
2016 年系统架构师论文	108

一、论软件系统架构评估	108
1、真题原文:	108
2、参考理论:	108
3、参考范文:	111
二、论软件设计模式及其应用	113
1、真题原文:	113
2、参考理论:	114
3、参考范文:	115
三、论数据访问层设计技术及其应用	121
1、真题原文:	121
2、参考理论:	122
3、参考范文:	123
四、论微服务架构及其应用	133
1、真题原文:	133
2、参考理论:	134
3、参考范文:	135
2015 年系统架构师论文	141
一、论应用服务器基础软件	141
1、真题原文:	141
2、参考理论:	141
3、参考范文:	142
二、论软件系统架构风格	142
1、真题原文:	142
2、参考理论:	143
3、参考范文:	146
三、论面向服务的架构及其应用	149
1、真题原文:	149
2、参考理论:	149
3、参考范文:	150

四、论企业集成平台的技术与应用	156
1、真题原文:	156
2、参考理论:	157
3、参考范文:	159
2014 年系统架构师论文	164
一、论软件需求管理	164
1、真题原文:	164
2、参考理论:	164
3、参考范文:	166
二、论非功能性需求对企业应用架构设计的影响	166
1、真题原文:	166
2、参考理论:	167
3、参考范文:	169
三、论软件的可靠性设计	172
1、真题原文:	172
2、参考理论:	172
3、参考范文:	174
四、论网络安全体系设计	180
1、真题原文:	180
2、参考理论:	180
3、参考范文:	182
2013 年系统架构师论文	183
一、软件架构建模技术与应用	183
1、真题原文:	183
2、参考理论:	183
3、参考范文:	184
二、企业应用系统的分层架构风格	189
1、真题原文:	189
2、参考理论:	190

3、参考范文：	191
三、论软件可靠性设计技术的应用	192
1、真题原文：	192
2、参考理论：	193
3、参考范文：	194
四、论分布式存储系统架构设计	197
1、真题原文：	197
2、参考理论：	197
3、参考范文：	198
2012 年系统架构师论文	201
一、论基于架构的软件设计方法及应用	201
1、真题原文：	201
2、参考理论：	202
3、参考范文：	203
二、论企业应用系统的数据持久层架构设计	206
1、真题原文：	206
2、参考理论：	206
3、参考范文：	207
三、论决策支持系统的开发与应用	207
1、真题原文：	207
2、参考理论：	208
3、参考范文：	210
四、论企业信息化规划的实施与应用	210
1、真题原文：	210
2、参考理论：	210
3、参考范文：	211
2011 年系统架构师论文	211
一、论模型驱动架构在系统开发中的应用	211
1、真题原文：	211

2、参考理论：	212
3、参考范文：	212
二、论企业集成平台的架构设计	212
1、真题原文：	212
2、参考理论：	213
3、参考范文：	214
三、论企业架构管理与应用	217
1、真题原文：	217
2、参考理论：	217
3、参考范文：	219
四、论软件需求获取技术及应用	219
1、真题原文：	219
2、参考理论：	220
3、参考范文：	220
2010 年系统架构师论文	224
一、论软件的静态演化和动态演化及其应用	224
1、真题原文：	224
2、参考理论：	225
3、参考范文：	227
二、论软件可靠性评价	227
1、真题原文：	227
2、参考理论：	227
3、参考范文：	229
三、论数据挖掘技术的应用	234
1、真题原文：	234
2、参考理论：	235
3、参考范文：	237
四、大规模分布式系统缓存设计策略	239
1、真题原文：	239

2、参考理论：	239
3、参考范文：	240
2009 年系统架构师论文	243
一、论基于 DSSA 的软件架构设计与应用	243
1、真题原文：	243
2、参考理论：	244
3、参考范文：	244
二、论信息系统建模方法	250
1、真题原文：	250
2、参考理论：	251
3、参考范文：	251
三、论基于 REST 服务的 Web 应用系统设计	254
1、真题原文：	254
2、参考理论：	254
3、参考范文：	255
四、软件可靠性及应用	258
1、真题原文：	258
2、参考理论：	258
3、参考范文：	259

2021 年系统架构师论文

一、论面向方面的编程技术及其应用（AOP）

1、真题原文：

论面向方面的编程技术及其应用（AOP）

面向过程编程是一种自顶向下的编程方法，其实质是对软件进行功能性分解。它适用于小型软件系统，例如某一算法的实现。在大型应用系统中，自顶向下逐步求精的方法无论在系统体系结构的确立，系统的进化和维护，以及软件重用性方面都存在其不足之处。

请围绕“论面向方面的编程技术及其应用（AOP）”论题，依次从以下三个方面进行论述。

- 1、概要叙述你所参与管理或开发的软件项目，以及你在其中所承担的主要工作。
- 2、叙述在项目实践过程使用 AOP 技术开发的具体步骤。
- 3、结合项目，论述使用 AOP 的原因，开发过程中存在的问题及所使用技术带来的实际应用效果。

2、参考理论：

AOP(Aspect-Oriented Programming，面向方面编程)，可以说是 OOP(Object-Oriented Programming，面向对象编程)的补充和完善。OOP 引入封装、继承和多态性等概念来建立一种对象层次结构，用以模拟公共行为的一个集合。当我们要为分散的对象引入公共行为的时候，OOP 往往会显得无能为力。也就是说面向对象往往合适定义从上到下的关系，但在定义从左到右的关系的时候，就会变得不合适了。

例如日志功能的代码就往往分布在所有的对象层次中，而与它所散步到的对象的核心功能毫无关系。对于其他类型的代码，如安全性，异常处理和透明的持续性也是如此。这种散步在各处的无关的代码被称为(cross-cutting)代码，在 OOP 设计中，它导致了大量代码的重复，而不利于各个模块的重用。

而 AOP 技术则恰恰相反，它利用一种称为横切的技术，破解开封装的对象内部，并那些

影响了多个类的公共行为封装到一个可重用的模块，并将其命名为"Aspect"，即方面。

所谓方面就是将那些与业务无关的，却为业务模块所共同调用的逻辑或责任都封装起来，以减少系统的重复代码，降低模块间的耦合度，并有利于未来的可操作性和可维护性。AOP 代表的是一个横向的关系，如果说对象是一个空心的圆柱体，其中封装的是对象的属性和行为；那么面向方面的编程就是一把利刃，将这些空心圆柱体剖开，以获得其内部的消息。而剖开的切面就是所谓的方面了。然后它又以巧夺天工的妙手将这些剖开的切面复原，不留痕迹。

使用“横切”技术，AOP 把软件系统分为两个部分：核心关注点和横切关注点。业务处理的主要流程是核心关注点，与之关系不大的部分是横切关注点。横切关注点的一个特点是，他们经常发生在核心关注点的多处，而各处都基本相似，比如权限认证、日志、事务处理。AOP 的作用在于分离系统中的各种关注点，将核心关注点和横切关注点分离开来。

AOP 应用程序包括以下三个主要的开发步骤：

1、区分横切关注点

将系统需求进行功能性分解，区分出普通关注点以及横切关注点，确定哪些功能是组件语言必须实现的，哪些功能可以以 aspect 的形式动态加入到系统组件中。

2、构造出系统的切面

单独完成每一个关注点的编码和实现，构造系统组件和系统 aspect。这里的系统组件，是实现该系统的基本模块，对 OOP 语言，这些组件可以是类，对于过程化程序设计语言，这些组件可是各种函数和 API。系统 aspect 是指用 AOP 语言实现的将横切关注点封装成的独立的模块单元。

3、组件代码与切面代码结合形成系统

用联接器指定的重组规则，将组件代码和 aspect 代码进行组合，形成最终系统。为达到此目的，应用程序需要利用或创造一种专门指定规则的语言，用它来组合不同应用程序片断。这种用来指定联结规则的语言可以是一种已有编程语言的扩展，也可以是一种完全不同的全新语言。

3、参考范文：

二、论系统安全架构设计及其应用

1、真题原文：

论系统安全架构设计及其应用

信息安全的特征是为了保证信息的机密性、完整性、可用性、可控性和不可抵赖性。信息系统的安全保障是以风险和策略为基础，在信息系统的整个生命周期中提供包括技术、管理、人员和工程过程的整体安全，在信息系统中保障信息的这些安全特征，并实现组织机构的使命。许多信息系统的用户需要提供一种方法和内容对信息系统的技术框架、工程过程能力和管理能力提出安全性要求，并进行可比性的评估、设计和实施。

请围绕“论系统安全架构设计及其应用”论题，依次从以下三个方面进行论述。

- 1、概要叙述你所参与管理或开发的软件项目，以及你在其中所承担的主要工作。
- 2、详细论述安全架构设计中鉴别框架和访问控制框架设计的内容，并论述鉴别和访问控制所面临的主要威胁，并说明其危害。
- 3、阐述你在软件开发的过程中都遇到了哪些实际问题及解决方法。

2、参考理论：

鉴别（Authentication）的基本目的就是防止其他实体占用和独立操作被鉴别实体的身份。鉴别提供了实体声称其身份的保证。鉴别的两种重要的关系背景：一是实体由申请者来代表，申请者与验证者之间存在着特定的通信关系（如实体鉴别）；二是实体为验证者提供数据项来源。

鉴别的方式主要有以下几种：

1. 已知的，如一个秘密的口令
2. 拥有的，如 IC 卡，令牌等
3. 不改变的特征，如生物特征
4. 相信可靠的第三方建立的鉴别（递推）
5. 环境（如主机地址等）。

访问控制（AccessControl）决定开放系统环境中允许使用哪些资源，在什么地方适合阻止未授权的访问的过程。在访问控制实例中，访问可以是对一个系统（即对一个系统通信

部分的一个实体) 或对一个系统内部进行的。

鉴别服务:

用户名+密码

数字证书

生物特征识别

访问控制:

自主访问控制、访问控制列表、强制访问控制、基于角色的访问控制、基于任务的访问控制

数据完整性: 阻止对媒体访问的机制: 隔离、访问控制、路由控制

探测非授权修改的机制: 数字签名、数据重复、数字指纹、消息序列

数据保密性: 通过禁止访问提供机密性

通过加密提供机密性

抗抵赖: 数字签名

3、参考范文:

范文 1:

[摘要]

我在一家证券公司信息技术部门工作, 我公司在 97-98 年建成了与各公司总部及营业网点的企业网络, 并已先后在企业网络上建设了交易系统、办公系统, 并开通了互联网应用。因将对安全要求不同、安全可信度不同的各种应用运行在同一网络上, 给黑客的攻击、病毒的蔓延打开了方便之门, 给我公司的网络安全造成了很大的威胁。作为信息技术中心部门经理及项目负责人, 我在资金投入不足的前提下, 充分利用现有条件及成熟技术, 对公司网络进行了全面细致的规划, 使改造后的网络安全级别大大提高。

本文将介绍我在网络安全性和保密性方面采取的一些方法和策略, 主要包括网络安全隔离、网络边界安全控制、交叉病毒防治、集中网络安全管理等, 同时分析了因投入资金有限, 我公司网络目前仍存在的一些问题或不足, 并提出了一些改进办法。

[正文]

我在一家证券公司工作, 公司在 98 年就建成了与各公司总部及营业网点的企业网络,

随着公司业务的不不断拓展，公司先后建设了集中报盘系统、网上交易系统、OA、财务系统、总部监控系统等等，为了保证各业务正常开展，特别是为了确保证券交易业务的实时高效，公司已于 2002 年已经将中心至各营业部的通讯链路由初建时的主链路 64K 的 DDN 和备链路 33.3KTSN, 扩建成主链路为 2M 光绞作为主链路和 256K 的 DDN 作为备链路，实现了通讯线路及关键网络设备的冗余，较好地保证了公司业务的需要。并且随着网上交易系统的建设和网上办公的需要，公司企业网与互联网之间建起了桥梁。改造前，应用系统在用户认证及加密传输方面采取了相应措施，如集中交易在进行身份确认后信息采用了 Blowfish128 位加密技术，网上交易运用了对称加密和非对称加密相结合的方法进行身份认证和数据传输加密，但公司办公系统、交易系统、互联网应用之间没有进行安全隔离，只在互联网入口安装了软件放防火墙，给黑客的攻击、病毒的蔓延打开了方便之门。

作为公司信息技术中心运保部经理，系统安全一直是困扰着我的话题，特别是随着公司集中报盘系统、网上交易系统的建设，以及网上办公需要，网络安全系统的建设更显得犹为迫切。但公司考虑到目前证券市场疲软，竞争十分激烈，公司暂时不打算投入较大资金来建设安全系统。作为部门经理及项目负责人，我在投入较少资金的前提下，在公司可以容忍的风险级别和可以接受的成本之间作出取舍，充分利用现有的条件及成熟的技术，对公司网络进行了全面细致的规划，并且最大限度地发挥管理功效，尽可能全方位地提高公司的网络安全水平…在网络安全性和保密性方面，我采用了以下技术和策略：

- 1、将企业网划分成交易网、办公网、互联网应用网，进行网络隔离。
- 2、在网络边界采取防火墙、存/取控制、并口隔离等技术进行安全控制。
- 3、运用多版本的防病毒软件对系统交叉杀毒。
- 4、制定公司网络安全管理办法，进行网络安全集中管理。

一、网络安全隔离

为了达到网络互相不受影响，最好的办法是将网络进行隔离，网络隔离分为物理隔离和逻辑隔离，我主要是从系统的重要程度即安全等级考虑划分合理的网络安全边界，使不同安全级别的网络或信息媒介不能相互访问或有控制的进行访问。针对我公司的网络系统的应用特点把公司证券交易系统、业务办公系统之间进行逻辑分离，划分成交易子网和办公子网，将互联网应用与公司企业网之间进行物理隔离，形成独立的互联网应用子网。公司中心与各营业部之间建有两套网络，中心路由器是两台 CISC07206, 营业部是两台 CISC02612, 一条通讯链路是联通 2M 光纤，一条是电信 256KDDN, 改造前两套链路一主一备，为了充分利用网络

资源实现两条链路的均衡负载和线路故障的无缝切换，子网的划分采用 VLAN 技术，并将中心端和营业部端的路由器分别采用两组虚拟地址的 HSRP 技术，一组地址对应交易子网，一组地址对应办公网络，形成两个逻辑上独立的网络。改造后原来一机两用（需要同时访问两个网络信息）的工作站采用双硬盘网络隔离卡的方法，在确保隔离的前提下实现双网数据的安全交换。

二、网络边界安全控制

网络安全的需求一方面要保护网络不受破坏，另一方面要确保网络服务的可用性。将网络进行隔离后，为了能够满足网络内的授权用户对相关子网资源的访问，保证各业务不受影响，在各子网之间采取了不同的存取策略。

（1）互联网与交易子网之间：为了保证网上交易业务的顺利进行，互联网与交易子网之间建有通讯链路，为了保证交易网不受互联网影响，在互联网与中心的专线之间安装了 NETSCEEN 委托防火墙，并进行了以下控制：

- a)、只允许股民访问网上交易相应地址的相应端口。
- b)、只允许信息技术中心的维护机地址 PING、TELNET 委托机和路由器。
- c)、只允许行情发送机向行情主站上传行情的端口。
- d)、其他服务及端口全部禁止。并且在互联网和交易网之间还采用了 SSL 并口隔离，进一步保证了交易网的安全。

（2）交易网和办公网之间：对于办公网与交易网之间的互访，采用 CISCO2501 路由器进行双向控制或有限访问原则，使受控的子网或主机访问权限和信息流向能得到有效控制，主要采用的策略主要是对具体 IP 进行 IP 地址与 O2 地址的绑定。

（3）办公子网与互联网之间：采用东大 NE7EYE 硬件防火墙，并进行了以下控制：

- a) 允许中心上网的地址访问互联网的任何地址和任何端口。
- b) 允许股民访问网上交易备份地址的 8002 端口。
- c) 允许短消息访问公司邮件 110、25 端口，访问电信 SP 的 8001 端口。
- d) 其他的都禁止。

三、病毒防治

网络病毒往往令人防不胜防，尽管对网络进行网络隔离，但网络资源互防以及人为原因，病毒防治依然不可掉以轻心。因此，采用适当的措施防治病毒，是进一步提高网络安全的重要手段。我分别在不同子网上部署了能够统一分发、集中管理的熊猫卫士网络病毒软件，同

时购置单机版 KV3000 和瑞星防病毒软件进行交叉杀毒；限制共享目录及读写权限的使用；限制网上软件的下载和禁用盗版软件；软盘数据和邮件先查毒后使用等等。

四、集中网络安全管理

网络安全的保障不能仅仅依靠安全设备，更重要的是要制定一个全方位的安全策略，在全网范围内实现统一集中的安全管理。在网络安全改造完成后，我制订了公司网络安全管理办法，主要措施如下：

1)、多人负责原则，每一项与安全有关的活动，都必须有两人或多人在场，并且一人操作一人复核。

2)、任期有限原则，技术人员不定期地轮岗。

3)、职责分离原则，非本岗人员不得掌握用户、密码等关键信息。

4)、营业部进行网络改造的方案必须经过中心网络安全小组审批后方可实施。

5)、跨网互访须绑定 IP 及 MAC 地址，增加互访机器时须经过中心批准并进行存取控制设置后方可运行。

6)、及时升级系统软件补丁，关闭不用的服务和端口等等。

保障网络安全性与网络服务效率永远是一对矛盾，在计算机应用日益广泛的今天，要想网络系统安全可靠，势必会增加许多控制措施和安全设备，从而会或多或少的影响使用效率和使用方便性。如，我在互联网和交易网之间设置了放防火墙的前提下再进行了 SSL 并口隔离后，网上交易股民访问交易网的并发人数达到一定量时就会出现延时现象，为了保证股民交易及时快捷，我只好采用增加通讯机的办法来消除交易延时问题。

在进行网络改造后，我公司的网络安全级别大大提高。但我知道安全永远只是一个相对概念，随着计算机技术不断进步，有关网络安全的讨论也将是一个无休无止的话题。审视改造后的网络系统，我认为尽管我们在 Internet 的入口处部署了防火墙，有效阻挡了来自外部的攻击，并且将网络分成三个子网较减少了各系统之间的影响，但在公司内部的访问控制以及入侵检测等方面仍显不足，如果将来公司投资允许，我将在以下几方面加强：

1、在中心与营业部之间建立防火墙，通过访问控制防止通过内网的非法入侵。

2、中心与营业部之间的通讯，采用通过 IP 层加密构建证券公司虚拟专用网（VPN），保证证券公司总部与各营业部之间信息传输的机密性。

3、建立由入侵监测系统、网络扫描系统、系统扫描系统、信息审计系统、集中身份识别系统等构成的安全控制中心，作为公司网络监控预警系统。

范文 2:

摘要:

2015 年 3 月,我所在公司承接了一款养老管理信息平台,该系统以养老为主线,其中包括养老档案、维护计划、服务审计、费用管理等方面的 60 多个业务功能模块。我在该项目中担任系统架构师,主要负责整个系统的架构设计。本文以该养老管理信息平台为例,主要论述了针对该系统的安全及保密性问题。我们所采用的技术手段及解决方案,在网络硬件层通过设置防火墙来保证内部网络安全性;在数据层通过设置数据加密及容灾备份机制,以此保证数据抗突发风险的安全;在应用层统一采用 RBAC 的授权机制,来保证整个写认证环节的安全性。实时证明,以上技术方案的实现对整个系统的安全和保密方面起到了很好的效果,最终该项目顺序上线,运行稳定。收到了广大用户的一致好评。

项目背景介绍根据实际情况编制

经过前期对全国几十家养老机构和相关合作的医疗单位的调研分析,结合原有的经验,对整个系统业务进行规划和相关设备的初步选型。即我们内部所说的“两版三端”,机构养老和社区养老两个子系统,同时能够在 PC 端、PAD 端、手机端进行呈现和交互。整个养老管理信息平台基于 B/S 模式的分层架构风格设计,同时结合当前先进的基于租户理念搭建的一个养老云平台(SAAS)。当前信息安全问题严重,养老云平台是一个养老大型综合管理平台,确保系统的安全性和保密性尤为重要,让使用我们平台的机构、社区、老人及相关亲人都安全放心。这样只有通过最终用户的信任,系统的价值才会发展的更好,所以我们在系统的安全性和保密性做了充分的设计。

1、网络硬件层安全方案

首先,因为网络和硬件是整个系统运行的基础,也是很多外部共计的主要途径,所以,在这块需要进行合理规划,我们对网络拓扑结构划分为外部网络、内部网络和 DMZ 三部分。在外部网络和内部网络之间设置了硬件防火墙,主要是防止外部的恶意攻击,在防火墙后,为了加强对病毒入侵的防范,有设置了硬件的防毒墙,实时保证最新的病毒特征库保证有效的病毒工具。为了方式外部客户通过 DNS 服务直接访问服务的目标地址,在内部网络中有增加了反向代理服务器,对外只提供代理服务的虚拟 ip,更好的保护了应用服务器被攻击的可能性。应用服务器和数据连接服务器做了物理隔离,内部人员也无法通过 ip 直接访问,只能通过跳板机有服务器管理员进行操作。阻断了外界通过内部客户端代理的访问对服务器造成攻击。对于一些核心的 FTP 服务器、DNS 服务器、WEB 服务器都规划到 DMZ 中。

2、数据层安全方案

其次，数据是整个系统平台的核心，其中设计大量个人隐私数据和重要信息资产，安全不容忽视。主要从数据存储、数据访问和数据容灾几方面进行了规划，由于系统主要存放的养老档案数据，其中涉及很多个人隐私部分数据，为了防止泄露，我们在存储时进行了加密处理，虽然在处理中损失了性能，但是，提高了数据的安全性和保密性。我们此采用商业数据库 Oracle 作为后台存储，由于极佳的商业口碑，对数据存储的安全提供强大包装。为了方式数据信息的泄露，加强访问权限限制。在数据库的访问权限上，都根据不同角色进行详细划分。包括对数据库、表、索引、记录等增、删、改、查进行了限制，严格包装非法用户对数据的恶意破坏。同事在数据本身，也制定了基于全量、增量、差量的按周备份计划，包装每时每刻数据库都可以及时恢复。在物理存储上也做了本地多机房的容灾备份，充分保证数据在突发风险的情况下的安全。

3、应用层安全方案

最后，由于我们系统采用分层架构风格，同时，优势采用“三端”的形式，即手机端、PAD 端及 PC 端，所以，在用户认证、接口传输等方面做了充分设计。由于系统设计到养老机构工作人员、老人及家属等。角色众多，统一采用 RBAC 的授权机制，即增强系统安全性满足业务需求，同时，减轻由于权限信息维护负担。在登录界面设计上，不仅仅需要提供用户密码，同时设置了校验码，而且同时还增加了动态口令，通过短信接收验证码以此来防止非法破解登录，保证整个系统认证环节的安全性。在密码提交和存储环节，一律通过 MD5 加密，即使密码泄露也不会被解密。接口设计是整个系统通信的安全保障，为了方式外部人员恶意调用和窃取信息，我们采用了令牌机制保障每次通信安全性，即在登录后，通过加密传输用户信息和时间获取到 token 令牌，然后在后续每次通信传输过程服务队对 token 进行校验，充分保证了接口调用的安全。

整个项目历史 10 个月开发完成，到目前为止运行稳定，通过用户一段时间的使用反馈，不论是 PC 端还是 PAD 端、手机端，在任何环境下使用系统，基本没有信息泄露的情况发生。但是，一些用户反馈，手机端登录时间较长，查询和保存老人档案信息时候需要等待，经过分析，登录时间长，由于登录需要对客户端用户、密码进行客户加密处理，加密算法耗费了移动端 cpu 资源，同时在服务端也对数据进行加密处理返回客户端 token 也消耗了不少时间。老人档案也是由于敏感信息加密的问题，耗费 cpu 的处理时间，我们后面通过选择效率更高加密算法改善了这个问题。

实践证明，项目能够顺利上线，并运行稳定，使用良好。与系统的安全性和保密性方面设置密不可分。系统安全是一个永久的话题，我们对系统安全性的完善是一个持续的过程。我们接下来还会继续完善系统安全方面的设计缺陷和不足，使整个养老平台更加安全可靠。

三、论企业集成平台的理解与应用

1、真题原文：

论企业集成平台的理解与应用

企业信息集成是解决“孤岛”问题的需要，技术发展的同时也推动了集成架构等相关的研究。企业集成平台的核心是企业集成架构，包括信息、过程、应用集成的架构。

请围绕“论企业集成平台的理解与应用”论题，依次从以下三个方面进行论述。

- 1、概要叙述你所参与管理或开发的软件项目，以及你在其中所承担的主要工作。
- 2、给出至少 4 种企业集成平台应具有的基本功能，并对这 4 种功能内涵进行简述。
- 3、阐述你在软件开发的过程中都遇到了哪些实际问题及解决方法。

2、参考理论：

集成平台是支持企业集成的支撑环境，包括硬件软件，软件工具和系统，通过集成各种企业应用程序的多样性，企业信息系统的功能和环境都非常复杂，因此为了能够较好地满足企业的应用需求，作为企业应用集成支持环境的集成平台，应该具有以下的基本功能。

1) 通信服务

它提供分布环境下透明的同步异步通信服务功能，使用户和应用程序无需关心具体的操作系统和应用程序所处的网络物理位置，而以透明的函数调用或对象服务方式完成它们所需的通信服务要求。

2) 信息集成服务

它为应用提供透明的信息访问服务，通过实现异种数据库系统之间的数据的交换，互操作，分布数据管理和共享信息模型定义（或共享信息数据库的建立），使集成平台上运行的应用，服务或用户端能够以一致的语义和接口实现对数据（数据库，数据文件，应用交互信息）的访问与控制。

3) 应用集成服务

它通过高层应用编程接口来实现对相应应用程序的访问,这些高层应用编程接口包含在不同的应用程序。这些接口以函数或对象服务的方式向平台的组件模型提供信息,使用户在无需对原有系统进行修改的情况下,只要在原有系统上加上访问的接口就可以将现有的,用不同的技术实现的系统互联起来,通过为应用系统提供数据交换和访问操作,使各种不同的系统能够相互协作。

4) 二次开发工具

二次开发工具是集成平台提供的一组帮助用户开发特定应用程序(如实现数据转换的适配器或应用封装服务等)的支持工具,其目的是简化用户在企业集成平台实施过程中(特定应用程序接口)的开发工作。

5) 平台运行管理工具

它是企业集成平台的运行管理和控制模块,负责企业集成平台系统的静态和动态配置,集成平台应用运行管理和维护,事件管理和出错管理等,通过命名服务,目录服务,平台的动态和静态配置,以及其中的关键数据的定期备份等功能来维护整个服务平台的系统配置及稳定运行。

3、参考范文:

范文 1:

一 摘要

2019 年 8 月,我司承接了某市医疗集团,智慧药房项目,该项目主要为集团下属 36 家社区卫生服务中心提供药品统一目录管理、药品集中采购、库存管理、处方合理用药审核、药品配发、自动化发药设备驱动提供软件支撑。在该项目中我担任了软件架构设计师一职,主要负责该项目的软件架构设计工作。

本文以智慧药房项目为例,主要论述了**企业集成架构设计**在项目中的应用,在**处方配发模块**,采用了**数据集成**,将需要调配的处方通过接口进行应用集成实现配发业务。在**统计报表模块**中,采用了**界面集成**,将各业务子系统的报表进行了统一界面展示,在**配发和采购模块**中,采用了**数据集成**,使得两模块通过数据集成实现了库存信息的共享,通过使用这些**企业集成架构设计**,提高了软件的设计质量和开发效率,最终项目成功上线,并获得了用户一致好评。

一 背景

根据深化医药卫生体制改革规范药求，推进药品集中采购，增加药品的供应保障能力，严格监督管理，保障药品的用药安全，所有医疗机构开出的处方，必须通过处方审核后，方可进入划价计费环节，未经审核的处方，不得进行计费和配发。2019 年 8 月，我司承接了某医疗集团智慧药房项目，该项目为医疗集团下属 36 家社区卫生服务中心提供软件支持，主要分为药品供应模块、药房管理模块、处方用药审核模块、药品配发模块、设备协同模块。药品供应模块负责药品库存预警，供社区卫生服务中心药房对药品进行集中采购，并将药品发送到省采招平台；药房管理模块主要进行一些基础信息的维护，及关注药品的库存管理，药品批号跟踪；处方用药审核模块，负责对医生开出的药品进行合理用药审核，将不合理的处方审核结果返回给医生，提醒医生修改处方；药口中配发模块，负责将处方中的药品进行调配，打印用法用量标签，确认发药后扣减相应的库存；设备协同模块主要是驱动药房中的一些自动化发药设备，将要药品信息发给设备上位机，根据设备药品的效期进行排序，通过上位机程序调用下位机，驱动设备出药。在该项目中我担任软件架构设计师职务，主要负责软件的架构设计及中间件等技术的选型工作。

一问题回答，根据提干来

企业信息化集成主要由界面集成、应用集成、数据集成等方式。界面集成，通过 UI 将不同业务模块或系统的界面进行集成整合，使用户查看或使用系统时，不需要打开 N 多个系统进行操作。应用集成，一般指功能或 API 集成，它可以使不同厂家开发的系统通过接口整合后，实现互联互通的目的，使得原本不相容的两个或多个系统，可以互相协同工作。数据集成是以数据共享的方式对不同系统中模块中的数据进行整合，使之整合成一个完整的数据信息，供不同系统时实现数据共享。

一简介

本文将从界面集成、应用集成、数据集成三方向对智慧药房项目进行论述。

一以三个维度阐述

界面集成，在统计报表模块，因为药剂科主任需要看到 36 家社区卫生服务中心的药品库存、药品近效期、设备的使用状态，药品的发药量统计。由于这些数据存在不同的系统中，并且各自都有着自己维度的一些信息统计，正好覆盖了主任的期望需求，为了提高开发进程，我们将各模块的报表及统计信息进行了整合，通过菜单跳转的方式，对 UI 进行了集成，降低了业务系统之间的耦合度，同时降底了开发成本。

应用集成，可以使不同模块的功能应用通过接口的方式进行有效整合，项目中有多个模块，在和 HIS 信息系统对接中，因为要对接多家 HIS 信息系统，每家 HIS 用的系统开发语言不同，有 Delphi，有 JAVA 有基于 C#语言给系统集成增加了复杂度，因此我们选择了基于 ESB 企业总线的集成方式，将不同的语言开发的软件都以 WebService 注册到服务中心，通过 ESB 进行传输协议及消息协议和格式的转换。最后以统一的 XML 格式传入我们的系统，实现了不同系统之间的业务对接，在我们内部系统中，采用了基于 Restful 风格的接口形式，对各个系统以 JSON 格式进行数据交互，并且在 ESB 中进行了用户授权，防止非法用户接入集成平台获取数据，设备协同则通过 MQ 消息中间件的形式进行数据获取，实现了业务的解耦。

数据集成，以数据共享的方式对不同应用模块中的数据进行整合共享，在药品采购模块中，由于药品的采购量需要通过当前的库存及药品销量数据，通过算法计算出相应的采购数据进行采购，药品销量数据及当前库存信息分别存在不同的数据库中有 MongoDB、Redis，销量数据存在 MongoDB 中，是因为每天的发药数据量较大，MongoDB 具备处理大数据的优势，而且非关系型数据不受字段的约束。对药品的库存查询更新频率较高，所以放在了 Redis 中，避免高并发给关系型数据库带来负载。采购模块则通过 MongoDB 及 Redis 中的数据获取，生成采购计划，并通过接口的方式将采购订单传到省采招平台，通过数据对接的方式将不同系统中的数据进行共享，减少了接口开发量，提高了集成效率。

— 结尾

项目从 2019 年 8 月启动到 2020 年 10 月历时 14 个月，圆满完成，顺利完成验收，并取得了客户的一致好评，该项目运行一年多，也出现过一些小的问题，由于医院使用的是集团内部局域网，与外网隔离，给排查问题及维护增加了困难，一次发药设备上的上位机不小心被药房老师删除后，不会重新安装操作，上报我司后，我们联系医院信息科老师，在信息科的老师协助下，对软件重新进行了安装，此次事件，导致了设备停止工作了几个小时，影响了药房发药效率。后来我司安排了 1 名售后维护该项目，这一年内也新增了一些发药设备，由于选用了合适的架构风格，使得设备的接入及服务的扩展变得非常容易，在售后同事的协助下，系统至今运行稳定。该项目的成功，让我意识到了使用了企业集成的作用和价值，坚定了我对企业集成技术应用的信心，合理选择合适的企业集成方式，能够大大的提高了软件设计的复用方法，加快开发的进程，在项目中起到事半功倍的作用。经过这次项目，我也看到了自己身上的不足之处，在未来还会不断地更新知识，完善本系

统的设计，使系统能够适应国家医改的变化需要，这是作为软件从业的我为之努力的动力和方向。

范文 2:

[摘要]

2004 年 10 月，我参加了某车站综合信息平台项目的开发，承担项目的方案设计任务。该项目力图通过对车站现有信息子系统的集成，以达到共事各子系统的数据库，优化企业运输作业流程，提高企业经营管理水平之目的。

本文结合笔者的实践，以该综合信息平台建设项目为例，讨论了企业应用集成技术。在本着集成、开放标准、管理配套的原则下，提出了基于 Java 技术的 J2EE 应用服务器作为统一的应用集成平台，以集成适配器作为系统集成架构模式的总体设计思路，并着力介绍了该项目关键部件——集成适配器的构建过程。还就项目的具体实施作了详细叙述。最后，提出了企业应用集成的持续性，并确定了下一步集成的目标。

[正文]

2004 年 10 月，我单位承接了某车站综合信息平台的建设任务。由于我具有多年的铁路行业软件项目开发经验，所以我有幸被单位指定为该项目的负责人，主要负责项目的方案设计工作。

该车站是一等客货运编组站，车站所在地是矿区，有规模不等的国有煤矿及个体煤矿数十个，车站主要以煤炭运输为主。近些年来，随着铁路 TMIS 系统（铁路运输管理信息系统）建设的逐步深入，该车站建立了若干相应的应用子系统，主要有列车确报系统、车站现车系统、货票制票系统、车号红外线自动识别系统、货运计划系统、货运安全系统及货车轨道衡计重系统等。车站希望通过综合信息平台的建设达到以下几个目标：

（1）实现各子系统间数据共享。

（2）能够实时地向企业客户（货主）发布请车、承认车及货物运价调整等相关信息。

（3）通过各子系统的应用集成，使车站运输作业流程得以优化。在设计综合信息平台建设总体方案时，我充分考虑了下面三个原则：

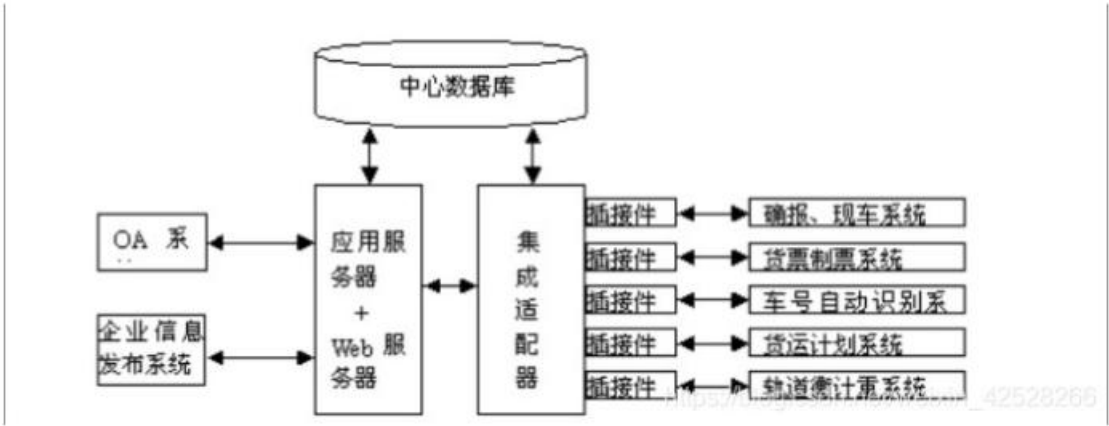
集成原则：综合信息平台最主要的目标是整合车站的信息资源，在考虑最大化集成各个信息子系统时，应该避免产生新的“信息孤岛”。

开放标准原则：在综合信息平台建设时，应该站在整个系统宏观的高度，采用开放的标

准和统一的架构来集成各信息子系统，避免各子系统“点到点”的低效落后的集成方式，以利于将来其他新的系统能够便利、无缝的整合。

管理配套原则：建设综合信息平台的目的是通过信息共事，达到业务流程的优化，以提高车站各方面整体管理水平。因此项目设计之时，应充分考虑企业管理方面的需求。

在遵循上述原则的基础上，经过对企业需求的认真分析，结合当今成熟的 EAI 技术，我提出了以基于 Java 技术的 J2EE 应用服务器作为统一的应用集成平台，以集成适配器作为系统集成架构模式的总体设计方案。如下图所示：



本设计方案从集成的广度来说，既包括了数据的集成，也包括了应用的集成；在集成的方法论方面来讲，大部分系统采用可白盒集成方法，少数系统采用的是黑盒集成方法。根据综合信息平台的总体设计方案，集成适配器是该信息平台的最关键部分，它负责不同子系统之间数据的采集、转换和交流。因此，集成适配器的选型或设计的合理与否，是项目成败的关键。由于车站各信息子系统存在操作系统平台和数据库的异构性，无法从现有的中间件中找到完全适合的产品，因此我们决定自行开发此中间件。由于铁路各车站的业务领域和业务流程存在高度相似性，所以，此中间件有较高的复用价值。

对于集成适配器的设计，我采用了一种“可插拔”的设计理念。即为每个需要集成的子系统单独设计一个插接件，该插接件负责为号之相连的子系统提供数据及应用接口。各插接件通过 XML 格式的装配文件，自由组装到集成适配器这个容器中。集成适配器为所有插接件提供一个统一的调度模块，来协调和指挥所有插件，使之能够协同运作。

在集成适配器的开发中，我选用了开源的集成开发环境 Eclipse 作为开发平台。对于集成适配器调度模块的开发，我们采用了 Eclipse 提供的 JobsAPI,JobsAPI 封装了 JDK (JavaDevelopmentKitTools) 的定时及同步方面底层 API,降低了编程的复杂性，提高了开发效率。

在插接件的开发过程中，我们充分利用 Eclipse 的高度可扩展特性，在因特网上搜集该项目可用的插件，以这些插件为扩展点，来扩展我们自己的插接件。最后我们利用 Eclipse 提供的 RCP (RichClientPlatform) 技术，集成我们的功能插件，并生成独立于 Eclipse 平台的、可独立执行的集成适配器。

各信息子系统具体集成过程及效果如下所述：

(1) 轨道衡计重系统

该车站的轨道衡设在矿区号车站之间，距车站站场大约 2 公里左右，从矿区到车站的重车要经过轨道衡检斤，对于超重货车，要通知车站相关部门处理。由于该系统是个独立的单机系统，计重软件采用的是 VFP 编制，数据库是单机的 DBF。因此，我们首先从网络方面进行了集成：利用一对网桥将该系统接入车站 TMIS 系统。重车通过轨道衡会产生结构为：(车辆顺位、车号、车种、铁重、计重) 数据，号轨道衡相连的插件会通 JDBC-ODK 桥，将数据转换为两个不同格式的副本，一份写入中心数据库，一份作为矿区站发来的确报报文，写入确报系统的到确报库中。

通过该系统的集成，轨道衡工作人员免去了每天去车站传送过衡报表的工作，车站相关部门可随时通过浏览器查询过衡数据，并能方便的生成各种统计数据；车站的车号员通过查收确报报文，即可掌握过衡列车的组成内容，减去了每列车都要到现场抄车号，回去录入的工作流程，大大减轻了劳动强度，提高了工作效率。

(2) 货票制票系统

该系统的集成我们采用了批量文件传输 (FTP) 方式。货票制票机在制票完成后，会自动向分局传输货票数据。因此我们在货票传输地址表中增加一个条目，使货票数据在传送分局的同时也传送到集成适配器，号之相应的插件将到达的 80 列格式货票数据进行解析，形成结构为：(票号、车号、车种、计重、运价、发货人、收货人) 的数据，利用 JDBC，一份写入中心数据库，另一份通过车号与现车系统的车辆库匹配，写入相应信息。

通过该系统的集成，减轻了编制出发列车确报的工作重，因为输入车号后，号该车号相关的货票信息会自动生成，免去了去查阅货票步骤。另外，该系统的集成，使货主通过因特网查询货票信息成为了可能。其他的信息子系统如确报、现车系统、货运计划系统、车号识别系统等等，也都通过各自的接口插件进行了数据或应用的集成，在此不再详述。

(3) OA 系统和企业信息发布平台

通过 TMIS 各子系统集成而建立的中心数据库，是办公自动化和企业信息发布平台的数

据基础。通过设在货服大厅的大屏幕显示系统,能够实时动态地发布号货运业务相关的系统,方便了货主,提高了办公的透明度。通过项目组成员努力工作,车站高层领导的高度重视,以及车站相关人员的通力合作,历时四个月,该车站的综合信息平台初步完成,达到了建设之初的需求目标,得到车站的一致好评。系统到目前为止运行稳定。

我们应该知道,企业应用集成是一个持续集成过程,是一项长期、不断进行的工程,不能指望短时间内达到深度集成。随着铁路信息化建设的深入,还会有新的应用需要整合。我们下一步的目标是建立该企业的企业门户,使我们前期的后台整合成果,能够在互联网+得以展现。

四、论微服务架构及其应用

1、真题原文:

论微服务架构及其应用

微服务提倡将单一应用程序划分成一组小的服务,服务之间互相协调、互相配合,为用户提供最终价值。每个服务运行在其独立的进程中,服务与服务间采用轻量级的通信机制互相沟通。在微服务架构中,每个服务都是一个相对独立的个体,每个服务都可以选择适合于自身的技术来实现。每个服务的部署都是独立的,这样就可以更快地对特定部分的代码进行部署。

请围绕“论微服务架构及其应用”论题,依次从以下三个方面进行论述。

- 1、概要叙述你所参与管理或开发的软件项目,以及你在其中所承担的主要工作。
- 2、简要描述微服务优点。
- 3、具体阐述如何基于微服务架构进行软件设计实现的。

2、参考理论:

微服务之所以能够盛行,必然是有它的独特的优势的,下面我们来分析微服务的优势。

1) 技术异构性

在微服务架构中,每个服务都是一个相对独立的个体,每个服务都可以选择合适于自身的技术来实现。如要开发一个社交平台,此时我们可能使用文档型数据库来存储帖子的内容,使用图数据来存储朋友圈的这些关系,可以将每一块的性能都充分发挥出来。

同时在应用新技术时，微服务架构也提供了更好的平台。因为对于单块的系统而言，采用一个新的语言，数据库或者框架都会对整个系统产生非常巨大的影响，这样就导致我们在尝试新技术的时候望而却步。但是微服务不同，我们完全可以只在一个微服务中采用新技术，然后等成熟之后再推广到其他的服务当中。

2) 弹性

系统中的一部分如果出现了故障会引起多大问题。在单块系统中一个部分出现问题，可能导致整个系统的问题。而微服务架构中，每个服务可以内置可用性的解决方案与功能降级方案，所以比单块系统强大。

3) 扩展

在单块的系统中我们如果要对系统进行扩展的话，必须是整体的进行扩展，而使用微服务的架构中，可以针对单个服务进行扩展。

4) 简化部署

对于大型单块的系统，哪怕是修改了一行代码，都要对其进行重新的整体的部署。这种部署影响很大，风险高因此不敢轻易重新部署。而在微服务中，每个服务都是独立部署的，而且还可以实现自动化的部署，这样就可以更快的对特定部分的代码进行部署。

5) 与组织结构相匹配

对于传统的单块系统来说，系统越大代码库越大，则越难管理，而且还会出现一系列管理方面的问题。体会团队是分布式（虚拟团队）的时候，那么管理的复杂度将会更高。而微服务的出现就很好的解决了这个问题。微服务架构可以将架构与组织结构相匹配，避免的了出现过大的代码库，从而获得理想团队大小和生产力。服务的所有权也可以在团队之间迁移，从而避免异地团队的出现。

6) 可组合性

在微服务架构中，系统会开放很多的接口供外部使用。当情况发生改变时，可以使用不同的方式构建应用，而整体化应用程序只能提供一个非常粗粒度的接口供外部使用。

7) 对可替代性的优化

在单块系统中如果删除系统中的上百行代码，也许不知道会发生什么，引起什么样的问题，因为单块系统中关联性很强。但在微服务架构中，我们可以在需要时轻易的重写服务，或者删除不再使用的服务。

3、参考范文：

范文 1：

一摘要

2019 年 8 月，我司承接了某市医疗集团，智慧药房项目，该项目主要为集团下属 36 家社区卫生服务中心提供药品统一目录管理、药品集中采购、库存管理、处方合理用药审核、药品配发、自动化发药设备驱动提供软件支撑。在该项目中我担任了软件架构设计师一职，主要负责该项目的软件架构设计工作。本文以智慧药房项目为例，主要论述了**微服务架构**在项目中的具体应用，**该项目主经由药品供应管理、处方审核、药品配发、设备协同及药房管理等模块组成，子模块之的业务相对单一，因为采用了微服务架构设计，将这些模块进行了微服务设计开发及部署，使他们在后期迭代过程中，开发及发布不会影响到其它的模块，通过使用微服务架构设计，提高了软件设计质量和开发效率，最终项目成功上线，并获得了用户一致好评。**

一项目背景

根据深化医药卫生体制改革规范药求，推进药品集中采购，增加药品的供应保障能力，严格监督管理，保障药品的用药安全，所有医疗机构开出的处方，必须通过处方审核后，方可进入计价计费环节，未经审核的处方，不得进行计费和配发。2019 年 8 月，我司承接了某医疗集团智慧药房项目，该项目为医疗集团下属 36 家社区卫生服务中心提供软件支持，主要分为药品供应模块、药房管理模块、处方用药审核模块、药品配发模块、设备协同模块。药品供应模块负责药品库存预警，供社区卫生服务中心药房对药品进行集中采购，并将药品发送到省采招平台；药房管理模块主要进行一些基础信息的维护，及关注药品的库存管理，药品批号跟踪；处方用药审核模块，负责对医生开出的药品进行合理用药审核，将不合理的处方审核结果返回给医生，提醒医生修改处方；药口中配发模块，负责将处方中的药品进行调配，打印用法用量标签，确认发药后扣减相应的库存；设备协同模块主要是驱动药房中的一些自动化发药设备，将要药品信息发给设备上位机，根据设备药品的效期进行排序，通过上位机程序调用下位机，驱动设备出药。在该项目中我担任软件架构设计师职务，主要负责软件的架构设计及中间件等技术的选型工作。

一就当回答面试题

传统的单体软件架构在构建部署和扩展方面有很大的局限性，耦合度比较高，更新升级，都需要整体进行，随着软件规模的增大，业务模块越来越多，分布式越来越复杂，使得软件的构建变得越来越复杂，对开发人员的要求也越来越高，微服务是一种架构风格，根据业务领域驱动分析，以垂直划分的方式将一个复杂的应用拆分成多个独立自治的服务，服务与服务之间通过松耦合的形式进行交互，每个微服务都是单一的职责，以 SOA 的核心思想，以服务的形式对外提供服务，配合 DevOps 的一些组件，CI/CD 持续集成每日构建，升级、扩容不中断业务

一简单介绍一下凑字数，后面分了三块介绍，主要是前后台分离，怎么部署的；代码层面是怎么垂直划分实现的

智慧药房项目中，将药品 SPD 采购入库模块、处方审核模块、药品配发模块、设备驱动模块等模块进行了微服务的划分，同时研发小组划分成了 4 个小组，每个小组负责模块软件的生命周期开发工作，从开发、集成、测试，再到微服务的发布升级。服务与服务之间通过 restful 接口进行控制集成。

前端表现层：由于智慧药房项目，有设备硬件，所以前端表现层，采用了 B/S+C/S 的混合架构风格，在药品采购入库模块中，采用了 B/S 的形式，将前端 Web，通过 Nginx 进行集群部署，当用户通过浏览器访问 Web 系统时，通过 Nginx 轮训加权算法，进行 Web 服务的访问分配。通过 Ajax 方式调用 平台服务层的 Restful API 接口。设备协同模块，为了方便给下位机发送命令驱动设备，因此设备上位机采用的 C/S 架构，通过 HttpClient 方式调用平台服务的 Restful 接口。

平台服务层：采用比较成熟的 SpringCloud 框架进行搭建，为表现层提供接口服务。将 SPD 采购入库模块、处方审核模块、药品配发模块、设备驱动模块等模块接口启动上线后通过 Nacos 注册中心进行注册，负责服务的注册、发现。通过 Gateway 网关将所有的服务地址暴露在外，对外进行接口统一管理，并且在网关层，做了统一的安全认证，当后端有多个服务时，通过服务命名管理，由 Ribbon 负责负载均衡的实现，当请求过多时，通过 Ribbon 路由到指定的服务端，通过 Skywalking 进行接口链路的监控，确保接口服务对外的可用性。服务应用通过编写 Dockerfile，通过 Docker Compose 将应用部署在 Docker 上，提高了应用发布的效率，降低 DevOps 的复杂度

业务服务：按采购入库模块、处方审核模块、药品配发模块、设备驱动模块等业务分了多个微服务。各服务之间的调用通过 Openfeign 进行 RPC 的动态调用。接口以 Restful

的形式进行交互，处方审核接口服务调用频率远高于其它服务。并且及时性要求比较高，因为为审方服务提供了五个实例节点做集群服务，结合负载均衡解决服务端的请求压力，审方合理用药规则有 18 万个药品目录，数据量较大，在做处方审核时对数据查询的要求比较高，因此引入了 Mongoddb 组件，MongoDB 适用于大数据的存储和分析。同时在设备协同模块中，引用了 Kafka MQ 消息中心件，将设备和业务系统进行解耦，配发模块将需要出药的处方数据，发给 Kafka，设备收到 Kafka 消息后，提取消息中的药品数据，进行效期和库存的排序，驱动下位机出药。

一、结尾

项目于 2019 年 8 月启动到 2020 年 10 月历时 14 个月，圆满完成，顺利完成验收，并取得了客户的一致好评，该项目运行一年多，也出现过一些小的问题，由于医院使用的是集团内部局域网，与外网隔离，给排查问题及维护增加了困难，一次发药设备上的上位机不小心被药房老师删除后，不会重新安装操作，上报我司后，我们联系医院信息科老师，在信息科的老师协助下，对软件重新进行了安装，此次事件，导致了设备停止工作了几个小时，影响了药房发药效率。后来我司安排了 1 名售后维护该项目，这一年内也新增了一些发药设备，由于选用了合适的微服务架构，使得设备的接入及服务的扩展变得非常容易，在售后同事的协助下，系统至今运行稳定。该项目的成功，让我意识到了使用了**微服务架构**的作用和价值，坚定了我对**微服务架构**应用的信心，合理选择合适的**微服务架构**，能够大大的提高了软件设计的复用方法，加快开发的进程，在项目中起到事半功倍的作用。经过这次项目，我也看到了自己身上的不足之处，在未来还会不断地更新知识，完善本系统的设计，使系统能够适应国家医改的变化需要，这是作为软件从业的我为之努力的动力和方向。

范文 2:

摘要

2016 年 7 月，我所在的公司为***系统的产品，我担任系统架构师职务，主要负责软件架构和安全体系设计的工作，该项目是基于互联网，为单位、企业和个人等公众群体提供 7*24 小时的查询申请服务，同时兼顾行***宣传。本文结合作者的实践，以行贿犯罪档案互联网查询系统为例，论述微服务架构及其应用。首先概述我参与管理和开发，并采用微服务

架构开发的工作，然后具体描述微服务架构的特点，最后结合项目描述软件的架构，说明该系统是如何采用微服务架构模式的，并说明采用微服务架构模式后，在软件开发过程中遇到的实际问题和解决方案。经过项目组近一年的努力，本产品已顺利开发完成，目前，已在***等多省上线使用，取得客户和公司领导的一致好评。

正文

近年来，随着互联网行业的迅猛发展，公司或组织业务的不断扩张，需求的快速变化以及用户量的不断增加，传统的单块（Monolithic）软件架构面临着越来越多的挑战，已逐渐无法适应互联网时代对软件的要求。在这一背景下，微服务架构模式（MicroserviceArchitecturePattern）逐渐流行。它强调将单一业务功能开发成微服务的形式，每个微服务运行在一个进程中；采用 HTTP 等通信协议和轻量级 API 实现微服务之间的协作与通信。这些微服务可以使用不同的开发语言以及不同数据存储技术，能够通过自动化部署工具独立发布，并保持最低限制的集中式管理。2015 年 7 月，我所在的公司为***的产品，我担任系统架构师职务，主要负责软件架构和安全体系设计的工作。本文结合作者的实践，论述微服务架构及其应用。首先概述我参与管理和开发，并采用微服务架构开发的工作，然后具体描述微服务架构的特点，最后结合项目描述软件的架构，说明该架构是如何采用微服务架构模式的，并说明采用微服务架构模式后，在软件开发过程中遇到的实际问题和解决方案。

项目概述根据实际情况编制。微服务的目的是充分地分解应用程序以促进敏捷开发和部署。在***系统项目的管理和开发中，我们按功能需求将系统划分为用户中心、查询申请、数据交换、预约排号、知识问答 5 个微服务，同时将项目团队划分为 3 个小组，根据功能的轻重缓急和工作量，安排各个微服务的研发。每个小组负责一个或多个组件完整的生命周期，即服务谁开发，服务谁运营。最后各个服务组件通过 RESTfulHTTP 协议和消息路由功能进行服务组装。

微服务架构的特点

传统的单块软件架构在构建部署和扩展伸缩方面有很大的局限性，传统的单块架构一般分为客户端用户界面、数据库、服务端应用程序三部分。系统中任何程序的改变都需要整个应用重新构建和部署新版本。另外传统的单块软件架构在进行水平扩展时也只能整个系统扩展，而不能针对某一个功能模块进行扩展。而微服务架构可以完美的解决统一风格架构所遇到的种种问题。微服务架构将系统以组件化的方式分解为多个服务，服务之间相对独立，单

一功能的改变只需要重新构建部署相应的服务即可。与单块架构相比，微服务架构有如下的特点：

- 1) 通过服务实现应用的组件化(ComponentizationViaServices)，在应用架构设计中，通过将整体应用切分成可独立部署及升级的微服务方式，进行组件化设计。
- 2) 围绕业务能力组织服务(OrganizedAroundBusinessCapabilities)，微服务架构采取以业务能力为出发点组织服务的策略，因此微服务团队的组织结构必须是跨功能的、强搭配的 DevOps 开发运维一体化团队，通常这些团队不会太大。
- 3) 基础设施自动化(InfrastructureAutomation)，云化及自动化部署等技术极大地降低了微服务构建、部署和运维的难度，通过应用持续集成和持续交付等方法有助于达到加速推出市场的目的。
- 4) 故障处理设计(DesignForFailure)，微服务架构所带来的一个后果是必须考虑每个服务的失败容错机制。因此微服务非常重视建立架构及业务相关指标的实时监控和日志机制。
- 5) 演进式的设计(EvolutionaryDesign)，微服务应用更注重快速更新，因此系统的计会随时间不断变化及演进。

微服务架构应用

在***系统中，采用通讯层、业务逻辑层、基础服务层组成系统的微服务架构。***系统微服务框架在微服务架构下，我们选择 RESTful 的进行通讯。每个微服务都统一对外提供 REST 服务。无论前端调用后端服务还是后端之间的服务调用都统一走 RESTful，这样就统一了协议栈。微服务架构可以支持各种异构系统服务间的交互。服务的注册与发现，服务之间需要创建一种服务发现机制，用于帮助服务之间互相感知彼此的存在。服务启动时会将自己的服务信息注册到注册中心，并订阅自己需要消费的服务。负载均衡是服务高可用的保证手段，为了保证高可用，每一个微服务都需要部署多个服务实例来提供服务。我们主要支持随机、轮询、最少链接数的策略将来自网络的请求随机分配给内部中的多个服务器。

目前主流的 RPC 框架包括 dubbo, thrift, grpc 等，我们采用 dubbo 为团队提供了一整套序列化，反序列化，网络框架，连接池，线程池超时处理等业务处理之外的能力。服务容错采用快速失败，失效切换的策略。如果连续失败多次则直接熔断，不再发起调用。这样可以避免一个服务异常拖垮所有依赖于他的服务。

遇到的问题及解决方案

在微服务实践中，我们主要遇到三个问题，一运维开销及成本增加，因为每个微服务需

独立运行，还可能采用多种语言环境，这将导致资源开销大；二代部分码重复，某些底层功能需要被多个服务所用；第三个问题是难以可视化及全面测试，在动态环境下服务间的交互会产生非常微妙的行为。因此，首先服务划分应尽量合理，不要划分太细太多，其次采用公共模块的方式提供底层服务，再次微服务可通过监控发现生产环境的异常，进而快速回滚，弥补可测性不足的问题。

通过项目实践证明，实施微服务架构收益会大于成本。在拥抱微服务之前，也需要认清它所带来的挑战。需要避免为了“微服务”而“微服务”。对传统企业而言，开始时可以考虑引入部分合适的微服务架构原则，对已有系统进行改造或新建微服务应用，逐步探索及积累微服务架构经验，而非全盘实施微服务架构。

范文 3:

1. 摘要

去年十月，我所在的公司组织了某部委工具中心项目的开发。该项目是以工具为单元对自动化运维工具集合进行管理的应用。工具中心为两级部署，一级工具中心作为其统一工具仓库，是各二级工具中心通用工具权威源，负责其通用工具的准入、全生命周期管理；二级工具中心为各省（市）工具仓库，存放来自一级工具中心的通用工具，负责各省（市）自建工具的准入、全生命周期管理，并作为工具最终运行平台，对外提供工具调用及监控服务。

在该项目中，我担任了架构师和项目负责人的职位，负责项目的架构评审、改造设计及实现。本文结合作者的实践，以工具中心为例，论述微服务架构及其应用。首先概述我参与管理和开发，并采用微服务架构开发的工作，然后具体描述微服务架构的特点，最后结合项目描述软件的架构，说明该系统是如何采用微服务架构模式的，并说明采用微服务架构模式后，在软件开发过程中遇到的实际问题和解决方案。经过项目组近一年的努力，本产品已顺利开发完成，目前已上线使用，取得客户和公司领导的一致好评。

2. 正文

近年来，随着互联网行业的迅猛发展，公司或组织业务的不断扩张，需求的快速变化以及用户量的不断增加，传统的单块（Monolithic）软件架构面临着越来越多的挑战，已逐渐无法适应互联网时代对软件的要求。在这一背景下，微服务架构模式（Microservice Architecture Pattern）逐渐流行。它强调将单一业务功能开发成微服务的形式，每个微服务运行在一个进程中；采用 HTTP 等通信协议和轻量级 API 实现微服务之间的协作与通信。这些微服务可以使用不同的开发语言以及不同数据存储技术，能够通过自动化部署工具独立

发布，并保持最低限制的集中式管理。

微服务的基本思想在于考虑围绕着业务领域组件来创建应用，这些就应用可独立地进行开发、管理和加速。在分散的组件中使用微服务云架构和平台使部署、管理和服务功能交付变得更加简单。微服务是利用组织的服务投资组合，然后基于业务领域功能分解它们，在看到服务投资组合之前，它还是一个业务领域。

微服务这一概念出现于 2012 年，是因软件作者 Martin Fowler 而流行，他承认这并没有精确地定义出这一架构形式，虽然围绕业务能力、自动化部署、终端智能以及语言和数据的分散控制有一些常见的特性。

(1) 项目概述

随着某部委信息化进程的不断推进，对信息通信运行保障能力也提出了更高的要求。运维过程中通过借助于运维工具实现运维的自动化管理，由于当前在运的运维工具数量快速增长，各单位都有自己的运维工具，为了更好的利用好这些运维工具，建立统一的工具中心管理系统，管理和共享这些运维工具已经迫在眉睫。

为了解决上述问题，我们设计和开发了工具中心项目，该项目主要通过本地管理和共享管理等业务流程来实现全网自动化运维工具的管理：

1. 本地管理负责省公司二级库这边工具的上传，发布审批，升级，部署，取消发布，删除等业务处理。
2. 共享管理完成从省公司二级库把工具上传到总部一级库的过程，总部审批通过后共享给其他省分。

为了实现上述业务目标，完成业务功能，工具中心通过工具商店、在运工具和本地库管理三个微服务来实现。同时将项目团队划分为 2 个小组，根据功能的轻重缓急和工作量，安排各个微服务的研发。每个小组负责一个或多个组件完整的生命周期，最后各个服务组件通过 RESTful HTTP 协议进行服务组装。

(2) 微服务架构的特点

传统的单块软件架构在构建部署和扩展伸缩方面有很大的局限性，传统的单块架构一般分为客户端用户界面、服务端应用程序和数据库三部分。系统中任何程序的改变都需要整个应用重新构建和部署新版本。另外传统的单块软件架构在进行水平扩展时也只能整个系统扩展，而不能针对某一个功能模块进行扩展。而微服务架构可以完美的解决统一风格架构所遇到的种种问题。微服务架构将系统以组件化的方式分解为多个服务，服务之间相对独立，单

缓存技术，读写分离技术和数据分片技术都可以起到提高性能，缓解单个物理节点的压力。

缓存

工作中比较常用的如 **Redis** 用来缓解数据库的压力。将热点数据预热到缓冲中，避免大量的访问压力直接给到数据库上面，给数据库减轻负担。

读写分离

配置实现主从数据库，将请求分为读/写两种类型，读请求走从库(slave)，写类型请求走主库(master)。比如 MySQL 自身提供的主从数据同步方案。主从库之存在较低(可接受范围)的数据同步延迟。

数据分片

如果单表/单库存在数据保存的性能问题，可使用分片将保存的数据分散到多个库表中，其中分为水平分片和垂直分片。

水平分片

统一类型的数据，分别放到不同的库/表中。每个分片包含了整体的数据集合的一部分。虽然可减轻单节点的访问压力，又迎来了分布式事务的问题。

垂直分片

存在一个宽表(即包含过多字段的表)，其中某几个字段属于热点数据，客户端请求某一条记录，大部分情况下都是要获取这条记录中的某几个热点字段。这个时候，将这张表拆分为主表和从表两张表，热点数据单独成表(从表)，这样数据访问/更新会避免在宽表上的大量操作。提前合计好对应的主从表。按照不同的业务模块拆分数据库，这样可以适当的减少单个服务器的压力。

三种数据分片方式:hash 方式,一致性哈希(consistenthash),按照数据范围(rangebased)

哈希分片

概念: 按照数据的某一特征来计算哈希值，并将哈希值与系统中的节点建立映射关系。

优点: 简单易于实现

缺点: 很难解决数据不均衡问题，再增加一个机器，每个机器对应的一个 hash 值的区域就发生改变。

补充:

假设这里面是按员工的薪水进行计算 hash 值，实际人群中，可能处于平均薪水 10k 左右的人比较多，高薪水的人比较少这导致某些机器上的数据很大，导致大量的数据集中到一

个物理节点上。

一致性哈希

概念：一致性 hash 相当与一个环。所有的数据都在这个环上，每个机器相当于环的一截，相比于上述的 hash 方式，一致性 hash 方式需要维护的元数据额外包含了节点在环上的位置。

优点：简单易于实现，在增删数据的时候只会影响到 hash 环上相邻的节点，不会发生大规模的数据迁移。

缺点：增加节点的时候，只能分摊一个已存在节点的压力

补充：

在实际工程中，一般会引入虚拟节点（virtualnode）的概念。即不是将物理节点映射在 hash 环上，而是将虚拟节点映射到 hash 环上。虚拟节点的数目远大于物理节点，因此一个物理节点需要负责多个虚拟节点的真实存储。操作数据的时候，先通过 hash 环找到对应的虚拟节点，再通过虚拟节点与物理节点的映射关系找到对应的物理节点。

按照数据范围

概念：就是按照关键值划分成不同的区间，每个物理节点负责一个或者多个区间。其实这种方式跟一致性 hash 有点像，可以理解为物理节点在 hash 环上的位置是动态变化的。

优点：

当达到这个阈值之后就会分裂成两个块。这样做的目的在于当有节点加入的时候，可以快速达到均衡的目的

缺点：

在数据可修改的情况下，如果块进行分裂，那么元数据中的区间信息也需要同步修改。

补充：

rangebased 这种数据分片方式应用非常广泛，比如 MongoDB, PostgreSQL, HDFS

比较

如果一个节点负责的数据只有一个区间，rangebased 与没有虚拟节点概念的一致性 hash 很类似；如果一个节点负责多个区间，rangebased 与有虚拟节点概念的一致性 hash 很

3、参考范文：

二、论云原生架构及其应用

1、真题原文：

论云原生架构及其应用论题，依次从以下三个方面进行论述：

- 1. 概要叙述你参与管理和开发的软件项目以及承担的主要工作。
- 2. 服务化、强性、可观测性和自动化是云原生架构重复的四类设计原则，请简要对这四类设计原则的内涵进行阐述。
- 3. 具体阐述你参与管理和开发的项目是如何向采用云原生架构的，并且围绕上述四类设计原则详细论述在项目设计与实现过程中遇到了哪些实际问题，是如何解决的。

2、参考理论：

云原生是一种方法，用于构建和运行充分利用云计算模型优势的应用。云计算不再将重点放在资本投资和员工上来运行企业数据中心，而是提供无限制的按需计算能力和根据使用情况付费的功能，从而重新定义了几乎所有行业的竞争格局。



器，负载均衡服务器会将请求转发到前端网站的 Nginx 集群，前端网站通过发起 http 请求来和后端交互，具体是通过 Ajax 方式来调用后端 RESTAPI 接口。用户访问网站通过前置的 Nginx 负载均衡服务器来转发到前端网站集群，以起到将用户请求进行分流的作用。当前端网站集群中的部分服务发生故障时，系统仍可正常地对外提供服务。前置 Nginx 负载均衡服务器使用软件反向代理的方式来实现负载均衡，部署为路由模式，系统内部网络与外部网络分属于不同的逻辑网络，以实现系统内部与外部网络的隔离。在负载均衡算法的选择上，使用最小连接法，每当用户的请求来临时，任务分发单元会将任务平滑分配给最小连接数的前端网站节点，这样的架构以廉价且透明的方式扩展了服务器和网络的带宽，可以大大提升系统的并发量，同时保证网站前端整体的稳定性和可靠性。

2. 平台保障服务

平台保障服务用以实现后端微服务的基础框架，包括 API 路由网关、服务注册中心、服务监控组件。API 网关收到前端的请求，不会直接调用后端的业务服务，而是首先会从服务注册中心根据当前请求来获取对应的服务配置，随后通过服务配置再调用已注册的服务。当后端微服务存在多个实例时，将采取负载均衡的方式调用。服务注册中心是整个云原生架构体系的核心部分，由 SpringCloud 的 Eureka 组件来实现，专门提供微服务的服务注册和发现功能，涉及三种角色：服务提供者、服务消费者和服务注册中心。API 路由网关、所有业务服务，以及服务监控平台组件都注册到服务注册中心。通过服务注册中心两两互相注册、API 路由网关向服务注册中心注册多个实例等方式，来实现后端整体服务的高可靠性。服务监控平台通过注册到服务注册中心，获取所有注册到服务注册中心的后端业务服务，从而监控到所有后端业务服务的运行状态信息，最后收集并展示整个微服务系统的运行状态，更进一步保证整个后端的服务质量。

3. 业务服务

业务服务按功能模块，相应划分为题库服务、评测机服务、考试服务、比赛服务、抄袭判定服务等。各服务单独打包，基于 Docker 容器，连同运行环境一起封装，根据实际情况可在一台或多台物理机同时部署多个实例，服务启动后会将自身信息注册到服务注册中心。服务间协同工作，通过松耦合的服务发现机制，动态调用对方 RESTAPI 接口。对于压力较大的服务，如评测机服务、抄袭判定服务等，将部署为多实例集群。以在线考试功能为例，用户进入考试时，考试服务核验考生信息通过，调用题库服务，题库服务返回试题，由考试服务组合成试卷，返回前端显示。用户交卷时，提交的程序代码到达考试服务，考试服务拆分

后分发给题库服务，题库服务将程序代码和测试用例送入 MQ 队列排队。然后由负载均衡机制，依次将队列中待评测程序分发给评测机服务编译、执行、判分，完成评测后，题库服务统计试题通过率，考试服务统计成绩并向前端显示。在此期间服务请求者无需了解其他服务对数据如何具体处理和分析。

总结

系统自 2019 年 10 月正式上线已运行一年有余，在学校的日常教学考试和竞赛培训中投入使用，至今已有 3000 名以上的学生用户，评测了 70000 份以上的程序代码，获得了单位同事领导和学校教师们的一致好评。在开发和试运行过程中，主要遇到了两个问题。一是跨域问题。OJ 系统前后端分离，前端通过 Ajax 访问后端服务。由于浏览器同源策略的限制，导致前端 UI 无法正常访问不同端口和 IP 的后端服务。我们利用 SpringBoot 后端的 Cors 跨域机制解决了该问题。二是评测机宕机问题。评测机服务需要执行用户提交的代码，部分用户短时间内提交了大量不安全代码，导致评测机集群中所有实例全部宕机。我们引入心跳机制、快照回滚机制，以及基于机器学习技术的预判断机制，使评测机宕机时能够在 10 秒内自动重置恢复运行，最终解决了该问题。

实践证明，OJ 系统项目能够顺利上线，并且稳定运行，与系统采用了合适的架构设计密不可分。经过这次云原生架构的方法和实施的效果后，我也看到了自己身上的不足之处，在未来还会不断更新知识，完善本系统在各方面的设计，使整个 OJ 系统能够更加好用，更有效地服务于高校师生。

三、论软件测试中缺陷管理及其应用

1、真题原文：

论软件测试中缺陷管理及其应用

围绕“论软件测试中缺陷管理及其应用”论题，依次从以下三个方面进行论述。

1. 概要叙述你参与管理和开发的软件项目以及承担的工作
2. 详细论述常见的缺陷种类及级别，论述缺陷管理和基本流程
3. 结合你具体参与管理和开发的实际项目，说明是如何进行缺陷管理的。请具体说明实施过程及应用效果。

开发工程师：修改 BUG

缺陷来源

Requirement：由于需求的问题引起的缺陷(需求不完全或逻辑错误)

Architecture：由于构架的问题引起的缺陷（登录 session 失效）

Design：由于设计的问题引起的缺陷（图片大小，页面元素显示问题等

Code：由于编码的问题引起的缺陷

Test：由于测试的问题引起的缺陷（软件测试的设计与实施发生错误。特别是系统级的功能测试）

Integration：由于集成的问题引起的缺陷

缺陷严重性和优先级

缺陷严重性和优先级

缺陷的严重性说明

0 级（致命）最严重等级，缺陷导致系统任何一个主要功能完全丧失、用户数据受到破坏、系统崩溃、悬挂、死机等

1 级（严重）系统的主要功能部分丧失、数据不能完全保存，系统的次要功能完全丧失，系统所提供的功能或服务收到明显影响

2 级（一般）系统的次要功能没有完全实现，但不影响用户的正常使用。例如，提示信息不太准确；或用户界面差、操做时间稍长等问题

3 级（微小）操作者不方便或遇到麻烦，但不影响功能的操做和执行，如字体不美观、按钮大小不很合适、字体排列不对齐等一些小问题

缺陷的优先级说明

立即解决（p1 级）缺陷导致系统几乎不能完全运行、使用，或严重妨碍测试的执行，需立即修正、尽快修正

高优先级（p2 级）缺陷严重，影响测试，需要优先考虑修正，如不超过 24 小时修正

正常排队（p3 级）缺陷需要修正，但可以正常排队等待修正

低优先级（p4 级）缺陷可以在开发人员有时间的时候被修正，如果没时间可以不修正

缺陷管理基本流程：

括哪些测试。计划测试执行并向应用程序测试人员分配任务。3) 运行测试: 自动或手动执行测试集中的测试。查看测试运行的结果, 以便确定是否在应用程序中检测到了缺陷。生成报告和图表, 帮助分析这些结果。

最后, 提交应用程序中检测到的缺陷并跟踪修复的进展情况。报告应用程序中检测到的新缺陷。质量保证测试人员、开发人员、项目经理和最终用户可以在测试流程的任何阶段添加缺陷。检查新缺陷并确定应修复哪些缺陷。更正决定要修复的缺陷。测试应用程序的新内部版本。继续执行该流程, 直至缺陷修复为止。生成报告和图表, 帮助分析缺陷修复的进展情况, 并协助确定发布应用程序的时间。

我们的疫情信息可视化系统进行缺陷管理, 一共六个步骤: 1. 准备阶段。首先获得进行缺陷分类方法操作的批准; 采取缺陷分类方法, 要获得开发团队和测试团队的支持。找到缺陷分类方法的关键人物; 调查项目当前的状况。给开发人员和测试人员指派缺陷分类方法角色。在一个缺陷跟踪工具上部署缺陷分类方法计划。最后进行必要的培训。2. 计划阶段。对属性进行确定。之后把项目划分成几个组件。评估缺陷分类方法需要设定项目检查点。然后建立管理计划文档。制定评审缺陷分类方法计划。3 和 4. 数据的输入和验证。数据输入之前, 确保所有的开发人员和测试人员都清楚地明白每个属性的含义。在数据输入过程中, 数据的格式应该由工具来控制。缺陷状态的转换必须保持一致。数据输入以后, 需要完全的验证。5. 评估阶段, 生成缺陷分类方法评估报告, 或者选择一些有意义的图表。6. 活动。这个步骤是最后一个步骤, 此时可以设计一个正式的活动计划。这个计划能够帮助我们持续不断地提高软件质量。这个活动计划包括来自项目文档的材料, 程序代码组件, 开发步骤以及测试方法等。其中最重要的是下一次发布或者下一次迭代所要采取的活动。这些活动的目标必须清楚而且是可度量的。因为下一个阶段中出现的问題往往是前一个阶段中出现的错误所导致的。

在开发我们的疫情信息可视化系统时在软件测试方面下了很大的功夫, 我们深受触动, 收获颇丰, 也意识到了从事软件缺陷测试与评估研究是一个有理论价值和实际意义、并且存在一定难度的课题。如何加强软件测试缺陷管理, 提高软件质量已经成为业界所关注而又必须解决的热点问题。

四、论企业集成架构设计及应用

1、真题原文：

论企业集成架构设计及应用

围绕“论企业集成架构设计及应用”为题

1. 概要叙述你参与的软件开发项目的及承担的主要工作
2. 详细说明三类企业集成架构设计技术分别要解决的问题及其含义,并阐述每种技术具体包含了哪些集成架构。
3. 根据你所参与的项目,说明用了哪些企业集成架构设计技术,实施效果如何。

2、参考理论：

集成是企业信息化和谐发展的阳光大道,要走上这条大道则需要建立企业的集成架构。

企业集成架构的问题不同于业务系统,即它更为关注企业的 IT 软件基础设施。

目前的“信息孤岛”、“流程割裂”、“企业运行全局不可见”等问题已经愈加明显,直接制约企业的业务发展。

打破信息孤岛的方法是,建立企业级的数据总线,连接不同的业务系统,业务系统数据通过统一的翻译处理后形成企业内部甚至是企业间/行业间的“世界语”,直接简化系统之间的连接方式和数据处理的紧密耦合,实现企业的数据同步和共享。

针对于企业应用,依据不同的关注点,架构可以分为如下几类:

业务架构 (BusinessArchitecture): 关注于业务及其流程;

应用架构 (ApplicationArchitecture): 关注于应用系统设计;

基础架构 (InfrastructureArchitecture): 关注于基础技术;

数据架构 (DataArchitecture): 关注于数据存储及其规划;

从发展和动态的角度看,集成技术将使企业有手段和机会不断调整和优化企业业务流程的运行,业务系统的专业化程度将增强,企业的业务流程将逐步梳理以趋于最优,基于集成架构的企业业务流程运行将更趋向自动化。业务流程的自动化体现业务系统之间的和谐共处。对于企业而言,业务流程自动化是追求的目标,也是企业信息化发展的方向。

企业集成架构发展的基石是开放的标准和规范,从共享和协同的特征看,企业集成架构

划分功能模块、确定每个模块的实现算法以及程序流程等，形成软件的具体设计方案。软件设计把许多事物和问题按不同的层次和角度进行抽象，将问题或事物进行模块化分解，以便更容易解决问题。分解得越细，模块数量也就越多，设计者需要考虑模块之间的耦合度。

请围绕“论软件设计方法及其应用”论题，依次从以下三个方面进行论述。

1. 概要叙述你所参与管理或开发的软件项目，以及你在其中所承担的主要工作。
2. 详细阐述有哪些不同的软件设计方法，并说明每种方法的适用场景。
3. 详细说明你所参与的软件开发项目中，使用了哪种软件设计方法，具体实施效果如何。

2、参考理论：

软件设计方法包括有：模型驱动设计，面向对象设计，结构化设计，快速应用开发，原型设计，信息工程法。

1) 模型驱动设计

模型驱动设计是一种系统设计方法，强调**通过绘制图形化系统模型描述系统的技术和实现**。通常从模型驱动分析中开发的**逻辑模型导出系统设计模型**，最终系统设计模型将作为构造和实现新系统的蓝图。

2) 结构化设计

它是一种面向过程的系统设计技术，它将系统过程分解成一个容易实现和维护的计算机程序模块。把一个程序设计成一个自顶向下的模块层次，一个模块就是一组指令：一个程序片段，程序块，子程序或者子过程，这些模块自顶向下按照各种设计规则和设计指南进行开发，模块需要满足高内聚松散耦合的特征。

3) 信息工程法

信息工程是一种用来计划，分析和设计信息系统的模型驱动的，以数据为中心的但对过程敏感的技术。信息工程模型是一些说明和同步系统的数据和过程的图形。信息工程的主要工具就是数据模型图（物理实体关系图）。

4) 原型设计

原型法是一种**反复迭代过程**，它需要设计人员和用户之间保持紧密的工作关系，通过构造一个预期系统的小规模的，不完整的但可以工作的示例来与用户交互设计的结果。原型设计方法鼓励并要求最终用户主动参与，这增加了最终用户对项目的信心和支持。更好地适应用户总是想改变想法的自然情况。原型是主动的模型，最终用户可以看到并与之交互。

敏感信息采用 MD5 数字摘要算法进行加密，对于重要的金额数据采用对称加密。首次复审会议后架构组进行了架构的细化，确定了通信协议种类，主要接口，核心业务设计模式。并且对主要核心类进行的属性和方法的识别。再次经过与客户专家组讨论与评估，最终确定了 XXAPP 电商系统的架构。

此架构在后期的实现过程中经历一些调整，如下单改成多线程模式，消息通知失败的重试机制等部分，但总体结构稳定。系统上线后经客户专家组的验收反馈，本系统架构解决了需求中的核心关注点，并取得了很好的效果，关键是可以对系统运算能力进行水平扩展，能满足未来一定时期的业务需要。

结论：

经历 XXAPP 电商系统架构完整过程使我认识到架构不只是技术，而是要用技术解决客户的业务中的核心关注点，遵循一定架构过程和原则才能设计一个好的系统架构。也让我从中学到构件识别技术，比如构件的独立性，无可见状态，接口规约的版本一致性；构件关系组及架构评审相关知识。

二、论软件系统架构评估及其应用

1、真题原文：

论软件系统架构评估及其应用

对于软件系统，尤其是大规模复杂软件系统而言，软件系统架构对于确保最终系统的质量具有十分重要的意义。在系统架构设计结束后，为保证架构设计的合理性、完整性和针对性，保证系统质量，降低成本及投资风险，需要对设计好的系统架构进行评估。架构评估是软件开发过程中的重要环节。

请围绕“软件系统架构评估及其应用”论题，依次从以下三个方面进行论述。

1. 概要叙述你所参与管理或开发的软件项目，以及你在其中所承担的主要工作。
2. 详细阐述有哪些不同的软件系统架构评估方法，并从评估目标、质量属性和评估活动等方面论述其区别。
3. 详细说明你所参与的软件开发项目中，使用了哪种评估方法，具体实施过程和效果如何

过多长时间对某个事件进行相应。可用性是指系统能够正常运行的时间比例。安全性实指系统在向合法用户提供服务的同时，阻止非法用户的使用或拒绝向其提供服务。可修改性是指能够快速以较低代价对系统进行变更的能力。可测试性是指系统发生故障以后，定位其故障的能力和进行测试的能力。可靠性是指系统在发生问题后，维持系统功能的能力。易用性是指用户使用该系统完成所期望功能的难易程度。

在使用架构权衡分析法（ATAM）进行系统架构评估时，我们首先成立了架构评估小组，评估小组主要包括关键用户、评估负责人、架构设计人员项目负责人、需求人员、开发人员、测试人员以及实施人员。小组按照架构权衡分析方法的描述和介绍、调查和分析、测试、报告这四个阶段进行架构评估工作。

在描述和介绍阶段，评估负责人首先描述了架构权衡分析法的执行方法与具体过程，架构权衡分析法是一种基于场景的架构评估方法，对系统的多个质量属性基于场景评估。评估后确定架构的各项指标是否满足需求。之后对关键客户描述的建设该系统的业务动机，该系统的首要目标是为其产品提供条码追溯能力，其次为了提高该厂商整个生产过程的自动化程度。并为决策者提供决策依据。最后我昨晚系统的架构负责人项目详细描述该系统采用面向对象和层次结构架构风格的原因，采用集群部署的原因以及各组件所采用的技术和实现的功能。

在调查和分析阶段，我作为架构负责人进一步对架构进行了拆解和说明。然后各方基于各自的考虑提出了需求，我们将需要进行整理随之产生质量效用树。质量效用树中主要包括了四个质量属性。分别是可用性、安全性、性能和可修改性。比如在性能方面，客户提出分别在全国多个位置的生产基地和供货仓库访问该系统页面的延迟不能超过 3 秒；又比如在可用性方面，客户提出生产基地采用 3 班倒 24 小时工作生产制度，要求系统必须高可用，不能出现宕机等停止服务的情况。再比如在安全性方面，客户提出该系统存储传输的数据涉及商业机密，必须进行加密传输，保证信息不外泄。

在测试阶段，我们开始通过投票的方式对质量效用树进行优先级排序，然后安装高优先级优先的原则分析架构是否满足各项指标。比如在性能和可靠性方面，系统采用了集群的部署方式，这样首先提高了系统的吞吐量，其次当有任何一台机器发生问题，也不会影响整个系统提供服务。但是客户提出性能方面是重点，不能因为性能问题影响生产，于是我们决定引入 Redis 内存数据库进一步提升性能。在安全性方面，我们权衡了性能和安全性因素，对传输数据采用了对称式和非对称式结合的加密技术。这样即满足了加密的强度，又保障了

加密解密效率，客户较为认可。

在报告阶段，我们最终形成了描述评估结果，其中详细描述了架构是否满足质量效用树的各项指标，也包含了架构中的风险点、敏感点与权衡点。最后报告中也给出了架构调整意见，比如为提高性能需要引入 Redis 内存数据库。报告和评估期间的会议记录最终进行了归档。

通过架构权衡分析法进行架构评估，不仅准确的获取了系统对各项质量属性的需求，也成功的完成了对架构的评估工作，为架构的优化提供了依据。该项目成功的在规定日期上线，截至目前一直平稳运行，得到了客户的充分肯定，也在相关行业内赢得了良好的口碑。当然我们在架构评估方面还有很多的不足，比如我们目前只掌握了 ATAM 方式的评估方法，对其他的架构评估方式如软件架构分析方法（SAAM）的使用上还有不足，我们将在后续的工作与实践中继续磨练与提升。

三、论数据湖技术及其应用

1、真题原文：

论数据湖技术及其应用

近年来，随着移动互联网、物联网、工业互联网等技术的不断发展，企业级应用面临的数据规模不断增大，数据类型异常复杂。针对这一问题，业界提出“数据湖（DataLake）”这一新型的企业数据管理技术。数据湖是一个存储企业各种原始数据的大型仓库，支持对任意规模的结构化、半结构化和非结构化数据进行集中式存储，数据按照原有结构进行存储，无须进行结构化处理；数据湖中的数据可供存取、处理、分析及传输，支撑大数据处理、实时分析、机器学习、数据可视化等多种应用，最终支持企业的智能决策过程。

请围绕“数据湖技术及其应用”论题，依次从以下三个方面进行论述。

1. 概要叙述你所参与管理或开发的软件项目，以及你在其中所承担的主要工作。
2. 详细阐述数据湖技术，并从主要数据来源、数据模式（Schema）转换时机、数据存储成本、数据质量、面对用户和主要支撑应用类型等 5 个方面详细论述数据湖技术与数据仓库技术的差异。
3. 详细说明你所参与的软件开发项目中，如何采用数据湖技术进行企业数据管理，并说明具体实施过程以及应用效果

件。数据湖通常是企业所有数据的单一存储，包括源系统数据的原始副本，以及用于报告、可视化、分析和机器学习等任务的转换数据。数据湖可以包括来自关系数据库（行和列）的结构化数据，半结构化数据（CSV，日志，XML，JSON），非结构化数据（电子邮件，文档，PDF）和二进制数据（图像，音频，视频）。

在数据来源方面，数据既包括内部数据，也包括来自第三方的外部数据，既包括传统业务处理采集的业务数据，也包括手机终端、传感器、机器设备、网站网络、日志等技术产生的数据。

数据作为数据资产管理的对象，在近些年体现出规模海量、来源多样、格式繁杂、采集实时等特征。在数据量方面，单一机构的数据规模由以前的GB级上升到TB级，甚至PB级、EB级，数据增速快。在数据格式种类方面，除传统的结构化数据之外，文本数据、图像数据、语音数据、视频数据等半结构化数据或非结构化数据占比越来越大，种类日益丰富。

由于数据来源和数据格式的多样化，以及数据量的指数级增长，依靠“手工人力”的电子表格数据治理模式即将被“自动智能”的“专业工具”取代，越来越多的数据管理员、业务分析师和数据领导者采用“平台工具”增强企业的数据管理能力，包括梳理元数据、管理主数据，优化数据集成、提升数据质量等。具体来说，机器学习和人工智能通过自动提取元数据，将不同的数据进行关联并分析；通过配置和优化主数据，使主数据的管理更加便捷和准确；通过语义分析实现相同数据源的连接，简化数据集成流程；通过增强数据的分析、清理和识别，提升数据质量。同时，随着智能优化技术不断引入到数据管理活动中，数据间的多维关系将被自动化识别和可视化展现，帮助用户高效探索数据和分析数据，降低数据使用门槛，有助于非专业人士成为数据科学家，扩大数据的使用对象和应用范围。

数据资产管理的使用不仅仅局限于拥有海量数据或强大数据处理能力的机构，任何一个机构都可以成为数据资产化管理的实践者。选择一个小型且效果明显的项目实施数据资产管理，也可以成为逐步构建完整数据资产管理体系的良好开端。此外，数据资产的应用范围已经从传统的企业内部应用为主发展为支撑内部和服务外部并重，数据资产应用和服务范围的扩大成为企业战略发展的一部分，实现数据资产保值到增值的跨越。

二、简析某科技创新大数据平台

下面以某科技创新大数据平台为例，详细说明系统的概念体系架构设计。系统共分为五层，其中包括数据源层、数据存储层、数据处理层、数据共享层和应用层。如图所示。

数据源层，主要用来管理各类数据源，数据分散到统计库、资源库、需求库、专家库以及企

程，还描述了数据在不同过程中发生的情况。它可以帮助分析信息的使用方式并追踪用于特定用途的关键信息位。

1.2 影响分析

影响分析帮助用户迅速了解分析对象的下游数据信息，快速掌握元数据变更可能造成的影响，以便更有效的评估变化该元数据带来的风险，从而帮助用户高效准确的对数据资产进行清理、维护与使用。

1.3 全链分析

全链分析是用来分析指定元数据前后与其有关系的所有元数据，不仅反应了元数据的来源与加工过程，也反应了元数据的使用情况，使用全链分析可清晰的了解该元数据的来龙去脉。

1.4 关联度分析

关联度分析是从关系数量的角度对指定元数据进行分析，来体现该元数据在系统中依赖程度的高低，从一定的角度可以反映出该元数据的重要程度。

1.5 属性差异分析

属性差异分析是用来比较同类型元数据之间属性值的差异，方便用户识别相似元数据之间的存在的微小差距。

2. 数据标准建设

2.1 配置灵活的数据标准属性

定义不同的数据标准可能存在需要录入不同的属性，为了满足不同项目对数据标准的设计，睿治数据治理平台提供了数据标准集管理，内置了业务属性、技术属性、管理属性、质量属性、主数据属性、生命周期属性等供用户选择使用，并支持自定义属性。

2.2 方式丰富的数据标准录入

河北省科技创新大数据平台提供方便灵活的操作界面，根据用户选择合适的方式，快速创建数据标准，支持用户手动创建数据标准，同时支持拾取元数据生成数据标准，简化数据标准创建的步骤，同时支持修改、删除等操作。除了手动创建外，还支持通过导入的方式进行批量创建。通过导出标准集，让用户在线下对数据标准进行整理，将整理完成的数据标准导入到平台后，成为一条可映射、评估的数据标准。

2.3 完备的数据标准审批

数据标准创建保存，并确认无误后，支持整集发起审批。审批支持通过、退回操作，

监控、数据保护方面提供全方位的软件支持，在该项目中我主要担任系统架构师岗位，负责整体架构设计。

本文以该保险一体化项目为例，主要讨论了负载均衡技术在 Web 系统中的具体应用。在网络请求到服务器的过程中，我们先使用了 F5 负载，利用 F5 负载均衡技术降低了服务器的压力。经过 F5 负载之后，我们又接入了 SLB 负载均衡，将访问流量根据转发策略分发到后端多台云服务器上，因为我们的项目是前后端分离的架构，经过 slb 负载之后，到达前端的服务器上，前端的配置里面增加了 nginx 配置，根据 nginx 的转发策略再请求到后端的服务器上，从而提升了 web 服务器的性能和安全性。

正文

随着该集团保险业务的迅猛扩展，之前的互联网上的入口已经不能满足每天几十万的流量控制和访问量，在订单量高发的情况下，系统总是会出现卡顿、反应缓慢等问题，极端的情况是整个服务器直接挂掉，系统直接不可用，极大的影响了业务的营销，影响了公司的正常运转。为了解决高并发和高流量的访问，该集团决定重新搭建集团保险一体化项目，实现系统的高可用、高负载，提高系统的访问性能和稳定性。2020 年 1 月，我被该集团任命为该保险一体化项目的系统架构师，架构小组共 6 人，我主要负责整体的架构设计，5 月份完成架构工作，整个项目共耗时 1 年，2021 年 1 月顺利通过验收，获得了业务的一致好评。在架构开始阶段，我们便意识到，之前系统出现不可用的情况，是因为是单台服务器导致的，当这台服务器出现异常的时候，直接就挂掉了。每次恢复都需要手动重启，影响业务使用。所以，为了解决这个问题，我们搭建了多台服务器，并从网络访问入口到后端服务器的整个调用链我们都做了负载均衡处理。

众所周知，常见的负载均衡算法有三种，（1）轮询法。轮询法就是将用户的请求轮流分配给服务器，就像是挨个数数，轮流分配。这种算法比较简单，具有绝对均衡的优点，但是也正是因为绝对均衡，它必须付出很大的代价，例如它无法保证分配任务的合理性，无法根据服务器承受能力来分配任务。（2）随机法。随机法是随机选择一台服务器来分配任务。它保证了请求的分散性达到了均衡的目的。同时它是没有状态的，不需要维持上次的选择状态和均衡因子。但是随着任务量的增大，它的效果趋向轮询后也会具有轮询法的部分缺点。（3）最小连接法。最小连接法将任务分配给此时具有最小连接数的节点，因此它是动态负载均衡算法。一个结点收到一个任务后连接数就会加 1，当结点发生故障时就将结点权值设置 0，不再给结点分配任务。最小连接法适用于各个结点处理的性能相似的情形。任务分发单元会

将任务平滑分配给服务器。但当服务器性能差距较大时，就无法达到预期的效果。因为此时连接数并不能准确表明处理能力，连接数小而自身性能很差的服务器可能不及连接数大而自身性能极好的服务器。所以在这个时候就会导致任务无法准确地分配到剩余处理能力强的机器上。本文将重点围绕 F5 负载均衡、SLB 负载均衡、nginx 反向负载均衡技术来进行论证。

F5 负载均衡是一种硬件负载均衡技术，建立在现有网络结构之上，它提供了一种廉价有效透明的方法扩展网络设备和服务器的带宽、增加吞吐量、加强网络数据处理能力、提高网络的灵活性和可用性。这里需要说明的是：F5 负载均衡设备不是基础网络设备，而是一种性能优化设备。对于网络应用而言，并不是一开始就需要负载均衡，当网络应用的访问量不断增长，单个处理单元无法满足负载需求时，网络应用流量将要出现瓶颈时，负载均衡才会起到作用。在该项目中，我们为了减轻后端服务器的压力和提高系统的响应时间，将从互联网上过来的流量请求平均的分配到后端的 slb 服务器上。因为是平均分配流量和请求，所以这里使用的负载均衡算法是轮询法。

上面经过 F5 硬件负载之后，又到了 SLB 软件负载，这里我们接入的 SLB 是阿里云的服务器，阿里云负载均衡 SLB 主要由 3 个基本概念组成：负载均衡器 LoadBalancer，监听器 Listener，后端服务器 BackendServer。负载均衡实例 ServerLoadBalancerinstances：一个负载均衡实例是一个运行的负载均衡服务，用来接收流量并将其分配给后端服务器。监听器 Listeners：监听器用来检查客户端请求并将请求转发给后端服务器。监听也会对后端服务器进行健康检查。后端服务器 BackendServers：一组接收前端请求的 ECS 实例，ECS 实例是真正提供服务访问的，后端服务器的数据要一致，对后端服务器的操作系统等配置没有要求，只要求它们的数据是一致的。应用接入 slb 负载均衡之后，可以在负载均衡实例下添加多台 ECS 实例。当其中一部分 ECS 实例发生故障后，将请求分发给正常运行的 ECS 实例，成功消除了单点服务的故障。同时，slb 还支持同城容灾(多可用区容灾)和跨地域容灾，提高了应用程序的可用性和可靠性。

经过上面的 slb 负载之后，请求就正常的连接到后端的服务器上面来，首先，接入的是前端服务器，在前端服务器请求后端的接口上面增加了一层 nginx 路由转发，这里主要是利用了 nginx 反向代理的特性，将来自 Internet 的连接请求以反向代理的方式动态转发给内部网络上的多台服务器进行处理，从而达到负载均衡的目的。基于反向代理则可以根据内部服务器的性能差异及实时负载情况进行动态负载均衡，当系统多个 Web 服务器性能存在明显差异或内部 Web 服务器出现故障时，负载均衡器可以更快做出响应，从而保证客户端的访

问效率。采用基于反向代理的负载均衡机制，可在代理服务器中引入调速缓存机制，对 Web 服务器返回的静态页面或图片等静态资源进行缓存，由代理服务器承担对原始服务器的静态资源访问请求，从而进一步降低原始 Web 服务器的负载。从安全性方面讲，采用基于反向代理的负载均衡机制，代理服务器屏蔽了客户端对真实 Web 服务器的直接访问，恶意用户无法对真实 Web 服务器进行攻击，且可以通过代理服务器为原本不安全的客户端与 Web 服务器之间的连接建立安全通道。因此采用基于反向代理的负载均衡机制可为系统提供更好的安全性保障。

经过上面的三层负载均衡，提高了该集团保险一体化项目的高可靠性，并使该系统能应对互联网上大量的并发和流量请求。项目运行至今，再也没有出现过系统不可用的情况，且运行一直良好，并在 2021 年 1 月成功通过用户验收，得到了一致好评。

但是，在架构设计中也存在不足。主要有 2 点，一是后端部署节点过多，在部署上面比较麻烦。二是部署节点缺少统一性的管理，扩容和缩容都有待提高。针对第一个问题，我们在系统部署的时候接入了 jenkins+Devops，每次提交代码的时候自动触发流水线，然后带出到生产环境，大大降低了人工操作带来的风险。针对第二个问题，我们接入了 k8s 集群管理技术，将 docker 运行的实例全部管理起来，大大提升了系统应用的自动扩容和缩容。

2018 年系统架构师论文

一、论软件开发过程 RUP 及其应用

1、真题原文：

论软件开发过程 RUP 及其应用

RUP (Rational Unified Process) 是 IBM 公司一款软件开发过程产品，它提出了一整套以 UML 为基础的开发准则，用以指导软件开发人员以 UML 为基础进行软件开发。RUP 汲取了各种面向对象分析与设计方法的精华，提供了一个普遍的软件过程框架，可以适应不同的软件系统、应用领域、组织类型和项目规模。

请围绕“论软件开发过程 RUP 及其应用”论题，依次从以下三个方面进行论述。

1. 概要叙述你参与管理和开发的软件项目以及你在其中所担任的主要工作。
2. 详细论述软件开发过程产品 RUP 所包含的 4 个阶段以及 RUP 的基本特征。

3. 结合你所参与管理和开发的软件项目，详细阐述 RUP 在该项目中的具体实施内容，包括核心工作流的选择、制品的确定、各个阶段之间的演进。

2、参考理论：

RUP 中的软件过程在时间上被分解为四个顺序的阶段，分别是初始阶段，细化阶段，构件阶段，交付阶段。每个阶段结束时都要安排一次技术评审，以确定这个阶段的目标是否已经满足。如果评审结果令人满意，就可以允许项目进入下一个阶段。

四个阶段的核心任务分别为

初始阶段

明确地说明项目规模。这涉及了解环境及最重要的需求和约束，以便于可以得出最终产品的验收标准。

计划和准备商业理由。评估风险管理，人员配备，项目计划和成本/进度/收益率折衷的备选方案。

综合考虑备选构架，评估设计和自制外购复用方面的折中，从而估算出成本，进度和资源。此处的目标在于通过对一些概念的证实来证明可行性。

该证明可采用可模拟需求的模型形式或用于探索被认为高风险区域的初始原型。初始阶段的原型设计工作应该限制在确信解决方案可行就可以了。该解决方案在细化和构建阶段实现。

准备项目的环境，评估项目和组织，选择工具，决定流程中要改进的部分

细化阶段

快速确定构架，确认构架并为构架建立基线。

根据此阶段获得的新信息改进前景，对推动架构和计划决策的最关键用例建立可靠的了解。

为构建阶段创建详细的迭代计划并为其建立基线。

改进开发案例，定位开发环境，包括流程和支持构件团队所需的工具和自动化支持。

改进构架并选择构件。评估潜在构件，充分了解自制外购复用决策，以便有把握地确定构建阶段的成本和进度。集成了所选构架构件，并按主要场景进行了评估。通过这些活动得到的经验有可能导致重新设计构架，考虑替代设计或重新考虑需求。

构建阶段

资源管理，控制和流程优化。

完成构件开发并根据已经定义的评估标准进行测试。

根据前景的验收标准对产品发布版进行评估。

提交阶段

执行部署计划

对最终用户支持材料定稿

在开发现场测试可交付产品

制作产品发布版。

获得用户的反馈

基于反馈调整产品

使最终用户可以使用产品

RUP 最核心的 3 个特征是：**用例驱动、以架构为中心的、迭代和增量。**

制品 (Artifact) ——what 的问题：制品是活动生成、创建或修改的一段信息。也可译为产品、工件等，和制品的意思差不多。

工作流 (Workflow) ——when 的问题：工作流描述了一个有意义的连续的活动序列，每个工作流产生一些有价值的产品，并显示了角色之间的关系。

3、参考范文：

二、论软件体系结构的演化

1、真题原文：

论软件体系结构的演化

软件体系结构的演化是在构件开发过程中或软件开发完毕投入运行后，由于用户需求发生变化，就必须相应地修改原有软件体系结构，以满足新的变化了的软件需求的过程。体系结构的演化是一个复杂的、难以管理的问题。

请围绕“论软件体系结构的演化”论题，依次从以下三个方面进行论述。

1. 概要叙述你参与管理和开发的软件项目以及你在其中所承担的主要工作。

2. 软件体系结构的演化是使用系统演化步骤去修改系统，以满足新的需求。简要论述系统演化的 6 个步骤。

3. 具体阐述你参与管理和开发的项目是如何基于系统演化的 6 个步骤完成软件体系结构演化的。

2、参考理论：

软件体系结构演化实际上指的是 ABSD 方法（基于架构的软件设计方法）中的最后一个阶段。体系结构演化是使用系统演化步骤去修改应用，以满足新的需求。主要包括以下六个步骤。

需求变动归类

首先必须对用户需求的变化进行归类，使变化的需求与已有构件对应。

对找不到的构件的变动，也要做好标记，在后续工作中，将创建新的构件，以对应这部分变化的需求。

制定体系结构演化计划

在改变原有结构之前，开发组织必须制订一个周密的体系结构演化计划，作为后续演化开发工作的指南。

修改增加或删除构件

在演化计划的基础上，开发人员可根据在第 1 步得到的需求变动的归类情况，决定是否修改或删除存在的构件、增加新构件。最后，对修改和增加的构件进行功能性测试。

更新构件的相互作用

随着构件的增加、删除和修改，构件之间的控制流必须得到更新

构件组装与测试

通过组装支持工具把这些构件的实现体组装起来，完成整个软件系统的连接与合成，形成新的体系结构。然后对组装后的系统整体功能和性能进行测试。

技术评审

对以上步骤进行确认，进行技术评审。评审组装后的体系结构是否反映需求变动，符合用户需求。如果不符合，则需要第 2 到第 6 步之间进行迭代。

行 1 年时间，得到了各级监管部门和企业一致认可和好评。

【背景】

质量技术监督在生产领域、市场流通领域等方面都起到至关重要的作用，关乎到企业生产和人民生活。某省质量技术监督局在信息化方面起步较早，先后建设了数套专项系统，但缺乏对业务的统一监管，为贯彻落实国家市场监督管理总局《关于加快推进质量体系监管信息化建设要求》，一体化平台的建设工作受到各级领导的关注。项目建设目标覆盖省、市、县区三级质量监管部门及全省三十多万家企业，为工业生产、报审、送检、勘验、巡检和执法工作提供支持。我所在公司长期从事相关行业的软件开发，对质量监督领域有着丰富经验，因此顺利中标承建了该项目，初期投入 15 名技术人员和 5 名测试人员，由我担任架构师职务，负责软件架构设计和中间件选型。

项目启动后通过对业务的梳理，初步将一体化平台划分为四大模块，其中政务服务模块针对的是通过互联网访问系统的企业用户，在功能上主要包括报审、送检、公示和政策法规宣传等功能；而其它三个模块针对的是通过电子政务外网访问系统的监管部门和事业单位用户，其中运行管理模块主要功能包括审批、勘验、颁证、建档；企业监管模块主要功能包括巡检、执法；电子监察模块主要功能包括实时监控、预警纠错和效能评估。

【论点】

在架构设计开始阶段，我意识到选择合适的架构风格对架构设计的重要性。由于客户单位已投入使用的数套业务系统，因建设年代不同、委托开发公司不同，导致这些系统的开发语言、架构设计、执行标准都各不相同，成为了各自为政的信息孤岛。客户为适应新形势下的监管工作急需进行业务整合，要求一体化平台打通从生产到监管的全业务流。而这其中就涉及到与特种设备、单兵监控、检验检疫及档案归档等多套专项系统进行交互。因此在综合考虑了标准化、松耦合、粗粒度服务和可移植性等多方面因素后，我决定使用 Java 作为开发语言，并采用面向服务的架构设计（Service-OrientedArchitecture, SOA），来满足客户对系统的要求。下面说明实现 SOA 主要的技术和标准及每种技术和标准的具体内容。

【回应论点一】

SOA 是一个组件模型，它将应用程序的不同功能单元进行拆分，并通过服务之间定义良好的接口联系起来，它强调服务共享和重用。与 SOA 紧密相关的技术主要有 UDDI、WSDL 和 SOAP。其中 UDDI（统一描述、发现和集成）提供了一种服务发布、查找和定位的方法，是服务的注册规范，主要包含数据模型、API 和注册服务三项内容；WSDL（服务描述语言）是对

家标准进行对比判断是否符合审批标准。在服务调用期间请求者不需要关心服务提供者的业务处理和技术实现，只需需关注自身业务，简化了开发流程提升了工作效率。

【结尾】

实践证明 SOA 架构的使用，降低了系统的耦合度，使软件具有了更好的开放性和扩展性，各方面均表现优异。但在开发过程中也遇到过问题，由于 SOAP 是基于 XML 方式通信效率较低，在企业每年 11 月集中年审时，服务间猛增的消息交互造成了大量积压，导致企业报审不能得到及时处理。我组织了客户和参与厂商进行了专项讨论，最终根据我提出的压缩消息体和增加硬件负载的方式及时解决了该问题，没有对项目造成实质影响。

通过项目组全体成员的共同努力，一体化平台于 2018 年 10 月完成验收工作正式上线，截至目前系统运行稳定运维工作量可控，得到了用户的一致认可和好评。通过这个项目本人更进一步了解到系统架构设计的重要性，同时积累了很多宝贵的经验。在今后的工作中我将不断总结和提升专业技术能力，争取为我国信息化事业贡献一份力量。

四、论 NoSQL 数据库技术及其应用

1、真题原文：

论 NoSQL 数据库技术及其应用

随着互联网 web2.0 网站的兴起，传统关系数据库在应对 web2.0 网站，特别是超大规模和高并发的 web2.0 纯动态 SNS 网站上已经显得力不从心，暴露了很多难以克服的问题，而非关系型的数据库则由于其本身的特点得到了非常迅速的发展。

NoSQL (NotonlySQL) 的产生就是为了解决大规模数据集合及多种数据类型带来的挑战，尤其是大数据应用难题。目前 NoSQL 数据库并没有一个统一的架构，根据其所采用的数据模型可以分为 4 类：键值 (Key-Value) 存储数据库、列存储数据库、文档型数据库和图 (Graph) 数据库。

请围绕“NoSQL 数据库技术及其应用”论题，依次从以下三个方面进行论述。

1. 概要叙述你参与管理和开发的软件项目以及你在其中所担任的主要工作。
2. 详细论述常见的 NoSQL 数据库技术及其所包含的主要内容，并说明 NoSQL 数据库的主要适用场景。
3. 结合你具体参与管理和开发的实际项目，说明具体采用哪种 NoSQL 数据库技术，并说

明架构设计过程及其应用效果。

2、参考理论：

NoSQL 的主要优势

避免不必要的复杂性

高吞吐量

高水平扩展能力和低端硬件集群

避免了昂贵的关系对象映射

NOSQL 的缺点

数据模型和查询语言没有经过数学验证

不支持 ACID 特征

功能简单

没有统一的查询模型

NoSQL 的分类

键值（Key-Value）存储数据库

这一类数据库主要会使用到一个哈希表，这个表中有一个特定的键和一个指针指向特定的数据。Key/value 模型对于 IT 系统来说的优势在于简单、易部署。但是如果 DBA 只对部分值进行查询或更新的时候，Key/value 就显得效率低下了。例如：TokyoCabinet/Tyrant,Redis,Voldemort,OracleBDB.

适应场景：会话、配置文件、参数、购物车等等。这些信息一般都和 ID（键）挂钩，这种情景下键值数据库是个很好的选择。

不适应场景：

1) 取代通过键查询，而是**通过值来查询**。Key-Value 数据库中根本没有通过值查询的途径。

2) **需要储存数据之间的关系**。在 Key-Value 数据库中不能通过两个或以上的键来关联数据。

3) 事务的支持。在 Key-Value 数据库中**故障产生时不可以进行回滚**。

列存储数据库

这部分数据库通常是用来应对分布式存储的海量数据。键仍然存在，但是它们的特点是指向了多个列。这些列是由列家族来安排的。

HBase: HBase 是一个分布式的、面向列的开源数据库，该技术来源于 FayChang 所撰写的 Google 论文“Bigtable: 一个结构化数据的分布式存储系统”。就像 Bigtable 利用了 Google 文件系统 (FileSystem) 所提供的分布式数据存储一样，HBase 在 Hadoop 之上提供了类似于 Bigtable 的能力。HBase 是 Apache 的 Hadoop 项目的子项目。HBase 不同于一般的关系数据库，它是一个适合于非结构化数据存储的数据库。另一个不同的是 HBase 基于列的而不是基于行的模式。

列存储数据库将数据储存在列族 (column family) 中，一个列族存储经常被一起查询的相关数据。举个例子，如果我们有一个 Person 类，我们通常会一起查询他们的姓名和年龄而不是薪资。这种情况下，姓名和年龄就会被放入一个列族中，而薪资则在另一个列族中。

产品: Cassandra、HBase

1. 适用的场景

1) **日志。** 因为我们可以将数据储存在不同的列中，每个应用程序可以将信息写入自己的列族中。

2) **博客平台。** 我们储存每个信息到不同的列族中。举个例子，标签可以储存在一个，类别可以在一个，而文章则在另一个。

2. 不适用场景

1) 如果我们**需要 ACID 事务**。Vassandra 就不支持事务。

2) **原型设计。** 如果我们分析 Cassandra 的数据结构，我们就会发现结构是基于我们期望的数据查询方式而定。在模型设计之初，我们根本不可能去预测它的查询方式，而一旦查询方式改变，我们就必须重新设计列族。

文档型数据库

面向文档数据库会将数据以文档的形式储存。每个文档都是自包含的数据单元，是一系列数据项的集合。每个数据项都有一个名称与对应的值，值既可以是简单的数据类型，如字符串、数字和日期等；也可以是复杂的类型，如有序列表和关联对象。数据存储的最小单位是文档，同一个表中存储的文档属性可以是不同的，数据可以使用 XML、JSON 或者 JSONB 等多种形式存储。

文档型数据库的灵感是来自于 LotusNotes 办公软件的，而且它同第一种键值存储相类似。该类型的数据模型是版本化的文档，半结构化的文档以特定的格式存储，比如 JSON。文档型数据库可以看作是键值数据库的升级版，允许之间嵌套键值。而且文档型数据库比键值数据库的查询效率更高。如：CouchDB, MongoDB. 国内也有文档型数据库 SequoiaDB，已经开源。

MongoDB: MongoDB 是目前在 IT 行业非常流行的一种非关系型数据库 (NoSql)，其灵活的数据存储方式备受当前 IT 从业人员的青睐。MongoDB 很好的实现了面向对象的思想 (OO 思想)，在 MongoDB 中每一条记录都是一个 Document 对象。MongoDB 最大的优势在于所有的数据持久操作都无需开发人员手动编写 SQL 语句，直接调用方法就可以轻松的实现 CRUD 操作。

SequoiaDB: SequoiaDB 是一款分布式非关系型文档数据库，可以被用来存取海量非关系型的数据，其底层主要基于分布式，高可用，高性能与动态数据类型设计 SequoiaDB 可以独立作为一款高性能可扩展的 NoSQL 数据库使用，也可与当前主流分布式计算框架 Hadoop 紧密集成。

产品：MongoDB、CouchDB、RavenDB

1. 适用的场景

1) 日志。企业环境下，每个应用程序都有不同的日志信息。Document-Oriented 数据库并没有固定的模式，所以我们可以使用它储存不同的信息。

2) 分析。鉴于它的弱模式结构，不改变模式下就可以储存不同的度量方法及添加新的度量。

2. 不适用场景

在不同的文档上添加事务。Document-Oriented 数据库并不支持文档间的事务，如果在这方面有需求则不应该选用这个解决方案。

图形 (Graph) 数据库

图形结构的数据库同其他行列以及刚性结构的 SQL 数据库不同，它是使用灵活的图形模型，并且能够扩展到多个服务器上。NoSQL 数据库没有标准的查询语言 (SQL)，因此进行数据库查询需要制定数据模型。许多 NoSQL 数据库都有 REST 式的数据接口或者查询 API。如：Neo4J, InfoGrid, InfiniteGraph.

图数据库允许我们将数据以图的方式储存。实体会被作为顶点，而实体之间的关系则会被作为边。比如我们有三个实体，Steve Jobs、Apple 和 Next，则会有两个“Founded by”的边将 Apple 和 Next 连接到 Steve Jobs。

产品：Neo4J、Infinite Graph、OrientDB

1. 适用的场景

- 1) 在一些关系性强的数据中
- 2) 推荐引擎。如果我们将数据以图的形式表现，那么将会非常有益于推荐的制定

2. 不适用场景

不适合的数据模型。图数据库的适用范围很小，因为很少有操作涉及到整个图。

3、参考范文：

2017 年系统架构师论文

一、论软件系统建模方法及其应用

1、真题原文：

论软件系统建模方法及其应用

软件系统建模（SoftwareSystemModeling）是软件开发中的重要环节，通过构建软件系统模型可以帮助系统开发人员理解系统、抽取业务过程和管理系统的复杂性，也可以方便各类人员之间的交流。软件系统建模是在系统需求分析和系统实现之间架起的一座桥梁，系统开发人员按照软件系统模型开发出符合设计目标的软件系统，并基于该模型进行软件的维护和改进。

请围绕“论软件系统建模方法及其应用”论题，依次从以下三个方面进行论述。

1. 概要叙述你参与的软件系统开发项目以及你所担任的主要工作。

2. 说明软件系统开发中常用的建模方法有哪几类？阐述每种方法的特点及其适用范围。
3. 详细说明你所参与的软件系统开发项目中，采用了哪些软件系统建模方法，具体实施效果如何。

2、参考理论：

软件设计方法包括有：模型驱动设计，面向对象设计，结构化设计，快速应用开发，原型设计，信息工程法。

1) 模型驱动设计

模型驱动设计是一种系统设计方法，强调**通过绘制图形化系统模型描述系统的技术和实现**。通常从模型驱动分析中开发的**逻辑模型导出系统设计模型**，最终系统设计模型将作为构造和实现新系统的蓝图。

2) 结构化设计

它是一种面向过程的系统设计技术，它将系统过程分解成一个容易实现和维护的计算机程序模块。把一个程序设计成一个自顶向下的模块层次，一个模块就是一组指令：一个程序片段，程序块，子程序或者子过程，这些模块自顶向下按照各种设计规则和设计指南进行开发，模块需要满足高内聚松散耦合的特征。

3) 信息工程法

信息工程是一种用来计划，分析和设计信息系统的模型驱动的，以数据为中心的但对过程敏感的技术。信息工程模型是一些说明和同步系统的数据和过程的图形。信息工程的主要工具就是数据模型图（物理实体关系图）。

4) 原型设计

原型法是一种**反复迭代过程**，它需要设计人员和用户之间保持紧密的工作关系，通过构造一个预期系统的小规模的，不完整的但可以工作的示例来与用户交互设计的结果。原型设计方法鼓励并要求最终用户主动参与，这增加了最终用户对项目的信心和支持。更好地适应用户总是想改变想法的自然情况。原型是主动的模型，最终用户可以看到并与之交互。

5) 面向对象设计

是一种新的设计策略，用于精炼早期面向对象分析阶段确定的对象需求定义，并定义新的与设计相关的对象。面向对象设计是面向对象分析的延伸，有利于消除数据和过程的分离。

（写作参考：在我们惯常的项目开发中，经常会使用各种设计工具来做系统分析，制作

各种设计图并以之为基础和客户进行讨论，经过确认达成一致后作为系统分析的产物。然而一旦进入到开发阶段，开发人员根本不会按照之前的设计来实现，很多时候代码只是实现了功能，大部分实现都经过重新设计，而分析模型则直接被扔在了一边。

导致这样做的一个根本原因，是我们缺乏一套建模范式。它可以采用统一的方式，并可以非常方便的将模型转化为代码设计并可以保持模型与设计之间的映射关系。面向对象设计（Object-Oriented Design）是目前为止公认的可以实现该要求的建模范式。后续文章则将在关注在如何使用面向对象设计来实现领域驱动的相关问题上。

抽象出关键的领域知识（名词定义）并支持有效的实现（设计范式），是模型驱动设计的核心二要素。

开发人员要意识到代码和模型是整个系统的一体两面，改变代码即意味着改变模型。如果忽略了这一点，到最后模型则将失去指导作用，并逐渐变得不可用。同时，建模人员也应花时间去了解代码。双方通过使用统一领域语言保持模型和实现的一致性。）

6) 快速应用开发

是一种系统设计方法，是各种结构化技术（特别是数据驱动的信息工程）与原型化技术和联合应用开发技术的结合，用以加速系统的开发。快速应用开发要求反复的使用结构化技术和原型技术来定义用户的需求并设计最终系统。

3、参考范文：

摘要：

2020年3月，本人就职的某互联网公司承担了“XXAPP 电子商务系统”的开发，该项目是集团为用户提供电子商务业务，拉新促活，提高交易效率的重点项目，主要实现消费者的网上购物和在线电子支付以及各种商务活动、交易活动。本人作为商城部门的核心成员有幸加入其中，担任架构师一职，主要负责该系统的需求分析和架构设计。该系统用户面向广大移动端和PC端互联网用户，主要用于提供用户网上订购、在线支付、广告宣传等服务，具有营销活动商品维护订单报表等服务能力。本文以该项目为例，最终确定本系统采用基于架构的软件设计方法，经过体系结构需求，体系结构设计，体系结构文档化，体系结构复审，体系结构演化等过程，为开发团队从结构上解决了业务难题，系统上线后得到了广大用户和集团对于系统的认可。

正文：

2020 年 3 月，我参与了公司电子商务系统”的开发，担任系统架构师职务，进行系统的架构工作。该系统的外部实体主要是分为 4 类组织，用户，商户，系统管理人员，金融支付机构。电商系统有几个特点，模型多且复杂度高，数据流链路长，业务变化快，应对突发流量性能要求高等特点。包括商品库存、订单支付、营销活动、售后配送、用户权益、商户管理等几大业务模型。随着业务不断迭代之后，每个模型的边界和管理会越来越复杂和逻辑冗余。

考虑到系统的复杂性，项目整体采用软件体系结构设计，指的是从宏观层次对需要开发的目标系统进行描述。用不同的视图从不同的视角对目标系统进行分析与设计的过程。具体的过程可以分为，体系结构需求，体系结构设计，体系结构文档化，体系结构复审，体系结构实现，体系结构的演化等重要过程。

1、体系结构需求

在体系结构需求阶段，本项目先对目标系统的质量目标进行分解并结合用户提出的需求，分别整理出，功能性需求和非功能性需求。架构必须同时满足功能性需求及非功能性需求。通过分析得到以下几个架构核心关注点：1. 数据的存储模型，由于每个商品可以分为普通商品和活动商品等不同类型，活动又分为预热期和活动期，以及周年大促、会员日、节日活动等类型，商户有自营商户和入驻商户，分别对应不同的后台信息管理类型等，订单分为实物、电子票券、酒店门票、充值等等，2. 由于面向广大互联网用户每次商品、订单等查询时间必须小于 2 秒，7*24 小服务，故障恢复时间不能大于 1 小时。3. 本系统是一个交易类系统必须有动态参数设置及监控界面。

2、标识构件

通过分析系统的核心需求，架构组决定采用构件化开发进行整体的架构设计。设计基于模型属性和模型能力的抽象开发方法，设计使用 event 事件作为一个原子，组装和重发事件驱动，设计无锁的队列存储优化操作相同资源的性能。使用消费事件的方式提升开发效率，对物理模型和业务逻辑进行解耦。

3、标识构件之间的关系

根据总体架构设计形成了高层的构件，包括对象及对象状态构件、外部实体行为构件、消息事件构件、服务框架管理构件。业务模型继承和实现这些构件，如此商品，库存，支付，营销，权益，配送等业务模型都统一的具备了状态，行为，事件的能力构件，骨架完成后再填充各自的业务逻辑。

外部实体行为构件：原子的业务逻辑完成后，通过策略选择，迭代模式来配置和组装原子逻辑成为完整的业务流程功能。例如可以通过业务的策略来选择支付方式，也可以配置多条迭代校验规则完成业务上对商品的校验，这些策略和迭代规则都是配置化可以随意拆分组装。

消息事件构件：事件体系也是交易系统内部抽象的模型，它定义唯一的事件名称和唯一的事件 ID，它支持同步或异步执行，支持不同线程上下文信息传递，支持写数据库设定，支持重试次数设定，支持自动的重试触发，支持无锁队列 ringbuffer 存放事件排队，支持多个事件组合成事件组，支持监控报警。以上是这个事件体系的特点。

服务框架管理构件：在下单支付等动作完成后，系统根据状态机映射触发对应的行为，行为中包括了系统服务调用和事件触发。在触发前把事件 id，上下文信息放入 redis 缓存或数据库，事件自动存入无锁队列，系统的事件 handle 自动从队列中拉取对象和逻辑处理。在处理发生异常时，事件根据配置自动存入数据库等待重试或者丢弃。对于处理失败有通用的重试机制和监控报警规则，让开发快速发现线上问题。

4，技术选择

由于主从架构已经在大数据领域成功应用，同时也有成熟的开源组件。通过与客户讨论决定采用 ZooKeeper 为核心注册及主从架构管理组件实现负载。数据存储采用两级存储，MySQL 作为核心数据存储，Redis 数据库作为数据缓存，减少核心数据库的读压力。RabbitMQ 作为消息核心组件。系统间通信采用基于 HTTP 协议的 Restful 接口形式。

5，架构文档化

根据设计意图，架构组采用 4+1 视图模型，对核心业务场景逻辑功能，开发架构，设计架构，部署架构，存储架构进行了描述并形成架构说明文档及需求质量设计说明书初版。

6，架构复审与细化

架构组与客户专家组对架构说明书，质量设计说明书进行了复审，并提出架构应考虑到数据的安全性，建议对数据库中的关键数据的加密，传输过程中的敏感数据采用加密和超时控制。架构组认为，数据库中的核心数据可以采用两种加密分别对不同数据加密。用户密码敏感信息采用 MD5 数字摘要算法进行加密，对于重要的金额数据采用对称加密。首次复审会议后架构组进行了架构的细化，确定了通信协议种类，主要接口，核心业务设计模式。并且对主要核心类进行的属性和方法的识别。再次经过与客户专家组讨论与评估，最终确定了 XXAPP 电商系统的架构。

此架构在后期的实现过程中经历一些调整，如下单改成多线程模式，消息通知失败的重试机制等部分，但总体结构稳定。系统上线后经客户专家组的验收反馈，本系统架构解决了需求中的核心关注点，并取得了很好的效果，关键是可以对系统运算能力进行水平扩展，能满足未来一定时期的业务需要。

结论：

经历 XXAPP 电商系统架构完整过程使我认识到架构不只是技术，而是要用技术解决客户的业务中的核心关注点，遵循一定架构过程和原则才能设计一个好的系统架构。也让我从中学到构件识别技术，比如构件的独立性，无可见状态，接口规约的版本一致性；构件关系组及架构评审相关知识。

二、论软件架构风格

1、真题原文：

论软件架构风格

软件体系结构风格是描述某一特定应用领域中系统组织方式的惯用模式。体系结构风格定义一个系统家族，即一个体系结构定义一个词汇表和一组约束。词汇表中包含一些构件和连接件类型，而这组约束指出系统是如何将这些构件和连接件组合起来的。体系结构风格反应了领域中众多系统所共有的结构和语义特性，并指导如何将各个模块和子系统有效地组织成一个完整的系统。

请围绕“论软件架构风格”论题，依次从以下三个方面进行论述。

1. 概要叙述你参与分析和设计的软件系统开发项目以及你所担任的主要工作。
2. 软件系统开发中常用的软件架构风格有哪些？详细阐述每种风格的具体含义。
3. 详细说明你所参与分析和设计的软件系统是采用什么软件架构风格的，并分析采用该架构风格设计的原因。

2、参考理论：

（注意本部分内容虽然题目要求是详细论述，但实际上不是文章的重心，问题 3 才是结合项目的详细论述部分）

软件架构风格分为五大类，数据流风格、调用/返回风格、独立构件风格、虚拟机风格

和仓库风格。其中：

1) 数据流风格

包括了批处理序列架构风格和管道过滤器架构风格

2) 调用返回风格

包括主程序和子程序架构风格，数据抽象和面向对象架构风格和层次结构架构风格。

3) 独立构件风格

包括进程通信架构风格和事件驱动架构风格

4) 虚拟机风格

包括解释器风格和基于规则的系统

5) 仓库风格

包括了数据库风格和超文本架构风格，黑板架构风格。

其他还有特定领域软件架构，状态转移等以及分布式处理等。其中分布式架构风格中有客户机/服务器风格，浏览器/服务器风格，CORBA，DCOM，EJB。

每一种具体的软件结构风格的模型如下：

批处理序列架构风格

组件为一系列固定顺序的计算单元，组件间只通过数据传递交互。每个处理步骤是一个独立的程序，每一步必须在前一步结束后才能开始，数据必须是完整的，以整体的方式传递。

管道过滤器架构风格

每个构件都有一组输入和输出。构件读输入的数据流，经过内部处理，然后产生输出数据流。这个过程通常通过计算和增加信息丰富数据，通过浓缩和删除精炼数据，通过改变记录方式转化数据，递增地转化数据等。在输入被完全消费之前，输出便产生了。这里的构件被称为过滤器，连接件就是数据流传输的管道，将一个过滤器的输出传到另一个过滤器的输入。

主程序/子程序架构风格

单线程控制，把问题划分为若干处理步骤，构件即为主程序和子程序。子程序通常可合成为模块。过程调用作为交互机制，即充当连接件。调用关系具有层次性，其语义逻辑表现为子程序的正确性取决于它调用的子程序的正确性

面向对象架构风格

数据抽象和面向对象架构风格。这种风格的构件是对象。对象是抽象数据类型的实例。

在抽象数据类型中，数据的表示和它们的相应操作被封装起来。对象的行为体现在其接受和请求的动作。连接件即是对象间交互的方式，对象是通过函数和过程的调用来交互的。对象具有封装性，一个对象的改变不会影响其他对象。对象拥有状态和操作，也有责任维护状态。这种结构风格中包含有封装、交互、多态、集成、重用等特征。

层次结构架构风格

层次系统组织成一个层次结构。构件在一些层实现了虚拟机。连接件通过决定层间如何交互的协议来定义，拓扑约束包括对相邻层间交互的约束。这个风格的特点是每层为上一层提供服务，使用下一层的服务，只能见到与自己邻接的层。大的问题分解为若干个渐进的小问题，逐步解决，隐藏了很多复杂度。修改一层，最多影响两层，而通常只能影响上层。上层必须知道下层的身份，不能调整层次之间的顺序。

进程通信架构风格

构件是独立的过程，连接件是消息传递。这种风格的特点是构件通常是命名过程，消息传递的方式可以是点对点、异步和同步方式、以及远过程调用等。

事件驱动的架构风格

构件不直接调用一个过程，而是触发或广播一个或多个事件。系统中的其他构件中的过程在一个或多个事件中注册，当一个事件被触发，系统自动调用在这个事件中注册的所有过程。一个事件的触发就导致了另一个模块中的过程的调用。这种风格中的构件是非命名的过程，它们之间交互的连接件往往是以过程之间的隐式调用(ImplicitInvocation)来实现的。基于事件隐式调用风格的主要优点是软件重用提供了强大的支持，为构件的维护和演化带来了方便，其缺点是构件放弃了对系统计算的控制。

解释器架构风格

一个解释器通常包括完成解释工作的解释引擎，一个包含将被解释的代码的存储区，一个记录解释引擎当前工作状态的数据结构，以及一个记录源代码被解释执行的进度的数据结构。具有解释器风格的软件中含有一个虚拟机，可以仿真硬件的执行过程和一些关键应用。其缺点是执行效率较低。

基于规则的系统

基于规则的系统包括规则集、规则解释器、规则/数据选择器以及工作内存

数据库架构风格

数据库架构是库风格最常见的形式。构件主要有两大类，一个是**中央共享数据源**，保存

当前系统的数据状态，另一个是**多个独立处理元素**，处理元素对数据元素进行操作。

黑板架构风格

黑板架构包括知识源、黑板、控制三部分。知识源包括若干独立计算的不同单元，提供解决问题的知识，知识源响应黑板上的变化，也只修改黑板。黑板是一个全局数据库，包含解域的全部状态，是知识源互相作用的唯一媒介。知识源响应是通过黑板状态的变化来控制。黑板通常应用在对于解决问题没有确定性算法的系统中，例如信号处理、问题规划、编译器优化等软件系统的设计中

客户/服务器风格

C/S 体系结构有三个主要组成部分：数据库服务器、客户应用程序和网络。

二层 C/S 结构是单一服务器且以局域网为中心的，所以难以扩展至大型企业广域网或 Internet 软、硬件的组合及集成能力有限，客户机的负荷太重，难以管理大量的客户机，系统的性能容易变坏，数据安全性不好。三层 C/S 体系结构是讲应用功能分成表示层、功能层和数据层三个部分，削弱二层 C/S 结构的局限性。

浏览器/服务器风格

浏览器/服务器风格就是三层 C/S 结构的一种实现方式，具体结构为浏览器/Web 服务器/数据库服务器。

C2 风格

C2 体系结构风格可以概括为，通过连接件绑定在一起按照一组规则运作的并行构件网络。

C2 架构风格是一种常见的层次体系架构风格。该架构风格概括而言，是由连接件绑定的按一定规则运行的并行构件网络，在该架构风格中，各构件之间不能直接连接，只能通过连接件的异步通信机制进行交互，使得构件的替换或更新不影响架构，这种方式体现了高内聚，松耦合的设计思想。

C2 风格中的系统组织规则如下：

系统中的构件和连接件都有一个顶部和一个底部；构件的顶部应连接到某连接件的底部，构件的底部则应连接到某连接件的顶部；构件与构件之间的直接连接是不允许的；一个连接件可以和任意数目的其他构件和连接件连接；

当两个连接件进行直接连接时，必须由其中一个的底部到另一个的顶部。

C2 架构的缺点：

效率低，若业务处理涉及多个构件层次，在系统执行过程中将存在性能损耗。

层次不清：难以划分出合适的，正确的层次结构，有时由于需求，需要跨层交互，增加复杂性。

3、参考范文：

范文 1:

— 摘要

2019 年 8 月，我司承接了某市医疗集团，智慧药房项目，该项目主要为集团下属 36 家社区卫生服务中心提供药品统一目录管理、药品集中采购、库存管理、处方合理用药审核、药品配发、自动化发药设备驱动提供软件支撑。在该项目中我担任了软件架构设计师一职，主要负责该项目的软件架构设计工作。本文以智慧药房项目为例，主要论述了**架构风格**在项目中的具体应用，**在处方审核模块中，采用了基于规则的系统风格，将 18 万种药品合理用药规则作为基础规则库供处方审核调用，在自动化发药设备协同中，采用了隐式调用风格，将处方配发任务和设备驱动进行解耦，在药品采购和配发模块中，采用了 B/S+C/S 的混合架构风格，通过使用这些架构风格，提高了软件设计质量和开发效率，最终项目成功上线，并获得了用户一致好评。**

— 背景

根据深化医药卫生体制改革规范药求，推进药品集中采购，增加药品的供应保障能力，严格监督管理，保障药品的用药安全，所有医疗机构开出的处方，必须通过处方审核后，方可进入计价计费环节，未经审核的处方，不得进行计费和配发。2019 年 8 月，我司承接了某医疗集团智慧药房项目，该项目为医疗集团下属 36 家社区卫生服务中心提供软件支持，主要分为药品供应模块、药房管理模块、处方用药审核模块、药品配发模块、设备协同模块。药品供应模块负责药品库存预警，供社区卫生服务中心药房对药品进行集中采购，并将药品发送到省采招平台；药房管理模块主要进行一些基础信息的维护，及关注药品的库存管理，药品批号跟踪；处方用药审核模块，负责对医生开出的药品进行合理用药审核，将不合理的处方审核结果返回给医生，提醒医生修改处方；药口中配发模块，负责将处方中的药品进行调配，打印用法用量标签，确认发药后扣减相应的库存；设备协同模块主要是驱动药房中的一些自动化发药设备，将要药品信息发给设备上位机，根据设备药

品的效期进行排序，通过上位机程序调用下位机，驱动设备出药。在该项目中我担任软件架构设计师职务，主要负责软件的架构设计及中间件等技术的选型工作。

一问题回答，根据提干来

软件架构风格描述了某一特定领域中的系统组织方式和惯用模式，反映了领域中众多系统所共有的结构和语义，并指导如何将各个构件有效地组织成一个完整的系统，提高了架构的复用性，架构风格定义了用于描述系统的术语表和一组指导构建系统的规则，一般分为数据流风格：批处理、管道-过滤器；调用/返回风格：主程序/子程序、面向对象、层次结构；独立构件风格：进程通信、事件驱动（隐式调用）；虚拟机风格：解释器、基于规则的系统；仓库风格：数据库系统、超文本系统、黑板系统。可以为从多项目提供通用的解决方案，能够极大的提高软件设计的重用方法和加快开发的进程。

一简介

在智慧药房项目中，我使用了虚拟机风格、独立构件风格以及 B/S+C/S 混合架构模式；虚拟机风格中的规则系统，能够在预先定义好的规则下，执行符合规则的一些行为，独立构件风格包括进程通信和隐式调用，减少系统的耦合度，提供了系统的可扩展性。B/S 架构风格是基于浏览器和服务器的软件架构，他主要通过 HTTP 协议进行通信和交互，降低了维护升级的成本，C/S 架构风格使得客户端程序能够处理更多的业务逻辑，更容易驱动硬件设备。以下正文将重点描述架构风格的应用实施过程和效果。

一以三个维度阐述

处方模块模块中，采用了虚拟机风格中的基于规则的系统风格，目前现有药品目录有 18 万种，一个患者在不同的时间或不同的科室就诊，每次就诊会开出相应的处方药品，判断这些药品之间是否存在相互作用，给药方式是否正确，是一项巨大的工程，所以在处方审核模块中，我使用了虚拟机风格中的基本规则的风格，《中国药典》、《临床药物治疗合理用药原则》、《配伍禁忌》中对现有 18 万种药品间的合理用药有明确使用规则说明，将这些药品的相互作用做为基础规则库，再加上药品说明书中明确指出的用药规则及药师制定的一些自定义规则，规则数据量较大，因此我们选择了适合处理大数据的非关系型数据库 MongoDB，在医生开出处方后，将处方中的药品本位码和规则库中的规则进行配对，如果发现有存在相互作用、用法用量有误等不合理处方时，及时将审核结果，不合理用药的原因，发送到医生工作站，让医生修改处方，处方审核模拟中使用了基于规则的架构风格，大大提高了药师人工审核的工作效率，保障了患者的用药安全。

在发药设备协同环节，我采用了隐式调用风格，来简化构件间的交互，隐式调用风格的思想是构件不直接调用一个过程，而是触发或广播一个或多个事件，项目中有多种不同的发药设备，如果在发药管理中直接驱动设备出药，增加了应用系统和设备之间的耦合度，降低了扩展性，新增一台发药设备时，不能快速对其进行扩展兼容，因此，项目中采用了开源的MQ消息中间件做边连接构件，这个是Apache基金下的核心开源项目Kafka，配发系统在处方药品调配时，作为生产者，将处方中的药品信息发送给Kafka，发药设备作为消息的消费者，从Kafka中获取发药数据。这种方式相对于同步调用的好处就是，消息发送者将消息发给Kafka后，就可以去处理其它任务了，无需等任务处理结果，发药设备如果在维护情况下，重启了设备，不会因为没收到数据，造成出药失败，只需要在启动后，重新到Kafka中再次消费数据即可，在上文中提到的，处方审核时，碰到不合理的处方通知医生，也同样采用了隐式调用风格。

在药师配发药和发药设备出药环境，我采用了B/S+C/S的混合架构风格，B/S的优点：维护方便，系统功能改造后，只需要服务器端程序更新，即可实现所有的用户同步更新，简化操作，客户端只需要安装浏览器，无需单独安装系统。输入系统服务地址就可以打开系统进行操作。缺点：可操控性不强，客户端不能处理太多的业务逻辑，大都逻辑是由服务端处理完成。所有用户使用的是同一套WEB程序，很难实现个性化的需求定制。相比较而言C/S的个性化定制和操控性要优于B/S，因此，我们在设备上位机上，选择了winform+应用服务器的C/S架构。不同的设备所具备的功能不同的，界面设置、功能也大不相同，C/S很好的满足了个性化定制的需求，而且上位面程序在订阅到Kafka消息时，会调用下位机PLC程序，驱动设备出药，这是Web程序无法实现的。在药房发药窗口电脑上的配发系统，采用了B/S架构，院内业务需求经常发生变化，采用B/S架构能够方便业务功能扩展和及时更新，降低了运维成本。

— 结尾

项目从2019年8月启动到2020年10月历时14个月，圆满完成，顺利完成验收，并取得了客户的一致好评，该项目运行一年多，也出现过一些小的问题，由于医院使用的是集团内部局域网，与外网隔离，给排查问题及维护增加了困难，一次发药设备上的上位机不小心被药房老师删除后，不会重新安装操作，上报我司后，我们联系医院信息科老师，在信息科的老师协助下，对软件重新进行了安装，此次事件，导致了设备停止工作了几个

小时，影响了药房发药效率。后来我司安排了 1 名售后维护该项目，这一年内也新增了一些发药设备，由于选用了合适的架构风格，使得设备的接入及服务的扩展变得非常容易，在售后同事的协助下，系统至今运行稳定。该项目的成功，让我意识到了使用了架构风格的作用和价值，坚定了我对架构风格应用的信心，合理选择合适的架构风格，能够大大的提高了软件设计的复用方法，加快开发的进程，在项目中起到事半功倍的作用。经过这次项目，我也看到了自己身上的不足之处，在未来还会不断地更新知识，完善本系统的设计，使系统能够适应国家医改的变化需要，这是作为软件从业的我为之努力的动力和方向。

范文 2:

摘要

2018 年 3 月我公司承接了某网省电力公司的电力物资仓储管理系统建设，系统主要功能有仓储管理、主数据管理、业务查询等。我在项目中担任架构师一职，负责项目整体架构设计、架构文档编写、架构实现等工作。本文以该项目为例，主要论述了软件架构风格在该项目的具体应用。本系统使用三层 B/S 的架构风格，降低系统耦合度，简化了业务开发，使视图可以更灵活的实现的同时简化了系统部署方式。使用隐式调用的架构风格，提高软件的可重用性，为构件的演化和维护提供方便，提升了系统的可扩展性和业务安全性，简化了用户操作。使用管道/过滤器架构风格，提高业务处理效率，简化业务流程数据的处理，降低了系统测试复杂度。经过将近一年的建设，于 2019 年 3 月所有功能全部上线，系统运行平稳，得到用户一致好评。

正文

随着企业信息化进程的加剧以及国家信息化战略的推进，越来越多的国有企业在逐步推进和完善企业信息化应用。因此网省电力公司为响应国家号召委托我公司为其承建电力物资仓储管理系统以下简称本系统。

本系统于 2018 年 3 月开始，组建包括开发、设计、测试在内的 38 人的开发小组。我在项目中担任架构师一职，负责项目整体架构设计、架构文档编写、架构实现等工作。本系统以主数据管理模块为基础，包括物料主数据管理、仓库主数据管理等功能；以仓储管理模块为核心，包括收货管理、发货管理、仓储作业管理、盘点管理等功能；以业务查询和统计报表为提升点，包括业务查询和报表查询功能；基于以上功能分析搭建了由一台应用服务器和一台数据库服务器构成的测试服务系统。同时随着系统开发进程搭建了由三台应用服务器和

两台数据库服务器构成的生产服务系统。在以上人力、服务器资源的基础上，本系统经过设计、开发、测试等将近一年的工作，于 2019 年 3 月顺利上线，上线后所有功能的实现符合用户的预期，并且运行平稳受到用户的一致好评。

在上述系统背景下我成功将系统架构风格运用到实际项目中并借此保证了系统顺利上线。系统架构风格是描述某一特定应用领域中系统组织方式的惯用模式。软件架构风格反映了领域中众多软件系统所共有的结构和语义特性，并指导如何将各个模块和子系统有效的组织成一个完整的系统。目前大家公认的有五大类架构风格，分别为数据流风格，调用返回风格，独立构件风格，虚拟机风格，仓库风格。其中数据流风格以数据处理为中心，典型的是批处理和管道/过滤器架构风格；调用返回风格适合于业务层次清晰的应用，主要代表为主程序子程序、面向对象、层次化结构；独立构件风格包括进程通信和事件驱动架构风格；虚拟机风格的主要特点为支持各平台运行，主要用于解释器和规则系统；仓库风格代表子分类为数据库系统，黑板系统。其优点为可更改性和可维护性，具备可重用的知识源，容错性和健壮性较好，但是开发难度也较大。

基于以上我对架构风格的理解以及风格特点的评估，本系统分别于系统集成处使用了隐式调用架构风格；于业务流转处使用了管道\过滤器架构风格；于整体架构层面使用了三层 B/S 架构。下文将详细介绍以上三种架构风格在本系统中具体实现方案。

（一）系统集成处隐式调用架构风格的实现及效果

由于隐式调用架构风格以事件驱动为主，构件不直接调用一个过程而是触发或广播一个或多个事件，同时系统中其他构件的过程在一个或多个事件中注册。所以在实际架构设计中我将本系统与 ERP 系统集成处将每种数据交互进行构件封装，将每个数据交互封装成一个小的构件，一个构件仅处理一种数据交互。在最小化业务构件的基础上将多个数据交互构件进行组合由此形成一个个大的业务构件。在业务构件的基础上将业务构件组合封装成事务码，当一个事务码被触发时，事务码调用对应业务构件，业务构件自动执行在业务构件中封装好的一个或多个数据交互构件。通过此类架构风格开发人员只用关注最小化的数据交互封装以及构件组合，而在数据交互构件完成后业务数据就能自动按照封装好的业务构件进行数据流转交互，大大降低了开发的工作量，同时降低了系统的耦合度，提升了系统的扩展性，为软件重用提供强大的支持，为构件的维护和演化带来了方便。

（二）业务流转处管道\过滤器架构风格的实现及效果

管道/过滤器架构风格中每个构件都有一组输入和输出，构件读输入的数据流，经过内部处理，然后产生输出数据流，经过处理，产生输出数据流。本系统中物资从采购、配送、交接、验收、入库、调拨、领料、出库以及后续盘点业务的数据流转都有明显的数据处理流程且数据处理规律及关系明显。因此在架构设计中我们将不同类型的物资业务数据封装成一个个构件，每个构件都根据业务固化一组输入和输出，将构件中数据处理封装为一个个数据处理的子构件，将一个构件加一个数据处理构件进行组合形成一个独立业务处理管道\过滤器组合。将多个管道\过滤器的组合连接起来构成整个业务数据流转体系，再将数据集成处事务码引用到每一段业务处理组合中形成整体的基于事务码加管道\过滤器的整体数据流转机制。在此机制的基础上开发人员只需关注组合构件中输入数据、输出数据、事务码。大大简化了开发的流程，同时也简化了系统的测试任务，使所有业务流程变更做到一次开发永久使用，大大提高了系统的稳定性和可重用性。

（三）整体架构上采用了三层 B/S 架构风格的实现

三层 B/S 架构风格中将系统整体分为三层进行系统开发及部署，三层分别为展现层、业务层、持久层。这样进行拆分后使系统耦合度大大降低，由于展现层和持久层分离，使系统可以在持久层不变的情况下支撑更复杂的视图展现效果；由于业务层和持久层分离，是系统可以灵活的扩展新业务而无需改变数据库模型；同时本系统持久层在实际开发中使用了工厂设计模式将数据库连接和数据访问进行隔离，提高了系统对异构数据的支撑，另外基于浏览器进行系统访问的模式，符合用户日常使用习惯，减少了培训成本。依托客户端浏览器进行产品呈现，简化了推广难度，三层架构使业务的实现更加清晰，代码的耦合度更低，扩展性更好。

基于以上三种软件架构风格在系统中使用，经过团队人员将近一年的努力，本系统于 2019 年 3 月顺利上线。本系统提升了各电力物资仓库的物资流转效率，切实减轻了仓库业务人员的工作压力，更好的支撑了业务的回溯及仓库业务开展，获得了物资公司的嘉奖。通过本项目的研发我对软件架构风格的理解更加深入，为后续的发展打下了更坚实的技术基础。

本项目在实际的研发过程中使用了数据流中管道/过滤器、独立构件中隐式调用的架构风格，在管道/过滤器风格中数据过滤中间件的抽象及定义遇到了各业务规则不一致，数据封装效果不一致的难题，由于仓储作业中收货、发货、转储等不同业务的业务规则不一致，但是数据一致性高，偏差较小，在实际构件的封装中开发人员需要针对每种业务进行单独编写，工作量大且重复度高。经过协商用适配器的设计模式为各种不同的业务规则编写了通用

项目于 2019 年 4 月上线。

系统能保质保量上线,质量保证不可缺少,所以开发过程中是尤为重视。根据项目实践,质量保证工作分为这么几部分:计划,审计,检查,跟踪,报告。计划阶段就是决定怎么做,谁来做,什么时间做等问题;审计则是对计划,报告,以及策略做评审,通过后才可以执行;检查就是对评审过的计划执行情况的 Check;跟踪就是对一些变更或者问题进行跟踪防止遗漏;报告是我对前面工作的总结和回顾,提取经验,反馈的下次迭代的流程。对于具体某个工作,上述几个阶段是串行的,但对于不同的点阶段可能是交叉的。同时结合实际项目敏捷的开发方法,这几个阶段也采用迭代的思想,不断反馈,优化流程。

实际项目中,这些阶段间隔是模糊的,在不同知识域他们迭代的阶段是不同的。其中审计包括两部分,即计划的审计和报告的审计,工作中一般都是直接在计划讨论完成后直接进行评审,或者 Review 介绍后直接进行评审,同时对于检查和跟踪也是一个人来做。所以这里就将质量保证分为三部分:计划,检查,总结。下面就结合项目实践从质量保证的三部分内容展开论述。

一. 保证计划:这里的计划包括计划和对计划的评审,其中计划决定做什么,怎么做,谁来做,什么时间做。质量保证保证的是软件的质量,软件的质量包括代码质量,文档质量,以及影响软件质量的流程。代码质量方面,在开发前就协同架构部和组内的一些高级工程师指定了代码的编码规范和编码规则,比如:命名,流程控制,注释,异常处理以及数据库 SQL 脚本的编写方法。文档方面,通过引入 CMMI 的一些文档模板,解决日常开发工作中文档模板,技术性的文档采用 wiki 进行管理,开发是前后端分离的开发思路,采用可以共享的 API 工具来管理。相对于代码质量和文档质量,流程贯穿于所有阶段的,也是涉及各个角色,不同关注点就有不同的流程。项目中涉及到比较重要的流程是代码管理流程,该流程包括分支管理,前后端联调,模块集成,持续发版,发版的 SQL 加班管理等串联而成。如果说流程是一条河,做什么就是河流的宽度,怎么做,谁来做,什么时间做,就是这条河流随着时间维度中不同阶段由不同的人来执行操作,因为不同的侧重,负责人,时间节点和方法论都不同,这里不做赘述,下面详细描述。

二. 过程检查:就是责任人根据计划来进行执行和对执行结果的检查,如果检查出来问题也需要进行评审。在文档管理方面,接口文档的正确和实时性很重要,如果采用传统的 wiki 进行管理,不仅仅是开发人员负担,同时在有变更的时候难免出现忘记和遗漏的情况,造成较大的沟通成本,后来采用了一个类似 PostMan 的互联网产品进行 API 管理,该产品可

码管理系统”。改系统采用 B/S 架构，是基于条码管理的，涵盖该厂商整个生产过程的综合性管理系统。改系统需要覆盖该厂商分布在全国各地的生产基地和生产数据的实时采集与记录，实现产品的可追溯性；二是通过在生产过程中对条码的扫描核对，实现对生产过程的规范控制，如批次控制先进先出控制等等；三是基于条码技术，实现物料防混，避免生产、发货过程物料不一致的情况；四是通过的系统中积累数据的分析，制定更有效的经营决策。该项目要求出于要为分布在全国的多个生产基地和供货仓库提供服务，对性能要求较高，另外该厂采用 24 小时 3 班倒工作制度，系统不能出现停机的情况，要就要求系统有较高的可用性。

作为该系统的架构负责人，本人主要负责架构主体设计和架构评估等工作。在该系统架构主体设计完成后，我们开始着手采用架构权衡分析法进行架构评估。软件系统架构评估所普遍关注的质量属性包括性能、可用性、安全性、可修改性、可测试性、可靠性、易用性等。其中前四个质量属性是质量效用树的重要组成部分。性能指系统的响应能力，就是系统要经过多长时间对某个事件进行相应。可用性是指系统能够正常运行的时间比例。安全性实指系统在向合法用户提供服务的同时，阻止非法用户的使用或拒绝向其提供服务。可修改性是指能够快速的以较低代价对系统进行变更的能力。可测试性是指系统发生故障以后，定位其故障的能力和进行测试的能力。可靠性是指系统在发生问题后，维持系统功能的能力。易用性是指用户使用该系统完成所期望功能的难易程度。

在使用架构权衡分析法（ATAM）进行系统架构评估时，我们首先成立了架构评估小组，评估小组主要包括关键用户、评估负责人、架构设计人员项目负责人、需求人员、开发人员、测试人员以及实施人员。小组按照架构权衡分析方法的描述和介绍、调查和分析、测试、报告这四个阶段进行架构评估工作。

在描述和介绍阶段，评估负责人首先描述了架构权衡分析法的执行方法与具体过程，架构权衡分析法是一种基于场景的架构评估方法，对系统的多个质量属性基于场景评估。评估后确定架构的各项指标是否满足需求。之后对关键客户描述的建设该系统的业务动机，该系统的首要目标是为其产品提供条码追溯能力，其次为了提高该厂商整个生产过程的自动化程度。并为决策者提供决策依据。最后我昨晚系统的架构负责人项目详细描述该系统采用面向对象和层次结构架构风格的原因，采用集群部署的原因以及各组件所采用的技术和实现的功能。

在调查和分析阶段，我作为架构负责人进一步对架构进行了拆解和说明。然后各方基于

各自的考虑提出了需求，我们将需要进行整理随之产生质量效用树。质量效用树中主要包括了四个质量属性。分别是可用性、安全性、性能和可修改性。比如在性能方面，客户提出分别在全国多个位置的生产基地和供货仓库访问该系统页面的延迟不能超过 3 秒；又比如在可用性方面，客户提出生产基地采用 3 班倒 24 小时工作生产制度，要求系统必须高可用，不能出现宕机等停止服务的情况。再比如在安全性方面，客户提出该系统存储传输的数据涉及商业机密，必须进行加密传输，保证信息不外泄。

在测试阶段，我们开始通过投票的方式对质量效用树进行优先级排序，然后安装高优先级优先的原则分析架构是否满足各项指标。比如在性能和可靠性方面，系统采用了集群的部署方式，这样首先提高了系统的吞吐量，其次当有任何一台机器发生问题，也不会影响整个系统提供服务。但是客户提出性能方面是重点，不能因为性能问题影响生产，于是我们决定引入 Redis 内存数据库进一步提升性能。在安全性方面，我们权衡了性能和安全性的因素，对传输数据采用了对称式和非对称式结合的加密技术。这样即满足了加密的强度，又保障了加密解密的效率，客户较为认可。

在报告阶段，我们最终形成了描述评估结果，其中详细描述了架构是否满足质量效用树的各项指标，也包含了架构中的风险点、敏感点与权衡点。最后报告中也给出了架构调整意见，比如为提高性能需要引入 Redis 内存数据库。报告和评估期间的会议记录最终进行了归档。

通过架构权衡分析法进行架构评估，不仅准确的获取了系统对各项质量属性的需求，也成功的完成了对架构的评估工作，为架构的优化提供了依据。该项目成功的在规定日期上线，截至目前一直平稳运行，得到了客户的充分肯定，也在相关行业内赢得了良好的口碑。当然我们在架构评估方面还有很多的不足，比如我们目前只掌握了 ATAM 方式的评估方法，对其他的架构评估方式如软件架构分析方法（SAAM）的使用上还有不足，我们将在后续的工作与实践继续磨练与提升。

二、论软件设计模式及其应用

1、真题原文：

论软件设计模式及其应用

软件设计模式（SoftwareDesignPattern）是一套被反复使用的、多数人知晓的、经过分类

Bridge（桥接）

Composite（组合）

Decorator（装饰）

Faqade（外观）

Flyweight（享元）

Proxy（代理）

行为型模式

关键字：对象之间职责的分配

该类模式主要用于对象之间的职责及提供的服务的分配，它不仅描述对象或类的模式，还描述它们之间的通信模式，特别是描述一组对等的对象怎样相互协作以完成其中任一对象都无法单独完成的任务

所包括的模式：

ChainofResponsibility（职责链）、Command（命令）

Interpreter（解释器）

Iterator（迭代器）

Mediator（中介者）

Memento（备忘录）

Observer（观察者）

State（状态）

Strategy（策略）

TemplateMethod（模板方法）

Visitor（访问者）

3、参考范文：

范文 1：

一 摘要

2019 年 8 月，我司承接了某市医疗集团，智慧药房项目，该项目主要为集团下属 36 家社区卫生服务中心提供药品统一目录管理、药品集中采购、库存管理、处方合理用药审

在该项目中，我们综合使用了多种设计模式，本文着重对工厂设计模式、策略模式、适配器模式等 3 种模式在该项目中的具体应用进行介绍。

一、工厂设计模式

在该后台管理平台中，需要接入不同的供应商，有些供应商的数据来源不统一，为了兼容不同的数据来源，我们定义了一个统一的操作数据库的接口，然后根据数据库的不同，由类工厂来决定实例化哪个类。在具体类中实现特定的数据库访问类。这样，就可以实现由客户端指定或根据配置文件来选择访问不同的数据库，从而实现应用程序与数据库无关。通过工厂设计模式的引入，可以有效解决客户需要变化对设计的影响，设计者无需知道哪个子类会被实例化，子类会根据具体情况自己决定实例化哪一个类，符合高内聚、低耦合的设计原则。当有新的供应商加入时，只要知道他的数据来源，调用一下相应的子类即可获取到他相应的数据。

二、策略模式

在系统的安全性方面，我们采用了用户名-密码-自动验证码相结合的办法，以保证系统访问安全性。根据验证码的使用环境，一般分为数字验证码、汉字验证码、英文验证码 3 种类型。而生成不同类型验证码的算法存在巨大差异，为此需要定义不同的生成验证码的算法。为解决此问题，可以利用策略模式将不同的算法封装起来，并使他们可以相互替换，使得算法独立于使用它的客户而变化。在设计策略模式中，我们定义了 3 个角色。环境角色：持有一个抽象策略角色 StrategyVerfyCode 接口的引用，并通过 StrategyVerfyCode 接口，来实现一个具体的策略算法。抽象策略角色：定义所有具体策略类所需的统一访问接口；具体抽象角色：包装了相关的算法或行为。在该项目中，我们按照数字、文字、字符 3 种类型，分别定义了 ShuziVerfy、ZifuVerfy、WenziVerfy 3 个具体的策略类。通过使用策略算法，将生成验证码的算法封装在一个个独立的策略类中，用户可以根据自己的需求从不同策略中进行选择，有效的避免了使用条件转移语句不易维护的缺点。而且策略模式利用组合代替继承，将算法的实现与算法的选择分离开来，降低了程序之间的耦合度，增强了代码的可扩展性和可维护性。

三、适配器模式

在后台管理平台中，考虑到该公司需要频繁的变更供应商，且每个供应商根据合同的谈判结果会得到不同的税率计算，为了尽可能少地在系统中修改或创建新类，我们使用了适配器模式。具体的实现方式是，增加一个类作为适配器，转换类的接口到客户端类期望的另一

分库后的单库做外键约束，就要求分库后每个单库的数据是内聚的，否则就只能靠应用层的判断、容错等方式了。

（3）合并查询

这个场景和前面的跨库 Join 还不同，跨库 Join 是在不同的逻辑表之间的 Join，在分库后这些 Join 可能需要跨多个数据库，而合并查询是针对一个逻辑表的查询操作，但因为物理上分到了多个库多个表，因而产生了数据的合并查询。

考虑方面：

1：排序：多个数据源查询出来后，在应用层进行排序

2：函数处理：即使用 Max、Min、Sum、Count 等函数对多个数据来源的值进行相应的函数处理。

3：求平均值，从多个数据来源进行查询时，需要把 SQL 改为查询 Sum 和 Count，然后对多个数据来源的 Sum 求和、Count 求和后，计算平均值，这是需要注意的地方。

4：非排序分页，这需要看具体实现所采取的策略，是同等步长地在多个数据源上分页处理，还是同等比例地分页处理。同等步长的意思是，分页的每页中，

来自不同数据源的记录数是一样的；同等比例的意思是，分页的每页中，来自不同数据源的数据数占这个数据源符合条件的数据总数的比例是一样的。

（六）数据访问层的设计与实现

数据访问层就是方便应用进行数据读/写访问的抽象层，我们在这个层上解决各个应用通用的访问数据库的问题。在分布式系统中，我们也把数据访问层称为分布式数据访问层，有时也简称为数据层。

（1）对外提供数据访问层的方式

第一种方式是为用户提供专有 API，不过这种方式是不推荐的，它的通用性很差，甚至可以说没有通用性。一般来说采用这种方式是为了便于实现功能，或者这种方式对一些通用接口方式有比较大的改动和扩展。

第二种方式是通用的方式。在 Java 应用中一般是通过 JDBC 方式连接数据库，数据层自身可以作为一个 JDBC 的实现，也就是暴露出 JDBC 的接口给应用，这时应用的使用成本就很低了，和使用远程数据库的 JDBC 驱动的方式是一样的，迁移成本也非常低。

还有一种方式是基于 ORM 或类 ORM 接口的方式，可以说这种方式介于上面两种方式之间。应用为了开发的高效和便捷，在使用数据库时一般会使用 ORM

一服务可以独立于其背后的技术，绑定到任何业务应用中去。这允许业务应用改变业务逻辑时，不用更改任何代码。这业务目标可以通过 workflow 模型控制。

Imixs 的微服务是基于 Imixs 的工作流引擎（Imixs-Workflow Engine）的复杂功能构建的，它可以以多种不同的方法来控制业务数据。Imixs 的微服务可以发送电子邮件推送消息、日志业务交换，还可以确保所有类型业务数据的安全。

Imixs 的工作流模型可以给业务处理模型（Imixs-Workflow Modeller）中的每种状态单独的设计一个 ACL。这许可了高度复杂的业务应用程序，并在每个流程实例周围驻起了安全层。

3、参考范文：

范文 1：

1. 摘要

去年十月，我所在的公司组织了某部委工具中心项目的开发。该项目是以工具为单元对自动化运维工具集合进行管理的应用。工具中心为两级部署，一级工具中心作为其统一工具仓库，是各二级工具中心通用工具权威源，负责其通用工具的准入、全生命周期管理；二级工具中心为各省（市）工具仓库，存放来自一级工具中心的通用工具，负责各省（市）自建工具的准入、全生命周期管理，并作为工具最终运行平台，对外提供工具调用及监控服务。

在该项目中，我担任了架构师和项目负责人的职位，负责项目的架构评审、改造设计及实现。本文结合作者的实践，以工具中心为例，论述微服务架构及其应用。首先概述我参与管理和开发，并采用微服务架构开发的工作，然后具体描述微服务架构的特点，最后结合项目描述软件的架构，说明该系统是如何采用微服务架构模式的，并说明采用微服务架构模式后，在软件开发过程中遇到的实际问题和解决方案。经过项目组近一年的努力，本产品已顺利开发完成，目前已上线使用，取得客户和公司领导的一致好评。

2. 正文

近年来，随着互联网行业的迅猛发展，公司或组织业务的不断扩张，需求的快速变化以及用户量的不断增加，传统的单块（Monolithic）软件架构面临着越来越多的挑战，已逐渐无法适应互联网时代对软件的要求。在这一背景下，微服务架构模式（Microservice Architecture Pattern）逐渐流行。它强调将单一业务功能开发成微服务的形式，每个微服务运行在一个进程中；采用 HTTP 等通信协议和轻量级 API 实现微服务之间的协作与通信。

独立运行，还可能采用多种语言环境，这将导致资源开销大；二代部分码重复，某些底层功能需要被多个服务所用；第三个问题是难以可视化及全面测试，在动态环境下服务间的交互会产生非常微妙的行为。因此，首先服务划分应尽量合理，不要划分太细太多，其次采用公共模块的方式提供底层服务，再次微服务可通过监控发现生产环境的异常，进而快速回滚，弥补可测性不足的问题。

通过项目实践证明，实施微服务架构收益会大于成本。在拥抱微服务之前，也需要认清它所带来的挑战。需要避免为了“微服务”而“微服务”。对传统企业而言，开始时可以考虑引入部分合适的微服务架构原则，对已有系统进行改造或新建微服务应用，逐步探索及积累微服务架构经验，而非全盘实施微服务架构。

范文 2:

摘要

2016 年 7 月，我所在的公司为***系统的产品，我担任系统架构师职务，主要负责软件架构和安全体系设计的工作，该项目是基于互联网，为单位、企业和个人等公众群体提供 7*24 小时的查询申请服务，同时兼顾行***宣传。本文结合作者的实践，以行贿犯罪档案互联网查询系统为例，论述微服务架构及其应用。首先概述我参与管理和开发，并采用微服务架构开发的工作，然后具体描述微服务架构的特点，最后结合项目描述软件的架构，说明该系统是如何采用微服务架构模式的，并说明采用微服务架构模式后，在软件开发过程中遇到的实际问题和解决方案。经过项目组近一年的努力，本产品已顺利开发完成，目前，已在***等多省上线使用，取得客户和公司领导的一致好评。

正文

近年来，随着互联网行业的迅猛发展，公司或组织业务的不断扩张，需求的快速变化以及用户量的不断增加，传统的单块（Monolithic）软件架构面临着越来越多的挑战，已逐渐无法适应互联网时代对软件的要求。在这一背景下，微服务架构模式（MicroserviceArchitecturePattern）逐渐流行。它强调将单一业务功能开发成微服务的形式，每个微服务运行在一个进程中；采用 HTTP 等通信协议和轻量级 API 实现微服务之间的协作与通信。这些微服务可以使用不同的开发语言以及不同数据存储技术，能够通过自动化部署工具独立发布，并保持最低限制的集中式管理。2015 年 7 月，我所在的公司为***的产品，我担任系统架构师职务，主要负责软件架构和安全体系设计的工作。本文结合作者的实践，论述微服务架构及其应用。首先概述我参与管理和开发，并采用微服务架构开发的工作，

被称为过滤器，连接件就是数据流传输的管道，将一个过滤器的输出传到另一个过滤器的输入。

主程序/子程序架构风格

单线程控制，把问题划分为若干处理步骤，构件即为主程序和子程序。子程序通常可合成为模块。过程调用作为交互机制，即充当连接件。调用关系具有层次性，其语义逻辑表现为子程序的正确性取决于它调用的子程序的正确性

面向对象架构风格

数据抽象和面向对象架构风格。这种风格的构件是对象。对象是抽象数据类型的实例。在抽象数据类型中，数据的表示和它们的相应操作被封装起来。对象的行为体现在其接受和请求的动作。连接件即是对象间交互的方式，对象是通过函数和过程的调用来交互的。对象具有封装性，一个对象的改变不会影响其他对象。对象拥有状态和操作，也有责任维护状态。这种结构风格中包含有封装、交互、多态、集成、重用等特征。

层次结构架构风格

层次系统组织成一个层次结构。构件在一些层实现了虚拟机。连接件通过决定层间如何交互的协议来定义，拓扑约束包括对相邻层间交互的约束。这个风格的特点是每层为上一层提供服务，使用下一层的服务，只能见到与自己邻接的层。大的问题分解为若干个渐进的小问题，逐步解决，隐藏了很多复杂度。修改一层，最多影响两层，而通常只能影响上层。上层必须知道下层的身份，不能调整层次之间的顺序。

进程通信架构风格

构件是独立的过程，连接件是消息传递。这种风格的特点是构件通常是命名过程，消息传递的方式可以是点对点、异步和同步方式、以及远过程调用等。

事件驱动的架构风格

构件不直接调用一个过程，而是触发或广播一个或多个事件。系统中的其他构件中的过程在一个或多个事件中注册，当一个事件被触发，系统自动调用在这个事件中注册的所有过程。一个事件的触发就导致了另一个模块中的过程的调用。这种风格中的构件是非命名的过程，它们之间交互的连接件往往是以过程之间的隐式调用(ImplicitInvocation)来实现的。基于事件隐式调用风格的主要优点是软件重用提供了强大的支持，为构件的维护和演化带来了方便，其缺点是构件放弃了对系统计算的控制。

解释器架构风格

一个解释器通常包括完成解释工作的解释引擎，一个包含将被解释的代码的存储区，一个记录解释引擎当前工作状态的数据结构，以及一个记录源代码被解释执行的进度的数据结构。具有解释器风格的软件中含有一个虚拟机，可以仿真硬件的执行过程和一些关键应用。其缺点是执行效率较低。

基于规则的系统

基于规则的系统包括规则集、规则解释器、规则/数据选择器以及工作内存

数据库架构风格

数据库架构是库风格最常见的形式。构件主要有两大类，一个是**中央共享数据源**，保存**当前系统的数据状态**，另一个是**多个独立处理元素**，处理元素对数据元素进行操作。

黑板架构风格

黑板架构包括知识源、黑板、控制三部分。知识源包括若干独立计算的不同单元，提供解决问题的知识，知识源响应黑板上的变化，也只修改黑板。黑板是一个全局数据库，包含解域的全部状态，是知识源互相作用的唯一媒介。知识源响应是通过黑板状态的变化来控制。黑板通常应用在对于解决问题没有确定性算法的系统中，例如信号处理、问题规划、编译器优化等软件系统的设计中

客户/服务器风格

C/S 体系结构有三个主要组成部分：数据库服务器、客户应用程序和网络。

二层 C/S 结构是单一服务器且以局域网为中心的，所以难以扩展至大型企业广域网或 Internet 软、硬件的组合及集成能力有限，客户机的负荷太重，难以管理大量的客户机，系统的性能容易变坏，数据安全性不好。三层 C/S 体系结构是将应用功能分成表示层、功能层和数据层三个部分，削弱二层 C/S 结构的局限性。

浏览器/服务器风格

浏览器/服务器风格就是三层 C/S 结构的一种实现方式，具体结构为浏览器/Web 服务器/数据库服务器。

C2 风格

C2 体系结构风格可以概括为，通过连接件绑定在一起按照一组规则运作的并行构件网络。

C2 架构风格是一种常见的层次体系架构风格。该架构风格概括而言，是由连接件绑定的按一定规则运行的并行构件网络，在该架构风格中，各构件之间不能直接连接，只能通过

连接件的异步通信机制进行交互，使得构件的替换或更新不影响架构，这种方式体现了高内聚，松耦合的设计思想。

C2 风格中的系统组织规则如下：

系统中的构件和连接件都有一个顶部和一个底部；构件的顶部应连接到某连接件的底部，构件的底部则应连接到某连接件的顶部；构件与构件之间的直接连接是不允许的；一个连接件可以和任意数目的其他构件和连接件连接；

当两个连接件进行直接连接时，必须由其中一个的底部到另一个的顶部。

C2 架构的缺点：

效率低，若业务处理涉及多个构件层次，在系统执行过程中将存在性能损耗。

层次不清：难以划分出合适的，正确的层次结构，有时由于需求，需要跨层交互，增加复杂性。

3、参考范文：

摘要

2018 年 3 月我公司承接了某网省电力公司的电力物资仓储管理系统建设，系统主要功能有仓储管理、主数据管理、业务查询等。我在项目中担任架构师一职，负责项目整体架构设计、架构文档编写、架构实现等工作。本文以该项目为例，主要论述了软件架构风格在该项目的具体应用。本系统使用三层 B/S 的架构风格，降低系统耦合度，简化了业务开发，使视图可以更灵活的实现的同时简化了系统部署方式。使用隐式调用的架构风格，提高软件的可重用性，为构件的演化和维护提供方便，提升了系统的可扩展性和业务安全性，简化了用户操作。使用管道/过滤器架构风格，提高业务处理效率，简化业务流程数据的处理，降低了系统测试复杂度。经过将近一年的建设，于 2019 年 3 月所有功能全部上线，系统运行平稳，得到用户一致好评。

正文

随着企业信息化进程的加剧以及国家信息化战略的推进，越来越多的国有企业在逐步推进和完善企业信息化应用。因此网省电力公司为响应国家号召委托我公司为其承建电力物资仓储管理系统以下简称本系统。

本系统于 2018 年 3 月开始，组建包括开发、设计、测试在内的 38 人的开发小组。我在项目中担任架构师一职，负责项目整体架构设计、架构文档编写、架构实现等工作。本系统

二. 版本控制活动

版本控制活动主要包括定义需求文档的版本格式、制订需求文档的修改模式和确定需求文档版本等三项工作。

三. 需求跟踪

需求跟踪活动主要包括定义对其他需求的跟踪能力（联系）链和定义和编制每个需求同系统元素之间的联系文档等两项工作。

四. 需求状态跟踪

需求状态跟踪活动主要包括定义需求状态和跟踪需求每一个状态等两项工作。

变更控制流程：

变更控制的工作程序依次为：提出与接受变更申请、对变更初审、变更方案论证、项目变更控制委员会审查、发出变更通知并开始实施、变更实施的监控、变更效果的评估、判断发生变更后的项目是否已纳入正常轨道。

提出与接受变更申请

提出变更申请应当及时以正式方式进行，并留下书面记录。变更的提出可以是各种形式，但在评估前应以书面形式提出。

对变更的初审

变更初审的目的是为了对变更提出方施加影响，确认变更的必要性，确保变更是有价值的；进行格式校验，完整性校验，确保评估所需信息准备充分；在干系人间就提出供评估的变更信息达成共识。

变更方案论证

变更方案的主要作用，首先是对变更请求是否可实现进行论证，如果可能实现，则将变更请求由技术要求转化为资源需求，以供 CCB 决策。常见的方案内容包括技术评估和经济评估，前者评估需求如何转化为成果，后者评估价值和风险。

项目变更控制委员会审查

审查过程由项目所有者据变更申请及评估方案，决定是否批准变更。审查通常是文档会签形式，重大的变更审查可以包括正式会议形式。审查过程应注意分工，项目投资人虽有最终的决策权，但通常在专业技术上并非强项。所以应当在评审过程中将专业评审、经济评审分开，对涉及项目目标和交付成果的变更，客户的意见应放在核心位置。

发出变更通知并开始实施

因此顺利中标承建了该项目，初期投入 15 名技术人员由我担任架构师职务，负责软件架构设计和中间件选型。

项目启动后通过对业务的梳理，初步将一体化平台划分为四大模块，其中政务服务模块针对的是通过互联网访问系统的企业用户，在功能上主要包括报审、送检、公示和政策法规宣传等功能；而其它三个模块针对的是通过电子政务外网访问系统的监管部门和事业单位用户，其中运行管理模块主要功能包括审批、勘验、颁证、建档；企业监管模块主要功能包括巡检、执法；电子监察模块主要功能包括实时监控、预警纠错和效能评估。

【回应论点二】

在架构设计开始阶段我意识到非功能性需求对架构设计的重要性。虽然在需求分析时功能性需求是客户普遍关注焦点，但作为架构师绝不能忽视对非功能性需求的分析，它与系统的总体特性相关，从质量属性和条件约束两个方面同时对架构设计产生影响。在架构设计中普遍关注的质量属性包含性能、可用性、安全性、可修改性、可靠性、易用性、可测试性等，其中前 4 项是质量属性效应树的重要组成部分，通过质量属性效应树又可进一步分析出架构设计中的敏感点、权衡点和问题决策带来的风险隐患。而条件约束主要来源于用户的限制包括了预算、机构政策和其它软硬件交互的外部因素，例如遵守什么样的行业标准、使用哪个厂商的产品、采用哪种开发技术等。下面从实施性能、可用性、安全性、可修改性四个质量属性和过程中受到的条件约束两方面，详细说明如何通过架构设计来满足非功能需求。

【回应论点三】

一、性能设计，性能是指系统响应能力，包括速度、吞吐量和持续高速性三方面的要求。针对性能方客户提出第一，系统应具有快速的响应能力，在网络负载正常情况下打开界面的平均响应时间小于 1.5 秒，在线查询和提交操作的处理时间小于 2 秒；第二，系统应能承受每年 11 月底全省企业集中年审带来的负载压力。对于性能需求在架构设计中可选的方案有优先级队列、增加计算资源、减少计算开销、引入并发机制、采用资源调度等。本次架构设计上我采用了优先级队列和增加计算资源两个方案，首先采用 RabbitMQ 消息组件使系统以消息为中心进行交互，可保证请求的处理顺序，特别是在高并发情况下可解决流量削峰问题。然后在数据层使用了读写分离技术，采用 Memcache 作为查询数据库，分担主库负载提高查询速度。但在接下来的主库设计上受到了用户方来自政策的约束，由于近年来政府部门推行国产化，新建系统必须使用国产数据库，经过对比分析最后选择了在政府部门应用较广的达梦数据库作为主库使用。

二、可用性设计，可用性是指系统能正常运行的时间比。针对这个方面客户提出第一，必须保证系统 7*24 小时持续提高服务的能力；第二，在网络失效或主机断电的情况下，系统必须保证在 3 分钟内启用备用设备恢复正常服务。对可用性需求在架构设计中可选的方案有心跳、Ping/Echo、冗余、进程监视器。本次架构设计上我采用了心跳技术和冗余技术，使用 keepalived 组件构件双机热备环境，用户通过虚拟 IP 访问，当 Master 宕机自动切换到 Backup 继续运行，防止因为单点故障导致系统瘫痪。但接下来在与信息中心沟通时受到了硬件资源上的约束，因服务器资源紧张部署策略不得不由双机热备后改为双机互备，在性能和可用性上求得了平衡。

三、安全性设计，安全性是指软件系统可向合法用户提供服务，以及阻止非授权使用服务的能力。针对这个方面用户提出必须保证企业用户通过互联网访问系统的安全防止欺诈和篡改。对安全性需求在架构设计中可选的方案有用户认证，用户授权，追踪审计，限制访问。本次架构设计上我采用了用户认证技术，使用 SSL 证书+Ukey 双向认证方式，只须插入 Ukey 即可登录系统，每个 Ukey 在服务端均与用户身份进行了绑定。但在方案评审时受到了用户方预算上的约束，由于因 CA 每年会向单个 Ukey 设备收取 200 元左右的年费，全省企业将是一笔巨款，因此经过折中确定为仅向大型国有企业发放 Ukey，并且费用自理，以节省资金预算。对于普通中小型企业采用了服务端 SSL 证书+客户端动态验证方式。

四、可修改性设计，可修改性是指能快速地并以较高性价比对系统进行变更的能力。针对这个方面用户提出近年来国家在不断推进政府改革工作，系统应该能够适应不断变化的业务环境，可快速的实现业务的新增、修改。对可修改性需求在架构设计中可选的方案有抽象、信息隐藏、限制通信路径、运行时注册。本次架构设计上我采用 Java 语言作为主开发语言，保证程序的抽象和信息隐藏，同时使用 Spring 框架管理 JavaBean 运行时注册，最后在架构上采用了服务化设计，将系统拆分成若干独立服务相互间通过消息进行通信，降低系统耦合度。但在设计与异构系统通信时，受到了来自用户现行技术标准上的约束，由于已投入使用的部分旧系统仅支持 webservice 技术，因此一体化平台在只能采用相同的方式与这些系统进行交互。

【结尾】

实践证明在架构设计中通过对非功能性需求的满足，保证了系统在质量属性和约束条件上与客户要求保持一致，使开发工作取得了成功。但项目过程中也遇到过问题，因客户单位部分用户派驻到办事大厅工作，窗口办公需要配合高拍仪、读卡器等硬件设备，而浏览器

知识，完善本系统的设计，使系统能够适应国家医改的变化需要，这是作为软件从业的我为之努力的动力和方向。

范文 2:

【摘要】

2017 年 5 月，我参加了某省质量技术监督局“生产制造一体化监管平台”项目（以下简称一体化平台），担任架构师职务负责整体架构设计与中间件选型。该系统以监管企业生产制造为主线，旨在帮助各级监督部门解决监管企业生产、报审、送检、勘验、巡检和执法等问题。本文以一体化平台为例，论述了软件的可靠性设计在项目中的应用，并阐述了选择检错技术、容错技术和降低复杂度技术的原因、实施过程和效果。最后总结了在开发过程中遇到的问题及解决方式。实践证明通过合理的可靠性设计及相关技术的应用，使开发工作取得了成功，一体化平台于 2018 年 10 月验收上线，目前已稳定运行 1 年时间得到了各级监管部门和企业用户的一致认可和好评。

【背景】

质量技术监督在生产领域、市场流通领域等方面都起到至关重要的作用，关乎到企业生产和人民生活。某省质量技术监督局在信息化方面起步较早，先后建设了数套专项系统，但缺乏对业务的统一监管，为贯彻落实国家市场监督管理总局《关于加快推进质量体系监管信息化建设要求》，一体化平台的建设工作受到各级领导的关注。项目建设目标覆盖省、市、县区三级质量监管部门及全省三十多万家企业，为工业生产、报审、送检、勘验、巡检和执法工作提供支持。我所在公司长期从事相关行业的软件开发，对质量监督领域有着丰富经验，因此顺利中标承建了该项目，初期投入 15 名技术人员分成三个组，由我担任架构师职务，负责软件架构设计和中间件选型。

项目启动后通过对业务的梳理，初步将一体化平台划分为四大模块，其中政务服务模块针对的是通过互联网访问系统的企业用户，在功能上主要包括报审、送检、公示和政策法规宣传等功能；而其它三个模块针对的是通过电子政务外网访问系统的监管部门和事业单位用户，其中运行管理模块主要功能包括审批、勘验、颁证、建档；企业监管模块主要功能包括巡检、执法；电子监察模块主要功能包括实时监控、预警纠错和效能评估。

【论点】

在架构设计开始阶段，我意识到可靠性设计对架构设计的重要性，因此在基本需求确定

后，我邀请了客户代表和领域专家，针对一体化平台可靠性设计进行了论证。系统建成后要承担起全省制造型企业从生产、送检、勘验到年审等各环节的监管工作，一旦出现故障将造成重大的经济损失和社会性事件，因此必须保证系统 7*24 小时稳定运行，常用 MTTF、MTBF 和 MTTR 指标进行衡量。根据经验保障大型复杂系统的可靠性并不能单纯靠一种或几种技术措施就能做好，而是需要从预防、检查和处理三方面来共同保障，对应架构设计则是降低复杂度技术、检错技术、容错技术，三者缺失任何一环都会造成可靠性的下降。

检错技术

检错是一种能够自动发现故障、确定故障性质和部位的技术，但必须配合容错技术才能排除故障。检错可用专门的程序实现属于程序设计的范围。常见的检错技术有判断返回结果、计算运行时间和状态检测等。相比航空科研等涉及大量计算的高精尖业务场景，一体化平台则更关注的是系统整体提供不间断服务的能力，通过风险分析活动识别出威胁系统可靠性主要的风险来自硬件设备和网络故障。对此在具体实现上我采取了两个措施，一是，针对数据交换业务，开发了一个网络监控模块，采用 ping/echo 程序监控前置机到生产库的网络情况，并基于图形化界面实时展示网络状态。另外，为了防止交换程序假死对 Kettle (ETL 工具) 组件进行了集成，可实时检测服务状态，当故障发生时首先停止相关服务记录异常，然后通过网关发送报警短信，由人工处理；二是，针对双机部署的 web 程序我采用了 Keepalived 组件做心跳连接，当检测到 Master 宕机可迅速切换到 Backup 继续服务，整个过程自动完成，无需人工干预。通过以上检错技术的应用成功的提高了系统故障检测和应急处理能力。

容错技术

容错是保证系统在某些组成部分出现故障时仍能正常工作的技术。常见的容错技术有冗余技术、双机技术、集群技术。针对一体化平台用户量大、可用性要求高的业务特点，我选择了采用双机技术和集群技术来满足不同场景的容错要求。在逻辑上我将一体化平台可划分为面向公众服务和内部办公服务两部分。对于面向公众服务，作为办事入口和宣传平台必须提供 7*24 不间断服务。根据这个业务特点我在部署方式上采用了高可用集群技术，首先使用一台 Nginx 服务器 + 三台应用服务器构成负载均衡集群解决并发问题。然后为避免 Nginx 单点故障，实际部署两套 Nginx 作为 Master 节点和 Backup 节点同时指向应用服务器，并使用 Keepalived 组件连接两台 Nginx 服务器做心跳检测，构成高可用集群保障服务的连续运行；对于内部办公服务，因部署的节点较多，考虑到资源限制采用的是双机互备技术，将两个独立的模块在两台机器上同时运行，但彼此均设为 Backup，当某一台服务器心跳检测失

四、论网络安全体系设计

1、真题原文：

论网络安全体系设计

随着社会信息化的普及，计算机网络已经在各行各业得到了广泛的应用。目前，绝大多数业务处理几乎完全依赖计算机和网络执行，各种重要数据如政府文件、工资档案、财务账目和人事档案等均依赖计算机和网络进行存储与传输。另一方面，针对计算机和网络的攻击活动日益猖獗，网络安全已经成为当前社会的主要安全问题之一。

在上述背景下，国家标准《信息处理系统工程开放系统互联基本参考模型——第二部分：安全体系结构》（GB/T 9387.2-1995）定义了基于 OSI 参考模型 7 层协议之上的信息安全体系，其核心内容是：为了保证异构计算机进程与进程之间远距离交换信息的安全，定义了认证服务、访问控制服务、数据机密性服务、数据完整性服务和抗抵赖性服务等 5 大类安全服务，以及提供这些服务的 8 类安全机制及相应的 OSI 安全管理，并根据具体系统适当配置于 OSI 模型的 7 层协议之中。

问题内容：

请以“网络安全体系设计”为题，依次从以下三个方面进行论述。

1. 概要叙述你参与管理和开发的软件项目以及你在其中承担的主要工作，并详细阐述该软件系统在网络安全方面的要求。
2. 请对 GB/T 9387.2-1995 中定义的 5 大类安全服务进行描述，阐述每类安全服务的定义和主要实现手段。
3. 请结合项目实际，具体阐述你在项目中实现了上述 5 大类安全服务中的哪些服务，具体运用了哪些实现手段。

2、参考理论：

五类安全服务：

一. 认证与鉴别服务

鉴别参与通信的对等实体和数据源的合法性。对等实体鉴别和数据源鉴别：由第 N 层实体提供，可向第 N+1 层实体证实。安全服务由第 N 层实体提供，可向第 N+1 层实体证实数据

源。

认证服务的基本目的，是为了防止其他实体占用和独立操作被认证实体的身份。认证服务提供了实体声称其身份的保证，只有在主体和验证者的关系背景下，认证才有意义。认证服务的主要实现方式包括以下 5 种：

已有的信息，如认证口令。

拥有的信息，如 IC 卡、令牌等。

不可改变的特性，如指纹、虹膜等生物特征。

相信可靠的第三方建立的认证。

环境，如主机地址等

二. 访问控制服务

能够阻止未经授权而利用通过 OSI 模型的可访问资源。

访问控制服务决定开放环境中允许使用哪些资源、在什么地方适合组织为授权访问的过程。在访问控制实例中，访问可以是对一个系统或对系统内部进行。常见的访问控制服务的实现方式包括以下三种方式：

自主访问控制（DAC）。

自主访问控制是一种接入控制服务，通过执行基于系统实体身份及其到系统资源的接入授权。包括在文件、文件夹和共享资源中设被许可。用户有权对自身所创建的文件、数据表等访问对象进行访问，并可将其访问权授予其他用户或收回其访问权限。允许访问对象制定针对该对象访问的控制策略，通常可通过访问控制列表来限定针对客体可执行的操作。

强制访问控制（MAC）

强制访问控制是系统强制主体服从访问控制策略，是由系统对用户所创建的对象，按照规定的规则控制用户权限及操作对象的访问。其主要特征是对所有主体及其所控制的进程、文件、段、设备等客体实施强制访问控制。

基于角色访问控制（RBAC）

基于角色访问控制主要通过对角色的访问进行控制，使权限与角色相关联，用户通过成为适当角色的成员而得到其角色的权限。用户可依其责任和资格分派相应的角色，角色可依新需求和系统合并赋予新权限，而权限也可根据需要从某角色中收回

三. 数据保密性服务

对数据提供保护，防止数据未经授权而被泄漏，防止在系统之间交换数据时被截取。它

还内含四项服务：连接保密性、无连接保密性、选择字段保密性、通信业务流保密性。

数据机密性服务的目的是确保信息仅仅是对被授权者可用，信息的保护可以通过确保数据被限制于授权者获得，或通过特定方式表示数据来获得。信息的机密性主要通过以下两种方式实现：

通过禁止访问提供机密性，即可以通过访问控制，以及通过物理媒体保护和路由选择控制保证机密性。

通过加密提供机密性，即防止数据泄漏在传输或存储中。加密机制包括基于对称的加密机制和基于非对称的加密机制。

除此以外，还可以通过数据填充、通过虚假事件（例如隐藏在不可信链路上交换的信息流总 1；）、通过保护 PDU 头和通过时间可变域提供机密性。

四. 数据完整性服务

防止系统之间交换数据，非法修改数据或丢失数据。数据完整性可分四类：实体完整性、域完整性、参照完整性、用户定义的完整性。

数据完整性服务的目的是通过阻止威胁或探测威胁，保护可能遭到不同方式危害的数据完整性和数据相关属性完整性，即保证数据不以未经授权方式进行改变或损毁。数据完整性的常见实现方式包括：

阻止对数据传输媒介访问的机制。包括物理隔离、不受干扰的信道，路由控制，访问控制等。

探测对数据非授权修改的机制。包括密封、数字签名、数据重复、与密码变换相结合的数字指纹和消息序列号等。

五. 禁止否认服务

阻止通信双方否认发送和接收数据的行为。

它也叫做抗抵赖服务，它是提供有关特定事件或行为的证据，包括证据的生成、验证和记录，以及在解决纠纷时随即进行的证据恢复和再次验证。抗抵赖性服务的实现方式主要包括数字签名、用户认证、操作日志等技术。

3、 参考范文：

2013 年系统架构师论文

一、软件架构建模技术与应用

1、真题原文：

软件架构建模技术与应用

软件架构用来处理软件高层次结构的设计和实现，它以精心选择的形式将若干结构元素进行装配，从而满足系统的主要功能和性能需求。软件架构设计的首要问题是如何表示软件架构，即如何对软件架构建模。根据建模的侧重点不同，可以将软架构模型分为结构模型、框架模型、动态模型、过程模型和功能模型。Kmhchten 在 1995 年提出了“4+1”视图模型，将 5 种模型有机地统一在一起。软件架构用来处理软件高层次结构的设计和实现，它以精心选择的形式将若干结构元素进行装配，从而满足系统的主要功能和性能需求。软件架构设计的首要问题是如何表示软件架构，即如何对软件架构建模。根据建模的侧重点不同，可以将软架构模型分为结构模型、框架模型、动态模型、过程模型和功能模型。Kmhchten 在 1995 年提出了“4+1”视图模型，将 5 种模型有机地统一在一起

请围绕“软件架构建模技术与应用”论题，依次从以下三个方面进行论述。

1. 概要叙述你参与管理和开发的软件项目以及你在其中所承担的主要工作。
2. 简要叙述“4+1”视图模型的主要内容。结合你参与项目的实际情况，详细说明该项目需求及所涉及的软件架构（包括使用到的视图模型、创建的架构模型及使用的建模工具等）。
3. 说明该项目软件架构的实施效果，分析其是否满足了项目的需求并说明原因。

2、参考理论：

“4+1”视图模型从 5 个不同的视角来描述软件架构，每个视图只关心系统的一个侧面，5 个视图结合在一起才能反映系统的软件结构的全部内容。这 5 个不同的视角包括逻辑视图、开发视图、进程视图、物理视图和场景视图。

1. 逻辑视图

逻辑视图主要支持系统的功能需求，即系统提供给最终用户的服务。在逻辑视图中，系

统分解成一系列的功能抽象，这些抽象主要来自问题领域。在 OO 技术中，通过抽象、封装和继承，可以用对象模型来代表逻辑视图，用类图来描述逻辑视图。

2. 实现视图（开发视图）

开发视图也称为模块视图，在 UML 中被称为实现视图，它主要侧重于软件模块的组织和管理。开发视图要考虑软件内部的需求。

3. 进程视图

进程视图侧重于系统的运行特性，主要关注一些非功能性需求。进程视图强调并发性、分布性、系统集成性和容错能力，以及逻辑视图中的功能抽象如何适应进程结构等，它也定义了逻辑视图中的各个类的操作具体是在哪一个线程中被执行。进程视图可以描述成多层抽象，每个级别分别关注不同的方面。

4. 物理视图

物理视图在 UML 中被称为部署视图，主要考虑如何把软件映射到硬件上，它通常要考虑到解决系统拓扑结构、系统安装和通信等问题。

5. 场景

场景可以看作是那些重要系统活动的抽象，它使 4 个视图有机联系起来。场景对应 UML 中的用例视图。

3、参考范文：

范文 1：

摘要：

2010 年，我参加了***矿业的污水监控平台的开发，在这个项目中，我担任系统设计和开发的工作。这个系统主要是对辰州矿业的排污进行数据采集，实时监控，超标报警，数据统计和管理。本文结合作者的实践，以辰州矿业监控平台系统架构建模为例，论述了 4+1 视图模型在工作中的运用。本论文先介绍 4+1 模型，然后结合我参与项目的实际情况，详细说明在这次项目中所涉及的软件架构，最后是分析该项目取得实践效果。

正文

软件架构用来处理软件高层次结构的设计与实施，它以精心选择的形式将若干结构元素进行装配，从而满足系统的主要功能和性能需求。软件架构设计的首要问题是如何表示系统架构，即如何对软件架构建模。根据建模的侧重点不同，可以将软件架构模型分为结

构模型，框架模型，动态模型，过程模型和功能模型。Kruchten 于 1995 年提出了 4+1 视图模型，将这 5 种模型有机的结合在了一起。

4+1 视图模型分为为：逻辑视图，物理视图，场景，进程视图和开发视图。逻辑视图，主要是支持系统的功能需求，即体现了系统提供一个什么样的服务。在逻辑视图中，系统分解成一系列的功能抽象，这些抽象主要来自于功能域。在 OO 技术中主要是通过类的继承，封装和抽象，用类图来描述逻辑视图。物理视图，即软件如何映射到硬件上，它通常考虑的是系统的拓扑结构，系统安装和通信等问题。场景，将 4 种视图有机的联系起来，在 UML 中一般用用例图来表示。进程视图，体现了系统在运行中表现出来的特性，如分布性，并发性和容错能力，系统集成性。它定义了逻辑视图的各个类操作具体是在哪一个线程中执行的。开发视图，主要描述系统的组织，与逻辑视图精密相关，在 UML 中对应的是实现图。其中用户对应的是逻辑视图，开发人员对应的是开发视图，系统工程师对应的是物理视图，系统集成工程师对应的是进程视图。

****矿业监控平台是一个对矿业排污进行数据采集，根据集团领导的需求：公司的监测人员能够随时监测排污的数据，当污染数据超标时能够即时通知相关的负责人员，能够查询历史的排污数据记录，管理员能够通过工控机操作并管理各个数据采集仪。经过我们的分析，结合需求，我们做了如下设计：系统由 B/S 和 C/S 两部分组成。其中 B/S 主要的功能包括：实时污染数据的监测和显示，数据统计和管理，超标报警等，由矿业的 PC 客户端浏览器和应用服务器，数据库服务器组成。C/S 的主要功能包括：控制数据采集仪器，进行数据采集，将采集的污染数据持久化，实时污染数据的显示等，由工控机，数据采集仪，和数据库服务器组成。**

在这次系统架构建模的工作中，我们用到的建模工具是 Microsoft Visio,用到的架构描述语言是 UML,结合项目的实际情况，用 4+1 视图模型进行建模：

1. 对情景进行建模：经过我们的分析，结合客户的需求，在这个系统中有两类角色，一类是矿业集团办公室的负责排污监测的各位工作人员，即用户类，他们一般会进行排污的数据查询工作，另外当排污超标时，系统会进行报警并通知。另外一类是矿业监控室的管理人员，他们一般在现场值班，负责对数据采集仪和工控机进行管理，监测各项排污数据，和进行排污数据的维护工作。所以系统的主要用例有如下几个：数据查询，超标报警，数据实时显示，数据采集，数据维护。

2. 对逻辑视图进行建模：在这个系统中，我们用 UML 的包图来描述逻辑视图，经过

我们的分析，在 C/S 部分，工控机和数据采集仪之间进行如下职责划分：

- * 应用层负责设备状态的显示，并提供模拟控制台供用户发送调试命令。
- * 应用层使用通讯层和嵌入层进行交互，但应用层不知道通讯的细节。
- * 通讯层负责在 SOCKETC 协议之上实现一套专用的"应用协议"。
- * 当应用层发送来包含调试指令的协议包，由通讯层负责按 SOCKET 协议将之传递给嵌入层。
- * 当嵌入层发送来原始数据，由通讯层将之解释成应用协议包发送给应用层。
- * 嵌入层负责对调试设备的具体控制，以及高频度地从数据采集器读取设备状态数据。
- * 设备控制指令的物理规格被封装在嵌入层内部，读取数采器的具体细节也被封装在嵌入层内部。

在 B/S 部分，客户端 PC 和应用服务器进行如下职责划分：表现层，逻辑业务层和数据访问层。

3. 对物理视图进行建模：我们用 UML 的部署图来描述进程视图，对系统的拓扑结构和通信进行描述。整个系统分为 B/S 和 C/S 两部分，B/S 中的客户 PC 端和应用服务器，应用服务器和数据库服务器都通过局域网进行连接。C/S 中工控机和数据库服务器通过局域网进行连接，工控机和采集仪通过数据专线进行连接，工控机和采集仪之间采用串口 SOCKET 通信，工控机和数据库采集仪之间通过 TCP/IP 进行通信，客户 PC 端和应用服务器通过 HTTP 进行通信。

4. 对进程视图进行建模：在 C/S 部分，表示层分为三类线程：数据持久化线程，数据显示线程和采集控制线程，分别用来对数据进行持久化，图形的显示，以及对采集仪进行控制，进行数据的采集。

通过对 4+1 视图的情景，逻辑视图和物理视图建模，分别从系统提供的服务和功能，系统的网络拓扑结构和通信进行了建模，对系统进行了详细的分析，为系统的设计和接下来的项目开发提供了有力的支持。

范文 2:

摘要

2019 年 3 月，我单位联合某高校研发了《程序在线评测比赛考试系统》。系统以程序代码在线提交自动评测功能为核心，分为题库模块、评测机模块、实验作业模块、考试模块、比赛模块、抄袭判定模块、用户管理模块等，支持对接教务平台。在项目中我担任系统架构

师，负责架构设计工作。

本文以该系统为例，主要论述了软件架构建模技术在项目中的具体应用，重点从 4+1 视图模型的场景视图、逻辑视图与物理视图三个方面介绍。场景视图以 UML 用例图对系统主要用户角色与涉及用例进行分析；逻辑视图通过 UML 包图从前端 Web 服务、平台保障服务、业务服务三个层次对系统功能进行建模；物理视图使用 UML 部署图对微服务在硬件环境的具体部署方法进行描述。最终系统顺利上线，获得用户一致好评。

正文

笔者在一个专为高校建设计算机专业智能教学一体化平台的单位任职，过往成果有《计算机组成原理仿真实验系统》等。2019 年 3 月，我单位联合某大学研发了《程序在线评测比赛考试系统》项目（以下简称为“OJ 系统”），以取代原有传统的编程上机考试平台。

系统以程序代码的在线提交自动评测功能为核心，主要分为题库模块、评测机模块、实验作业模块、考试模块、比赛模块、抄袭判定模块、用户管理模块等。题库模块主要负责试题和测试用例的管理，用户根据试题要求编写程序代码提交到系统，系统将测试用例与程序代码发送给评测机模块，由评测机自动编译、执行、判分，并将结果发送给其他相关模块进行统计；实验作业模块用于在线布置作业，从题库中选取试题，设置截止日期等要求；考试模块用于学生在线考试，按教师预先设置的参数自动从题库随机抽题生成试卷，以及向教务平台上传考试成绩；比赛模块主要用于 ACM 竞赛的培训；抄袭判定模块用于鉴定代码与他人代码雷同率；用户管理模块负责用户信息的管理。在这个项目中，我担任了系统架构师的职务，主要负责系统的架构设计相关工作。

4+1 视图模型从 5 个视角来描述软件架构，分别为：逻辑视图、物理视图、进程视图、开发视图和场景视图。逻辑视图主要描述系统的功能需求，将系统分解为一系列的功能抽象，可用对象模型来代表，用类图来描述；开发视图主要侧重于软件模块的组织和管理，考虑软件内部的需求；进程视图侧重于系统的运行特性，主要关注非功能需求，强调并发性、分布性、系统集成性和容错能力，以及逻辑视图中的功能抽象如何适应进程结构，定义了具体线程执行的操作；物理视图主要考虑软件向硬件的映射，解决系统的拓扑结构、安装和通信问题。场景视图可看做重要系统活动的抽象，使其他 4 个视图有机联系。

OJ 系统使用微服务架构开发，基于 4+1 视图建模，建模工具为 Rational Rose，描述语言为 UML，这里着重从场景视图、逻辑视图、物理视图三个方面展开介绍。

1. 场景视图

场景视图使用 UML 模型中的用例图来进行建模。OJ 系统功能主要面向高校学生程序设计语言的在线学习、考试、比赛，我们经过分析，结合用户的需求，在系统中划定了四类用户角色，这些角色分别为：在校学生、任课教师、系统管理员、校外人员。在校学生用户是 OJ 系统学生端的主要使用者，学生涉及的主要用例有：登录系统、提交代码、自由练习、参加考试、提交实验作业、参加比赛、信息维护、查看系统帮助、交流讨论等；任课教师用户属于 OJ 系统的管理者，教师涉及的主要用例有：登录系统、信息维护、班级管理、助教管理、学生管理、考试管理、题库管理、实验作业管理、比赛管理、代码查重、论坛管理等；系统管理员拥有 OJ 系统最高的系统权限，在具备学生与教师所有用例的基础上，还增加了教师管理、评测机管理、系统管理、服务器管理等用例。校外人员用户主要面向社会以及其他高校的编程学习爱好者，仅具备登录系统、提交代码、自由练习、参加比赛、查看系统帮助五种用例。

2. 逻辑视图

逻辑视图使用 UML 模型中的包图来进行建模。我们经过分析，决定采用微服务架构风格开发，将系统分为前端 Web 服务、平台保障服务、业务服务三部分。前端 Web 服务由负载均衡与服务器集群结合，解决前台界面并发问题；平台保障服务分为 API 网关、服务注册中心、监控平台，用以实现基础服务框架，所有业务服务都注册到服务注册中心；业务服务分为多个微服务，实现具体业务功能，解决协同问题。当用户通过网络访问系统时，首先会访问到前置的负载均衡服务器，负载均衡服务器会将请求转发到前端网站的集群，前端网站通过发起 http 请求来和后端交互。API 网关收到前端的请求，会从服务注册中心根据当前请求，来获取对应的服务配置，随后通过服务配置再调用已注册的服务。当后端微服务存在多个实例时，将采取负载均衡的方式调用。后端微服务通过同步消息、异步消息、工作流三种方式协同工作，完成具体业务功能。服务监控平台注册到服务注册中心，获取所有后端业务服务的运行状态信息。

3. 物理视图

物理视图使用 UML 模型中的部署图来进行建模。系统微服务采用分布式的部署方法，每个微服务单独编译、打包、部署，基于 Docker 容器连同运行环境一起封装，根据实际情况可在同一台物理机器或多台物理机器上同时部署多个实例来提高系统的性能。服务启动后会将自身信息注册到已部署好的分布式服务注册中心，所有客户端请求一律进入路由网关，路由网关根据请求通过服务注册中心来进行服务发现，然后通过反向代理的方式调用具

体服务。在分布式部署多个服务的基础上，前端服务器集群通过 Nginx 负载均衡服务器做统一代理访问，部署为路由模式，系统内部网络与外部网络分属于不同的逻辑网络，以实现系统内部与外部网络的隔离。在负载均衡算法的选择上，使用了最小连接法，每当客户端的请求来临时，任务分发单元会将任务平滑分配给最小连接数的微服务节点，这样的部署方法以廉价且透明的方式扩展了服务器和网络的带宽，可以大大提升系统的并发量，同时保证系统整体的稳定性和可靠性。

总结

通过 4+1 视图的场景视图、逻辑视图和物理视图等建模方式，对系统进行了详细的分析，为系统的设计和接下来的项目开发提供了有力的支持。

系统自 2019 年 10 月正式上线已运行一年有余，在学校的日常教学考试和竞赛培训中投入使用，截至目前已有 3000 名以上的学生用户、评测了 70000 份以上的程序代码，获得了单位同事领导和学校教师们的一致好评。不可避免的，我们在设计过程中，也存在一些问题和不足，不少开发人员在实现过程中有时还是习惯于原有的结构化设计方法，对 4+1 视图模型的使用有些抵触。而且，这些视图在应用过程中，往往不是单独使用，需要多个视图综合运用。这方面，我们还缺少相关的经验。

实践证明，OJ 系统项目能够顺利上线，并且稳定运行，与系统采用了合适的软件架构建模技术密不可分。经过这次软件架构建模的方法和实施的效果后，我也看到了自己身上的不足之处，在未来还会不断地更新知识，完善本系统在各方面的设计，使整个系统能够更加好用，更有效地服务于高校师生。

二、企业应用系统的分层架构风格

1、真题原文：

企业应用系统的分层架构风格

软件架构风格是描述一类特定应用领域中系统组织方式的惯用模式，反映了领域中诸多系统所共有的结构特征和语义特征，并指导如何将各个模块和子系统有效组织成一个完整的系统。分层架构是一种常见的软件架构风格，能够有效简化设计，使得设计的系统结构清晰，便于提高复用能力和产品维护能力。

由于大量企业应用系统都由界面呈现、业务逻辑、数据存储三类功能构成，因此广泛采用分

层架构风格进行系统设计。

请围绕“企业应用系统的分层架构风格”论题，依次从以下三个方面进行论述。

1. 概要叙述你参与管理和开发的企业应用系统建设项目以及你在其中所承担的主要工作。
2. 请结合项目实际情况，指出应用系统都有哪些层次以及每个层次的主要功能。
3. 请结合项目实际情况，指出设计每个层次时需要注意的问题及相应的解决方案。

2、参考理论：

分层架构设计是一种常见的架构设计方法，能够有效简化设计，使设计的系统结构清晰，便于提高复用能力和产品维护能力。一般来说，企业应用系统的架构可以分为表现层、中间层和持久层三个层次。

1. 表现层

表现层主要负责接收用户的请求，对用户的输入、输出进行检查与控制，处理客户端的一些动作，包括控制页面跳转等，并向用户呈现最终的结果信息。表现层主要采用 MVC 结构来实现。控制器负责接收用户的请求，并决定应该调用哪个模型来处理；然后，模型根据用户请求调用中间层进行相应的业务逻辑处理，并返回数据；最后，控制器调用相应的视图来格式化模型返回的数据，并通过视图呈现给用户。

2. 中间层

中间层主要包括业务逻辑层组件、业务逻辑层工作流、业务逻辑层实体和业务逻辑层框架四个方面。业务逻辑层组件分为接口和实现类两个部分，接口用于定义业务逻辑组件，定义业务逻辑组件必须实现的方法。通常按模块来设计业务逻辑组件，每个模块设计为一个业务逻辑组件，并且每个业务逻辑组件以多个 DAO 组件作为基础，从而实现对外提供系统的业务逻辑服务。业务逻辑层工作流能够实现在多个参与者之间按照某种预定义的规则传递文档、信息或任务的过程自动进行，从而实现某个预期的业务目标，或者促进此目标的实现。业务逻辑层实体提供对业务数据及相关功能的状态编程访问，业务逻辑层实体数据可以使用具有复杂架构的数据来构建，这种数据通常来自数据库中的多个相关表。业务逻辑层实体数据可以作为业务过程的部分 I/O 参数传递，业务逻辑层的实体是可序列化的，以保持它们的当前状态。业务逻辑层是实现系统功能的核心组件，采用容器的形式，便于系统功能的开发、代码重用和管理。

3. 持久层

持久层主要负责数据的持久化存储，主要负责将业务数据存储在文件、数据库等持久化存储介质中。持久层的主要功能是为业务逻辑提供透明的数据访问、持久化、加载等能力。在持久层设计时需要考虑 MVC 模型中的模型、视图和控制器分别对应哪些组件；在中间层设计时需要考虑框架与业务组件之间的关系；在持久层设计时需要考虑如何支持对多种类型数据的透明访问。

3、参考范文：

参考素材：

软件架构风格描述一类特定应用领域中系统组织方式的惯用模式，反映了领域中诸多系统所共有的结构特征和语义特征，并指导如何将各个模块和子系统有效组织成一个完整的系统。分层架构是一种常见的软件架构风格，能够有效简化设计，使得设计的系统结构清晰，便于提高复用能力和产品维护能力。

由于大量企业系统都由界面呈现、业务逻辑、数据存储 3 类功能构成，因此广泛采用分层架构风格进行系统设计。

本文主要论述笔者参与管理和开发的企业应用系统建成项目，以及笔者在其中所承担的主要工作。结合项目实际情况，指出应用系统的各个层次，以及各个层次的主要功能。指出设计每个层次时需要注意的问题及相应的解决方案。

结合项目实际情况，指出所开发的应用系统总体架构，特别是架构的层次关系。分层架构设计是一种常见的架构设计方法，能够有效简化设计，使设计的系统架构清晰，并与提高复用能力和产品维护能力，一般来说，架构可以分为表现层、中间层和持久层 3 个层次。

表现层，主要负责接收用户的请求，对用户的输入、输出进行检查和控制。处理客户端的一些动作，包括控制页面跳转等，并向用户呈现最终的结果信息。表现层主要采用 MVC 结构来实现，控制器负责接收用户的请求，并决定应该调用哪个模型来处理。然后模型根据用户请求调用中间层进行相应的业务逻辑处理，并返回数据。最后控制器调用相应的视图来格式化模型返回的数据，并通过视图呈现给用户。

中间层，中间层主要包括业务逻辑层组件、业务逻辑层工作流、业务逻辑层实体和业务逻辑层框架 4 个方面，业务逻辑层组件分为接口和实现两个部分，接口用于定义业务逻辑组件必须实现的方法。通常按木块来设计业务逻辑组件，每个模块设计为一个业务逻辑组件。

并且每个业务逻辑组件以多个 DAO 组件作为基础，从而实现对外提供系统的业务逻辑服务；业务逻辑层 workflow 能够实现在多个参与者之间按照某种预定义的规则传递文档、信息或任务的过程自动进行，从而实现某个预期的业务目标，或者促进此目标的实现；业务逻辑层实体提供对业务数据以及相关功能的状态编程访问，业务逻辑层实体数据可以使用具有复杂架构的数据来构建，这种数据通常来自数据库中的多个相关表。业务逻辑层实体数据可作为业务过程部分 IO 参数传递，业务逻辑层的实体可以是可序列化的，以保持他们的当前状态。业务逻辑层是实现系统功能的核心组件，采用容器的形式，便于系统功能的开发、代码的重用和管理。

持久层，主要负责数据的持久化存储，即将业务数据存储在文件、数据库等持久化存储介质中，主要功能是为业务逻辑提供透明的数据访问、持久化、加载等能力。

三、论软件可靠性设计技术的应用

1、真题原文：

论软件可靠性设计技术的应用

随着软件的日益普及，系统中软件成分不断增加，使得系统对软件的依赖越来越强。软件的可靠性对系统可靠性的影响越来越大。而实践证明，保障软件可靠性最有效、最经济、最重要的手段是在软件设计阶段采取措施进行可靠性控制，为此提出了软件可靠性设计的概念。软件可靠性设计就是在常规的软件设计中，应用各种方法和技术，使软件设计在兼顾用户功能和性能需求的同时，全面满足软件的可靠性要求。软件可靠性设计应和软件的常规设计紧密结合，贯穿于软件设计过程的始终。

请围绕“论软件可靠性设计技术的应用”论题，依次从以下三个方面进行论述。

1. 概要叙述你参与管理和开发的软件项目以及你在其中所承担的主要工作。
2. 结合项目实际，论述你在项目开发过程中，进行软件可靠性设计时遵循的基本原则；论述你在该项目中所采用的具体可靠性设计技术。
3. 阐述你在具体的可靠性设计工作中，为了分析影响软件可靠性的主要因素，所采用的可靠性分析方法。

三. 降低复杂度设计

软件复杂性与软件可靠性有着密切的关系，是产生软件缺陷的重要根源。在设计时考虑降低软件的复杂性，是提高软件可靠性的有效方法。降低复杂度设计的思想是在保证实现软件功能的基础上，简化软件结构，缩短程序代码，优化软件数据流向，降低软件复杂度，从而提高软件可靠性。

3、参考范文：

- 摘要

2019 年 8 月，我司承接了某市医疗集团，智慧药房项目，该项目主要为集团下属 36 家社区卫生服务中心提供药品统一目录管理、药品集中采购、库存管理、处方合理用药审核、药品配发、自动化发药设备驱动提供软件支撑。在该项目中我担任了软件架构设计师一职，主要负责该项目的软件架构设计工作。

本文以智慧药房项目为例，主要论述了**可靠性技术**在项目中的具体应用，**通过将平台服务层进行集群化部署，数据库分布式主从模式，以及服务器 RAID 磁盘阵列，对平台服务层、数据库以及硬件存储多维度的容错、容灾机制，提高了项目硬件的可用性，安全性和可扩展性。**通过**可靠性技术**的应用，提供了软件运行的质量，保障一软件能够 7*24 小时的稳定运行，最终项目成功上线，并获得了用户的一致好评

-- 背景(所有论文同一个项目)

根据深化医药卫生体制改革规范药求，推进药品集中采购，增加药品的供应保障能力，严格监督管理，保障药品的用药安全，所有医疗机构开出的处方，必须通过处方审核后，方可进入划价计费环节，未经审核的处方，不得进行计费和配发。2019 年 8 月，我司承接了某医疗集团智慧药房项目，该项目为医疗集团下属 36 家社区卫生服务中心提供软件支持，主要分为药品供应模块、药房管理模块、处方用药审核模块、药品配发模块、设备协同模块。药品供应模块负责药品库存预警，供社区卫生服务中心药房对药品进行集中采购，并将药品发送到省采招平台；药房管理模块主要进行一些基础信息的维护，及关注药品的库存管理，药品批号跟踪；处方用药审核模块，负责对医生开出的药品进行合理用药审核，将不合理的处方审核结果返回给医生，提醒医生修改处方；药口中配发模块，负责将处方中的药品进行调配，打印用法用量标签，确认发药后扣减相应的库存；设备协同模块主要是驱动药房中的一些自动化发药设备，将要药品信息发给设备上位机，根据设

- (3) 形成候选的决策方案（目标、规划、方法和途径等）；
- (4) 构造决策问题的求解模型（如数学模型、运筹学模型、程序模型、经验模型等）；
- (5) 建立评价决策问题的各种准则（如价值准则、科学准则、效益准则等）；
- (6) 多方案、多目标、多准则情况下的比较和优化；
- (7) 综合分析，包括决策结果或方案对实际问题可能产生的作用和影响的分析，以及各种环境因素、变量对决策方案或结果的影响程序分析等。

3、参考范文：

四、论企业信息化规划的实施与应用

1、真题原文：

论企业信息化规划的实施与应用

企业信息化建设是一项长期而艰巨的任务，不可能在短时间内完成。信息化规划是企业信息化建设的纲领和向导，是信息系统设计和实施的前提和依据。信息化规划以整个企业的发展目标和战略、企业各部门的目标与功能为基础，同时结合行业信息化方面的实践和对信息技术发展趋势的掌握，制定出企业信息化远景、目标和发展战略，从而达到全面、系统地指导企业信息化建设的目的。

请围绕“企业信息化规划的实施与应用”论题，依次从以下三个方面进行论述。

1. 概要叙述你参与的企业信息化规划项目以及你在其中所担任的主要工作。
2. 简要叙述企业信息化规划的主要内容。结合你参与的项目的实际情况，详细分析有关企业的信息化规划目标及规划的具体内容。
3. 说明你所参与实施的企业信息化规划的步骤及效果，介绍其是否达到了预期的目标并分析原因。

2、参考理论：

企业信息化规划不仅涉及到信息系统规划，同时与企业规划、业务流程建模等紧密相关，是

是不符合规定的。采用需求研讨论会，可以从车间里各个岗位的负责人的工作经验中提取出正确的业务流程，保证软件功能的正确性和严谨性。

现在总结一下采用需求研讨论会的实施过程。在召集会议之前，我们先把用户交给我们的系统描述细致阅读了，从中抽出了系统的需求，把功能清晰明亮的需求用用例模型描述出来，把不清晰的部分列举出来，整理出会议讨论主题；接着我们根据车间的生产流程，在每个岗位上都确定一个人员来参加会议，确定完人员名单后，我们把会议要讨论的主题发给各位与会人员，让各个与会人员提前看看讨论的主题，如果是关系人可以确定的问题就可以提前把解决方案发给我们，这样就可以减少专题讨论会的工作量和时间；在开会的时候，我们让经验丰富的产品经理来当会议的主持人，负责引导与会人员围绕我们不明确的需求来展开讨论和进行头脑风暴，直到需求确定下来为止；会议结束后，我们当天就把会议的纪要，以及需求说明书整理出来，发送给各位干系人查看并确认，确保用户同意按需求说明书的要求开发系统。通过这个会议，我们成功地把绿色车间检测系统不明确的需求都确定下来了。

需求研讨论会虽然能帮我们快速地把需求确定下来，但该技术不够灵活，因为很难找到所有关系人都有空的时间来集中开会，尤其是项目比较庞大的时候，组织会议就会有点吃力，因为涉及到的人员越多，集中开会的时间越难确定，开会的频率也不能太高，很难不妨碍客户的正常工作。

通过需求研讨论会我们基本可以把系统的业务需求、用户需求、功能需求确定下来，但是系统的业务流程及信息需求等其他的需求一时间也难以确定下来。我们一致认为可以借助其他的需求获取技术来帮助，把功能业务需求、用户需求、功能需求以外的其他需求确定下来，比如信息需求。我们一开始不清楚工

2、参考理论：

静态演化 (Static Evolution)。

静态演化是指软件在停机状态下的演化。其优点是不用考虑运行状态的迁移，同时也没有活动的进程需要处理。然而停止一个应用程序就意味着中断它提供的服务，造成软件暂时失效。

动态演化 (Dynamic Evolution)

动态演化是指软件在执行期间的软件演化。其优点是软件不会存在暂时的失效，有持续可用性的明显优点。但由于涉及状态迁移等问题，比静态演化从技术上更难处理。

软件静态演化是指发生在应用程序停止时的软件修改和更新，即一般意义上的软件维护和升级。静态演化的优点是没有状态迁移或活动线程的问题要解决，缺陷是停止应用程序意味着停止它所提供的服务，也就是使软件系统暂时失效。在软件交付之后，静态演化（类似于一般意义上的软件维护）就成为软件变更的一个常规过程。变更可以是一种更正代码错误的简单变更，也可以是更正设计错误的较大范围的变更，还可以是对描述错误进行修正或提供新需求这样的重大改进。有三种不同的软件维护：改正性维护、适应性维护和完善性维护。维护过程一般包括变更分析、版本规划、系统实现和向客户交付系统等活动。

在面向对象技术中，使用子类型方法来扩展程序，它适合于软件静态演化和代码重用。子类型化一个类意味着保留父类中的参数和方法，并尽可能地增加新的参数和方法。

另外，使用重载和多态性作为主要的演化机制。实际上，建立类的新版本，最简单的机制是创建它的子类，然后重载需要变更的方法，最后，使用多态性调用新创建的方法。在基于构件的软件技术中，构件采取接口和实现相分离的技术，构件之间只能通过接口进行通信，这使得具有兼容接口的不同构件实现可以相互取代，从而成为软件静态演化的一条途径。

动态演化是最复杂也是最有实际意义的演化形式。动态演化使得软件在运行过程中，可以根据应用需求和环境变化，动态地进行软件的配置、维护和更新，其表现形式包括系统元素数目的可变性、结构关系的可调节性和结构形态的动态可配置性。软件的动态演化特性对于适应未来软件发展的开放性、动态性具有重要意义。

动态演化是指软件在运行期间的演化。在许多重要的应用领域中，例如金融、电力、电信及空中交通管制等，系统的持续可用性是一个关键性的要求，运行时刻的系统演化可减少因关机和重新启动而带来的损失和风险。此外，越来越多的其他类型的应用软件也提出了运行时刻演化的要求，在不必对应用软件进行重新编译和加载的前提下，为最终用户提供系统

定制和扩展的能力。

动态演化可分为两种类型：预设的和非预设的。在 Web 环境中，软件应用常常需要处理多种类型的信息，因此它们常被设计为可以动态下载并安装插件以处理当前所面临的新类型的信息；

而分布式 Web 应用也常常需要增减内部处理节点的数目以适应多变的负载。这些动态改变都是软件设计者能够预先设想到的，可实现为系统的固有功能。另有一些必须对系统配置进行修改和调整的情况是直到系统投入运行以后才发现的，这就要求系统能够处理在原始设计中没有完全预料到的新需求。这种情况下一般需要关闭整个系统，重新开发、重新装入并重新启动系统。然而，为了进行局部的修改而关闭整个系统在某些情况下是不允许的（例如，关键运行系统）或者代价太高。精心设计的动态演化技术可以在不关闭整个系统的前提下修改系统的结构配置，并尽量使未受影响的部分继续工作以提高系统的可用度。

为支持软件的动态演化性，已在语言、机制和环境等方面做了大量工作。在程序语言的层次上，引进各种机制以支持软件动态演化，例如动态装载技术允许增加代码到已运行的程序中，延迟绑定是在运行时而不是编译时决定类和对象的绑定。Java hotswap 允许在运行时改变方法：当一个方法终止时，这个方法的新版本可以有效地替换旧版本，在类层次上代码的二进制兼容被支持。Gilgul 语言也允许更换运行时对象。但程序语言层次上的动态演化机制仅局限于函数、类方法和对象等小粒度的替换，只支持预设的有限变更，变更由事件触发。

通过标准化运行级构件的规约，依靠构件运行平台（中间件平台）提供的基础设施，使软件在构件层次上的动态演化成为可能。中间件中具有的如命名服务、反射技术和动态适配等机制，为运行态构件的动态替换和升级提供支撑，从而推动了软件动态演化的发展。命名服务就是给构件实例提供一个名称，以便客户通过这些名称来获取构件实例。对工业标准构件 EJB 和 CORBA 构件的引用都可以通过中间件平台的命名服务进行。同一构件标识可以被映射到多个构件实例，从而根据具体情境对某一名字的构件引用导向到不同的构件实例。反射技术是系统的一种自描述（self-representation）和自推理的技术，它提供了关于自身行为的表示，这种表示可以被检查和调整，且与它所描述的系统行为是因果相联（causally connected）的。因果相联，意味着对自表示的改动将立即反映在系统的实际状态和行为中，反之亦然。将反射性引入中间件能够以可控的方式开放平台内部的实现，从而提高中间件的定制能力和运行时的适应能力。动态适配机制中比较著名的是 CORBA 提供的动态接口服务：

块主要是驱动药房中的一些自动化发药设备，将要药品信息发给设备上位机，根据设备药品的效期进行排序，通过上位机程序调用下位机，驱动设备出药。在该项目中我担任软件架构设计师职务，主要负责软件的架构设计及中间件等技术的选型工作。

——中间正文主体 800~1000 字左右

软件可靠性设计主要通过冗余、心跳等技术对软件进行可靠性的设计，确保软件的稳定运行。冗余可以从硬件、软件两个方面着手，硬件可以使用多台服务器做容灾处理，大型项目会应用异地部署服务器的方式，防止因服务器故障、断网、断电或地震等不可抗拒的自然灾害造成服务不可用，给企业和用户造成损失，软件层面可以使用多应用部署集群，确保在一个应用出现异常时，可以继续工作。同时应用负载均衡，CDN 等技术进行用户请求分流，保证用户访问性能的同时，对应用进行减压，数据库一般可以采用主从复制的方式部署。

——简介——中间过度作用

本文以智慧药房项目为例，将从集群化部署、数据库主从、软件冗余部署方面论述了可靠性架构设计在项目中的应用。

——介绍三种的应用

集群化部署：项目为了防止一台服务器出现故障，导致应用系统不可用，采用了多服务器冗余方式，以 N+1 的标准选了三台服务器做冗余，并且对服务器磁盘做了 RAID5 阵列，项目采用前后端分离的方式，前台采用 Vue.js，后端平台服务采用的 Java 语言开发，前端 Web 应用系统分别在服务器上部署了三个节点，并且通过使用 Nginx 做反向代理，使得应用不直接对外暴露，提高了安全性，同时配置了集群模式，将用户请求进行分流，提交了系统的可靠性同时也提高了性能，平台服务层以 Docker 的方式进行发布，并且通过 K8S 进行管理，同时引入了 SkyWalking 对接口链路进行监控，并且通过 Spring Admin 对服务节点进行监控，一旦链路应用节点发生故障，立马发送邮件给运维人员，让其及时干预，Java 服务采用 SpringCloud 做微服务架构，通过 Nacos 进行服务注册与发现，通过网关将服务暴露给前端 Web 层调用。

数据库主从：项目中数据层采用了关系型数据库 MySQL，以及 NoSQL 非关系型 MongoDB 同进为了减轻数据库的压力，引入了 Redis 缓存。MySQL 主要存放一些业务数据，如医生开出的处方数据，以及药品目录，ymtlwyth 以及采购订单，配送入库单等，这些数据的特点是使用频率不同，但极其重要，采用了主从复制的方式对 MySQL 进行了冗余，在

措施就能做好，而是需要从预防、检查和处理三方面来共同保障，对应架构设计则是降低复杂度技术、检错技术、容错技术，三者缺失任何一环都会造成可靠性的下降。

检错技术

检错是一种能够自动发现故障、确定故障性质和部位的技术，但必须配合容错技术才能排除故障。检错可用专门的程序实现属于程序设计的范围。常见的检错技术有判断返回结果、计算运行时间和状态检测等。相比航空科研等涉及大量计算的高精尖业务场景，一体化平台则更关注的是系统整体提供不间断服务的能力，通过风险分析活动识别出威胁系统可靠性主要的风险来自硬件设备和网络故障。对此在具体实现上我采取了两个措施，一是，针对数据交换业务，开发了一个网络监控模块，采用 ping/echo 程序监控前置机到生产库的网络情况，并基于图形化界面实时展示网络状态。另外，为了防止交换程序假死对 Kettle (ETL 工具) 组件进行了集成，可实时检测服务状态，当故障发生时首先停止相关服务记录异常，然后通过网关发送报警短信，由人工处理；二是，针对双机部署的 web 程序我采用了 Keepalived 组件做心跳连接，当检测到 Master 宕机可迅速切换到 Backup 继续服务，整个过程自动完成，无需人工干预。通过以上检错技术的应用成功的提高了系统故障检测和应急处理能力。

容错技术

容错是保证系统在某些组成部分出现故障时仍能正常工作的技术。常见的容错技术有冗余技术、双机技术、集群技术。针对一体化平台用户量大、可用性要求高的业务特点，我选择了采用双机技术和集群技术来满足不同场景的容错要求。在逻辑上我将一体化平台可划分为面向公众服务和内部办公服务两部分。对于面向公众服务，作为办事入口和宣传平台必须提供 7*24 不间断服务。根据这个业务特点我在部署方式上采用了高可用集群技术，首先使用一台 Nginx 服务器 + 三台应用服务器构成负载均衡集群解决并发问题。然后为避免 Nginx 单点故障，实际部署两套 Nginx 作为 Master 节点和 Backup 节点同时指向应用服务器，并使用 Keepalived 组件连接两台 Nginx 服务器做心跳检测，构成高可用集群保障服务的连续运行；对于内部办公服务，因部署的节点较多，考虑到资源限制采用的是双机互备技术，将两个独立的模块在两台机器上同时运行，但彼此均设为 Backup，当某一台服务器心跳检测失败时，服务可用在短时间内进行切换，从而保证了业务的持续性。通过以上容错技术的应用成功的保障了系统的稳定运行。

降低复杂度设计

随着系统规模不断的扩大，架构复杂度也在不断的攀升，对于大型复杂系统有效的管理

软件复杂度，已成为保障系统可靠性的一个重要手段。常见的降低复杂度方式有应用架构设计，使用构件和设计模式等。针对架构设计本次项目采用 J2EE 技术开发，其本身就具有 N 层架构的特点，结合系统业务我采用了三层 B/S 架构设计来构建系统，其层次划分为表现层、功能层和数据层。表现层，运行在客户机浏览器上，是应用的用户接口担负着用户与应用间对话功能；功能层，运行在 Web 服务器上，负责响表示层的用户请求、执行业务逻辑；数据层，就是数据库管理系统，它运行在数据库服务器上负责管理数据读写和执行事务。另外，在项目中我使用了多种构件来降低系统复杂度，例如使用 RabbitMQ 消息中间件，使系统以消息为中心进行交互有效降低各模块间耦合度；使用 SpringMVC 框架解耦视图和模型，提高代码重用度；使用 Hibernate 框架解耦持久层和数据库，使持久层专注于面向对象编程，无需关心具体实现，降低了程序复杂度。最后在开发过程中还应用了抽象工厂、策略、责任链等多种模式来解耦代码。通过以上方式成功的降低了软件的复杂度，提高了系统可靠性。

【结尾】

由于在项目中多种可靠性技术的成功应用，有效的保障了系统的稳定性和可靠性，使开发工作取得了成功。但在项目过程中也遇到过问题，因客户单位部分用户派驻到办事大厅工作，窗口办公需要配合高拍仪、读卡器等硬件设备，而浏览器出于安全考虑隔离了 web 程序调取本地资源仅提供有限支持，给系统推广带来了困难。对此我采用了富客户端技术，使用 Electron 框架封装了一个客户端专门负责调取硬件资源及时解决了该问题，没有对项目造成实质影响。

通过项目组全体成员的共同努力，一体化平台于 2018 年 10 月完成验收正式上线，截至目前系统运行稳定运维工作量可控，得到了监管部门和企业用户的一致认可和好评。在今后的的工作中我将不断总结和提升专业技术能力，争取为我国信息化事业贡献一份力量。

三、论数据挖掘技术的应用

1、真题原文：

论数据挖掘技术的应用

随着信息技术的高速发展，各组织机构积累的数据量急剧增长。如何从海量的数据中提取有

用的知识成为当务之急。数据挖掘 (DataMining) 就是为顺应这种需要而发展起来的数据处理技术, 是知识发现的关键步骤。数据挖掘就是从大量的、不完全的、有噪声的、模糊的、随机的实际应用数据中, 提取隐含在其中的、人们事先不知道的、但又是潜在有用的信息和知识的过程。

请围绕“论数据挖掘技术的应用”论题, 依次从以下三个方面进行论述。

1. 概要叙述你参与分析和开发的软件项目以及你所担任的主要工作。
2. 数据挖掘的主要任务是什么? 具体论述你在项目中使用数据挖掘技术所解决的问题。
3. 数据挖掘的方法主要有哪些? 分析并讨论你所选择的数据挖掘方法, 简述其具体实现过程和实际应用效果。

2、参考理论:

数据挖掘的任务主要是关联分析、聚类分析、分类、预测、时序模式和偏差分析等。

1. 关联分析

两个或两个以上变量的取值之间存在某种规律性, 就称为关联。数据关联是数据库中存在的—类重要的、可被发现的知识。关联分析的目的是找出数据库中隐藏的关联网。一般用支持度和可信度两个阈值来度量关联规则的相关性。

2. 聚类分析

聚类是把数据按照相似性归纳成若干类别, 同一类中的数据彼此相似, 不同类中的数据相异。聚类分析可以建立宏观的概念, 发现数据的分布模式, 以及可能的数据属性之间的相互关系。

3. 分类

就是找出一个类别的概念描述, 它代表了这类数据的整体信息, 即该类的内涵描述, 并用这种描述来构造模型, 一般用规则或决策树模式表示。分类是利用训练数据集通过一定的算法而求得分类规则。分类可被用于规则描述和预测。

4. 预测

预测是利用历史数据找出变化规律, 建立模型, 并由此模型对未来数据的种类及特征进行预测。预测的精度和不确定性被重点关注, 通常用预测方差来度量

5. 时序模式

时序模式是指通过时间序列搜索出的重复发生概率较高的模式。与回归一样, 它也是用

覆盖正例排斥反例方法

它是利用覆盖所有正例、排斥所有反例的思想来寻找规则。首先在正例集合中任选一个种子，到反例集合中逐个比较。与字段取值构成的选择子相容则舍去，相反则保留。按此思想循环所有正例种子，将得到正例的规则。比较典型的算法有 Michalski 的 aql1 方法等。

统计分析方法

在数据库字段项之间存在两种关系：函数关系（能用函数公式表示的确定性关系）和相关关系（不能用函数公式表示，但仍是相关确定性关系），对它们的分析可采用统计学方法，即利用统计学原理对数据库中的信息进行分析。可进行常用统计（求大量数据中的最大值、最小值、总和、平均值等）、回归分析（用回归方程来表示变量间的数量关系）、相关分析（用相关系数来度量变量间的相关程度）、差异分析（从样本统计量的值得出差异来确定总体参数之间是否存在差异）等。

模糊集方法

利用模糊集合理论对实际问题进行模糊评判、模糊决策、模糊模式识别和模糊聚类分析。系统的复杂性越高，则模糊性越强，一般模糊集合理论是用隶属度来刻画模糊事物的亦此亦彼性的。

3、参考范文：

摘要：随着我国科技事业的不断发展，人们进入了信息化时代，互联网逐渐渗透到各个领域，带领人们走入大数据时代。互联网为数据挖掘技术提供了十分丰富的资料来源，这些资料会随着实际情况不断更新，最终形成更加完善的信息资料库。基于此，文章就数据挖掘技术的功能和应用展开了论述。

关键词：互联网；数据挖掘技术；应用

1 数据挖掘技术的功能

数据挖掘技术有各种各样的功能，包括数据分类、数据估计、数据预测和数据关联分组等，这些功能在不同的领域发挥着与众不同的作用。正因为数据挖掘具有这样的功能和作用，才会被各个领域所应用。1) 数据分类：它可以将研究的对象按照不同的属性和特点进行分类，最终建立不同种类和类别组。例如，把数据挖掘技术运用到教育领域，可以把它按照教育程度进行数据分类，向不同年龄段的学生推荐相应层次的教学课程[1]。2) 数据估计：根据它连续性的价值数据资料，来获取它未知的价值。3) 数据预测：通过之前一系列实际的

数据，推测它未来价值的变化。4) 数据关联分组：把相关对象放在一起，设计能吸引眼球的产品组合，提高购买率[2]。

2 数据挖掘技术的应用

数据挖掘技术已经逐渐渗入不同的领域，像电子商务、销售、教育、金融、音乐领域等，它不仅带动这些领域的发展，还可以有效推动我国社会的整体发展，真正发挥数据挖掘技术自身价值。

2.1 在教育领域的应用

数据挖掘技术不仅涉及各个领域，还更好地促进了这些领域得到了前所未有的发展。现如今数据挖掘技术把人们带入一个新大数据时代。在教育领域中就更好地体现出了数据挖掘技术在互联网中的应用，它改变了以往传统的教学方式，弥补了传统课堂教学所带来的弊端，它把传统、局限的“小课堂”转变成为与世界接轨的“大课堂”，与此同时，还把各个领域的教学内容汇总化，使不同类别、不同学科的教学内容都全部展现在互联网中，更利用学者进行知识学习和总结。它成功带动了网络教育的发展，并且这种教育方式受到了人们的一致好评。互联网给学生提供了多种多样、丰富的知识资源，数据挖掘技术给教育领域提供了更大、更好的发展空间。这些大数据信息给学者带来十分便利的学习资源，它成功地跨越了时间和空间的局限，学者可以随时随地学习，节省了更多学习时间和空间。在传统的教育领域中，大部分学生都是在教学课堂上学习，如今学者就可以应用互联网技术进行相关学科知识内容的学习，并且对较难、不理解的知识可以利用互联网技术反复的观看和研究，这样大大提高了教学质量和教学水平。例如：学者应用互联网技术学习的过程中，把学习的课程、知识检测分数、学习方式都通通的展现在互联网的数据库中。这些看似比较复杂的信息，但是通过数据挖掘技术就可以掌握其中所蕴含的规律[3]。分析和总结大量信息，就可以基本了解学者学习的重点、难点和易错点，不断更改和创新教学课程，最终有效提高课堂的教学质量，使网络教学课程的点击率增高。

2.2 在音乐领域的应用

近年来在音乐平台和网络平台中有大部分的人点击歌曲和视频的数据量逐渐增多，这使音乐网络平台不仅得到了迅速的发展，而且还把听众喜欢的歌曲进行反馈分析。对于各大音乐平台来讲，音乐资源和用户体验度是反映一个平台的好坏的重要指标。可以把音乐资源和用户体验度进行数据挖掘分析和整理，一方面它可以更好地分析出当下最流行的乐种和歌曲，提高网络音乐平台的整体点击率，另一方面通过听众经常听取的歌曲类型和歌手进行数

据分析,更好地为听众提供相似和类似种类曲风的歌曲和歌手,还可以随时更新歌手演唱会的演出时间,为歌迷提供更多机会[4]。通过数据挖掘技术在音乐领域的应用,它不仅推动网络音乐平台的全面发展,而且通过销售歌手专辑的方式,还给个人和经济公司带来十分丰厚的资金来源,从而更好、更加有效地促进音乐的发展。

3 结束语

要顺应时代的发展趋势,妥善应用互联网,推动社会全方面的发展,尽快步入大数据时代。在这种情况下,通过数字挖掘技术来推动网络互联网的发展具有一定的重要性,它被运用到各个领域,并且发挥出来挖掘技术的重要价值。

四、大规模分布式系统缓存设计策略

1、真题原文:

大规模分布式系统缓存设计策略

大规模分布式系统通常需要利用缓存技术减轻服务器负载、降低网络拥塞、增强系统可扩展性。缓存技术的基本思想是将客户最近经常访问的内容在缓存服务器中存放一个副本,当该内容下次被访问时,不必建立新的数据请求,而是直接由缓存提供。良好的缓存设计是一个大规模分布式系统能够正常、高效运行的必要前提。在进行大规模分布式系统开发时,必须从一开始就针对应用需求和场景对系统的缓存机制进行全面考虑,设计一个可伸缩的系统缓存架构。

请围绕“大规模分布式系统缓存设计策略”论题,依次从以下三个方面进行论述。

1. 概要叙述你参与实施的大规模分布式系统开发项目以及你所担任的主要工作。
2. 从不同的用途和应用场景考虑,请详细阐述至少两种常见的缓存工作模式,并说明每种工作模式的适应场景。
3. 阐述你在设计大规模分布式系统的缓存机制时遇到了哪些问题,如何解决。

2、参考理论:

单实例模式

单实例模式是一种较为简单的缓存模式,多个应用服务器共享一个中央的缓存服务器。通过共享缓存的数据,能够极大地提高系统的性能。该模式的主要限制在于缓存服务器的内

正文

笔者在一个专为高校建设计算机专业智能教学一体化平台的单位任职，过往成果有《计算机组成原理仿真实验系统》等。2019 年 3 月，我单位联合某大学研发了《程序在线评测比赛考试系统》项目（以下简称为“OJ 系统”），以取代原有传统的编程上机考试平台。

系统以程序代码的在线提交自动评测功能为核心，主要分为题库模块、评测机模块、实验作业模块、考试模块、比赛模块、抄袭判定模块、用户管理模块等。题库模块主要负责试题和测试用例的管理，用户根据试题要求编写程序代码提交到系统，系统将测试用例与程序代码发送给评测机模块，由评测机自动编译、执行、判分，并将结果发送给其他相关模块进行统计；实验作业模块用于在线布置作业，从题库中选取试题，设置截止日期等要求；考试模块用于学生在线考试，按教师预先设置的参数自动从题库随机抽题生成试卷，以及向教务平台上传考试成绩；比赛模块主要用于 ACM 竞赛的培训；抄袭判定模块用于鉴定代码与他人代码雷同率；用户管理模块负责用户信息的管理。在这个项目中，我担任了系统架构师的职务，主要负责系统的架构设计相关工作。

系统需要支撑全校学生编程课程的学习考试，以及大量校外用户的使用，为保证系统的性能和用户体验，以缓存技术进行优化尤为重要。常见的缓存分为三种工作模式：单点模式、主从模式和集群模式。单点模式各个应用服务共享同一个缓存实例，主要瓶颈在于缓存服务器的内存大小、处理能力和网络带宽，适合缓存要求简单、数据量和吞吐量小、性能要求低的场景；主从模式将缓存的数据复制到多台机器上，数据彼此同步，可分散压力、获得更高的吞吐能力，适合数据量不大、改动频率不大、性能要求高的场景；集群模式节点间共享数据，部分冗余保证一定程度的可用性，可存储超过单个服务内存容量数据，通过增加服务器缓解数据增长和压力增加，适合总体数据量大、可伸缩性需求高、客户端数量庞大的场景。

OJ 系统使用 Spring Cloud 微服务架构开发，基于 Redis 内存数据库实现缓存，运用了多种缓存设计策略，本文着重从工作模式的选择、高可用性的设计、缓存一致性与分布式算法三个方面的问题展开论述。

1. 工作模式的选择

OJ 系统需要支撑全校学生编程课程的学习考试，以及大量校外用户的使用，对可靠性和性能要求高，因此单点模式不予考虑。经过对 OJ 系统业务需求的分析，我们归纳了三

种需要缓存的数据类型。第一种是系统配置项、角色权限分配等元数据信息。这部分信息使用频率很高但数据量小，不会经常改动，我们采用主从模式来缓存，主缓存节点供系统配置模块写入数据，从缓存节点分别供具体业务服务调用。在系统启动时，自动将相关数据装入缓存。信息修改时，先写入主缓存节点，再同步更新至从缓存节点。第二种是用户登录的会话状态，这部分信息使用频率高、更新频繁，且几乎所有的业务服务都需要依赖此类数据，为保证性能与可靠性，我们采用集群方式来缓存，集中管理用户会话，各个业务服务通过数据访问层来调用的具体的缓存节点。第三种是业务逻辑中一定时间内不会变化，但数据量大的信息，如试题信息、实验作业信息等，以及统计机制自动识别的高频访问信息，也采用集群方式缓存。

2. 高可用性的设计

为实现 OJ 系统缓存的高可用性，一方面我们开启了 Redis 数据持久化功能，并配置了相应的备份策略，能够有效地解决数据误操作和数据异常丢失的问题，另一方面，我们设计了一些冗余机制。在主从模式中，采用多机主备架构实现冗余，在主节点故障时，自动进行主备切换，将从节点提升为主节点继续服务，保证服务的平稳运行。如果主节点和从节点之间连接断开，重新连接时从节点会进行数据的部分重同步，当无法进行部分重同步时，会进行全量重同步。在集群模式中，采用 Cluster 技术实现冗余，每个节点保存数据和整个集群状态，负责一部分哈希槽，每个节点都和其他所有节点连接并共享数据。为了使在部分节点失效或者大部分节点无法通信的情况下集群依然可用，在集群内部使用了主从复制模型，每个节点都会相应地有 1~N 个从节点，当某个节点不可用时，集群便会将它的从节点提升作为新的主节点继续服务。缓存服务发生异常时，可通过 OJ 系统的服务监控平台产生报警，提醒运维人员及时处理。

3. 缓存一致性与分布式算法

为保证 Redis 缓存与原数据库的数据一致性，当读取数据时，会先读取 Redis 缓存中的数据，如果 Redis 缓存中不存在所要的数据，则从原数据库中读取，并同步写入至 Redis 缓存中，当写回/删除数据时，写入到原数据库中，并同步淘汰 Redis 缓存中的数据。业务服务通过使用专门的数据访问层来调用增加缓存后的数据库，使数据缓存机制对应用透明。在缓存内部实现一致性，通过分布式哈希算法来实现，为考虑到日后缓存集群的扩展需要，因

此不能使用简单的模 N 哈希法，我们在 OJ 系统中采用了哈希环的算法。该算法构造一个长度为 2^{32} 的整数环，将 Redis 节点放置于环上，当业务服务调用缓存时，首先以服务的应用 ID 作为键值计算哈希在环上定位，然后沿顺时针方向找到最近的 Redis 节点，完成映射。当缓存服务集群中有节点故障，以及添加新节点时，只会影响上一个节点的数据，避免了发生缓存雪崩的情况，提高了容错性和扩展性。哈希环中缓存节点过少时易发生缓存倾斜，我们通过增加虚拟节点的方式解决了该问题。

总结

我们在这次系统设计中，还使用了很多其他的缓存设计策略，这里不再一一赘述。系统在经过性能测试、负载测试、压力测试、稳定性测试后，自 2019 年 10 月正式上线已运行一年有余，在学校的日常教学考试和竞赛培训中投入使用，截至目前已有 3000 名以上的学生用户、评测了 70000 份以上的程序代码，获得了单位同事领导和学校教师们的一致好评。日常使用过程中最高出现过 600 余用户同时在线进行实验作业提交、评测的情况，基本未出现页面加载缓慢、请求超时的问题，满足了高校编程课平台的基本性能需求。

实践证明，OJ 系统项目能够顺利上线，并且稳定运行，与系统采用了合理的缓存设计密不可分。经过这次大规模分布式系统缓存设计的方法和实施的效果后，我也看到了自己身上的不足之处，在未来还会不断地更新知识，完善本系统在各方面的设计，使整个系统能够更加好用，更有效地服务于高校师生。

2009 年系统架构师论文

一、论基于 DSSA 的软件架构设计与应用

1、真题原文：

论基于 DSSA 的软件架构设计与应用

软件架构设计的一个重要课题是如何解决软件重用问题。特定领域软件架构（Domain Specific Software Architecture, DSSA）是一种有效实现特定领域软件重用的手段。按照 Tracz 的说法，DSSA 就是一个特定的问题领域中由领域模型、参考需求、参考架构等组成的开发基础架构，其目标就是支持一个特定领域中多个应用的生成。

DSSA 的基本活动包括领域分析、领域设计和领域实现。领域分析的主要目的是获得领域模型，领域模型描述领域中系统之间共同的需求，即领域需求；领域设计的主要目标是获得 DSSA，DSSA 描述领域模型中表示需求的解决方案；领域实现的主要目标是依据领域模型和 DSSA 开发和组织可重用信息。

请围绕“基于 DSSA 的软件架构设计与应用”论题，依次从以下三个方面进行论述。

1. 概要叙述你参与管理和开发的软件项目以及你在其中所承担的主要工作。
2. 就你所熟悉的领域，请给出针对该特定领域，在基于 DSSA 的软件设计开发中所涉及的领域模型、参考需求和参考架构以及相应的支持环境或设施。
3. 具体阐述你参与管理和开发的项目中使用 DSSA 的情况，包括领域分析、领域设计和领域实现等活动是如何具体实施的，最终实际效果如何。

2、参考理论：

一. 领域分析

定义领域范围，确定领域应用需要满足的用户需求；

定义领域特定的元素、领域字典和领域术语；

定义领域特定的设计和实现需求约束；

二. 领域设计

在此基础上，定义领域模型，产生该领域的参考架构，并说明构件的语法和语义；

三. 领域实现

最后，产生、搜集可重用的产品单元，为 DSSA 增加构件，为问题域实现新应用提供支持。

所给出的 DSSA 应该具备以下 4 个方面的特征：

一个严格定义的问题域或解析域

具有普遍性，使其可以用于领域中某个特定应用的开发

对整个领域能有合适程度的抽象

具备该领域固定的，典型的在开发过程中的可重用元素

3、参考范文：

范文 1：

摘要：

2020 年 4 月，本人所在的某市金融投资集团启动了集团综合管理系统建设，该项目实现基金、融资租赁、资金管理、转贷、融资担保、保理等金融业务信息化及人力资源、智能办公、法务管理等内部管理功能。在此项目中，我担任了架构师，负责项目总体架构设计工作。本文以该综合管理系统为例，主要论述了基于 DSSA 的软件架构设计与应用。领域分析阶段，以集团已有系统资源出发，综合分析新业务系统需求，获取集团各业务所共有的领域需求，整合产生领域模型；领域设计阶段，分析领域模型，设计出通用业务架构，实现高层次领域实现；领域实现阶段，根据领域模型和 DSSA，一面从已有系统剥离，一边重头开发，产生领域共有可重用元素。最终项目顺利上线，运行平稳，获得了领导的高度认同。

正文：

本人所在的某市金融投资集团已经建设了移动办公系统、财务系统、融资租赁系统、基金管理系统等应用系统，但仍有部分业务缺乏信息化支撑，已建成的系统比较分散，主要是“竖井式”方式建设，系统与系统之间信息不共享以及信息与业务流程相脱节。针对这些问题，2020 年 4 月，经集团党委决定，启动集团综合管理系统建设，实现以下功能。一、实现转贷、保理、融资担保、资金管理、人力资源、智能办公等业务流程信息化，同时整合基金管理系统、移动办公系统、财务系统等已有信息资产，促进部门、子公司之间横向协同。二、建立统一共享的信息平台，利用数据报表平台为管理层提供决策信息支持，实现纵向管控。三、建立涵盖分布式文件存储、单点登录、分布式事务支持、短信邮件通知、可控任务调度等服务的基础 IT 设施，提供高质量、可重用的平台服务。

在此项目中，我担任架构师一职，负责系统总体架构设计。通过分析，由于集团已有成熟的业务系统，且集团对于所有业务设计了一套通用的业务流程，且大致业务实体类似，因此采用基于 DSSA 的软件架构设计将大大加快系统的推进，并有助于提升代码质量，提高系统可靠性。

在集团金融业务内部管控领域中，领域需求主要包括项目立项、项目尽调、项目方案调整、项目合同签订、项目投放、收付款、项目收尾等业务需求，每一项业务步骤均可能包括审批、材料审核等流程。领域模型由项目、尽调、合同、投放、收尾等一系列业务实体类组成，并且涵盖业务经理、审核人员、业务主管等角色，包括了项目立项、项目尽调等上述功能的用例图，对于复杂用例如项目调整，采用活动图详细描述整个过程，形成整体领域模型。基于层次系统架构风格设计 DSSA，分为表现层、中间层、数据访问层三层架构，从融资租赁

系统中抽离出领域共有部分，对其进行裁剪、抽象、细化、扩充，形成通用业务构件，实现各领域需求，包含项目、合同、投放各业务流程，特定业务开发时，以通用业务构件为基础，在此之上进行个性化扩展，实现快速开发。相应的支持环境或设施有，基于 OSS 的分布式存储机制、基于 seata 的分布式事务机制、支持多种方式的登录鉴权机制、访问控制机制、增删改查代码快速生成模块、基于 activity 的工作流机制、邮件短信同志模块等基础设施。本文以集团综合管理系统为例，从领域分析、领域设计和领域实现三个活动出发详细阐述基于 DSSA 的软件架构设计与应用。

一、领域分析获取领域模型。

集团已有包括融资租赁系统及基金管理系统在内的业务管理系统，且集团有一套基本的管控流程，所有业务流程均在此套流程上进行裁剪、细化、扩充实现。因此，通过与业务领域专家的探讨，我们决定从融资租赁系统中及集团综合管控流程入手，分析领域需求，获取领域模型。由集团基本管控流程分析得知，业务基本分为立项流程、尽调流程、合同审查用印流程、投放流程、收付款流程即合同执行流程以及合同收尾流程。另外根据融资租赁系统分析得知，各业务系统均有自己个性化需求，如租赁强调租赁物的价值，以及该租赁物抵押所带来的风险。另外系统还包含文件共享、登录、数据访问控制、分布式事务等基础设施需求。以这些获取到的领域需求为基础，我们设计了包含项目、合同、投放等业务实体，包含立项、尽调、合同审查等流程，以及各公共基础设计的领域模型，全面表达了领域内的范围、特定元素及环境约束。

二、领域设计获取 DSSA。

通过参考融资租赁系统，我们决定采用层次架构风格设计系统，实现基于 B/S 系统的表现层、中间层及数据层。从领域维度又分为基础设施层、通用业务层、特定业务层。基础设施层涵盖基于 OSS 的分布式存储机制、基于 seata 的分布式事务机制、登录鉴权机制、短信邮件通知服务等基础设施，该层次的功能基本与具体业务无关，为其他所有业务的开发提供基础功能支持，便于快速开发，由于这些基础功能的大规模复用，经过实战的考验，又为系统的稳定运行提供保障。通用业务层涵盖集团整体管控流程，需做到通用，能够普遍适应其他特定业务，因此它是可配置的、可扩展的，通过配置调整参数，可以控制流程的流转，比如对于不需要投放流程的转贷，可以通过配置跳过投放，对于融担不走线上审批的业务，可以跳过审批，并转而上传文件来完成业务。对于每个业务实体，我们包括两个部分，一部分为通用实体部分，另一个是特定实体基类，对于特定业务，如果无法通过配置来实现，则

通过继承基类扩展实现。这样，我们就得到了集团业务领域的 DSSA。

三、领域实现获取可重用元素。

在得到领域模型和 DSSA 后，我们分析得知我们需要的领域可重用元素包括通用业务构件及分布式存储构件、分布式事务构件、登录鉴权构件、短信邮件通知构件、 workflow 构件等基础设施。由于集团已存在部分信息化系统，我们对遗留系统进行分析，确定了分布式存储构件、 workflow 构件、短信通知构件等基础构件可从融资租赁系统中提取获得，登录鉴权构件也通过提取并扩展登录方式得到。对于通用业务构件，提取出的代码由于与原系统具有非常大的耦合性，缺乏对于整个领域设计的高度抽象，因此我们决定仅做参考，对其进行重新设计实现。项目界面采用 VUE 框架实现，后端采用 Spring MVC，基于 spring cloud 实现微服务架构，用 mybatis 实现 ORM 方式的数据访问层设计。实体数据分为基础数据和个性化数据，个性化数据由具体业务定制实现，然后在 Dao 层对其进行汇聚，产生具体的实体数据，在此基础上，service 层对数据层下所发生的事情完全透明，只需要按照预定去获取数据即可。以这样的方式，我们得到了大量的可重用元素，为后续的特定业务开发提供参考。

2021 年 9 月，项目顺利上线，运行平稳，获得了领导和同事的一致好评。由于项目采用了基于 DSSA 的架构设计方法，项目上线时间大大早于预期，并且领域实现的可重用元素均由资深实现人员实现，因此质量情况较好，系统可靠性较高，也因为领域构件会被大规模复用，因此对这些构件的改动需要考虑之前的兼容性，并编写足够的单元测试测试保证质量，对这方面的重视也大大提高系统整体的质量。另外由于系统业务也在不断扩展、深化，因此领域模型、DSSA 及领域可重用元素并不是一成不变的，也需要不断迭代自己，通过进化、调整以适应变化的需求，另外也通过自身能力的提高，不断地驱动着业务的扩展，如数据分析能力为业务的开展提供数据支撑。基于 DSSA 的软件设计方法也为后续新业务开展打下了坚实的基础，如后续小贷业务的开展。

范文 2:

【摘要】

去年三月份，我所在的公司启动国网电力用户用电信息采集系统项目，我被任命为项目负责人。国网电力用户用电信息采集系统是国家电网公司坚强智能电网建设的一部分。由于公司之前为南网（主要是广东省）开发过类似用电信息采集系统，且公司准备在电力行业做强做大，我提出了采用 DSSA 技术来研发国网用电信息采集系统，得到公司领导层的一致赞同。

由于项目功能实现上具有明显的阶段性，我决定采用演化方式来实现 DSSA 及完成应用产品开发。一是对原有系统、文档及国网用电信息系统功能规范进行分析，完成 DSSA；二是对原有系统进行部件提取，做为核心资源的公共部件；三是加强对核心资源的管理，方便研发工程师查找部件及扩展部件。

经过近一年的努力，终于完成了公司用电信息采集系统核心资源的建立，也完成了国网电力用户用电信息采集系统项目。

【正文】

去年三月份，我所在的公司启动国网电力用户用电信息采集系统项目，我被任命为项目负责人。国网电力用户用电信息采集系统是国家电网公司坚强智能电网建设的一部分。公司之前开发过广东电网公司计量营销一体化系统，类似于用电信息采集系统。

我对广东电网公司计量营销一体化系统的功能规范和国网电力用户用电信息采集系统的功能规范进行分析，发现除了系统内各自的通信协议不同外，其它的功能需求大体上相同。整个采集系统都是分三层实现，主站层，采集终端层和电能表层。由于电能表已经规范化了，有专门的表计生产厂家，这一层不需要投入资源进行研发。从公司目前现状来看，主站层投入研发工作量较少，一是主站的开发中模块化做得比较好；二是用户的需求基本一致。国网用电信息采集系统仅需要在广东电网公司计量营销一体化系统主站进行界面调整和支持国网用电信息采集系统通信协议即可达到要求。

根据之前开发的经验，用电信息采集系统开发的重点是采集终端的开发。因为采集终端需要安装到现场，而现场的用电环境各异，能够到达的远程信道也不同。采集终端可维护性低或可靠性低，则会产生大量的维护工作，影响公司品牌及利润。根据用电信息采集系统的要求，采集终端分为集中抄表终端、专变采集终端和公变采集终端。广东电网公司计量营销一体化系统的采集终端大体上也分为上述三类：低压集抄终端、负荷管理终端、配变监测终端。通过对采集终端的功能要求进行分析，可以看出它们归属于一个产品家族。我在项目组启动会议上提议采用 DSSA 技术进行采集终端产品的研发，建立公司用电信息采集系统核心资源，同时将计量营销一体化系统的采集终端也归结到产品家族中。

众所周知，DSSA（特定领域软件架构）就是在一个特定的问题领域中支持一组应用的开发，这些应用形成产品家族。DSSA 是软件重用的一种手段，它由领域模型、参考需求、参考架构组成重用元素。

用电信息采集系统各终端基本需求都是对外接的电能表或测量点的读数进行采集，稍做

处理后通过 GPRS/CDMA 信道远程传输给采集系统主站端。采集终端的功能模块一般包括测量点采集模块，表计规约模块，现场总线模块，PPP 拨号模块，主站命令模块，本地维护模块，程序升级模块，数据存储模块，交流采样模块，负荷控制模块等等。

由于采集终端在现场使用的特殊性，它的非功能性要求主要集中在可靠性、可修改性和易用性。现场用电环境复杂，信道各异，要求采集终端具有高可靠性。由于市场上的电能表支持的规约各异及现场总线发展快速，要求采集终端可扩展性强，能快速支持新的表计规约和现场总线，且支持远程升级操作。由于在现场施工时多是由工程队进行安装，工程队人员的素质高低不齐，要求采集终端在本地操作具有一定的智能化，且要求调试简单。

根据以上分析，采集终端软件架构采用分层设计比较合适。分层设计的软件可修改性和可扩展性比较好。由于分层开发，将关注点分离到各层，将系统的复杂度分到各层中，相应可靠性也可以得到提高。

在用电信息采集系统研发中，我决定采用演化方式进行开发。

首先对原有系统、文档及国网用电信息系统功能规范进行分析，完成 DSSA。在项目起始阶段，我对计量营销一体化系统及用户需求文档及设计文档进行分析，将用户需求用 EXCEL 表格列出来。然后再对国网用电信息采集系统的功能规范进行分析，用同样的方式列出用户需求，需求比对后发现它们之间的功能要求大体上是一样的。但由于通信协议不同，会导致一些功能在实现上有差别，如主从终端连接功能，用电信息采集系统采用一条命令完成主从终端的所有通信，而计量营销一体化系统分成建链、传输、断链三条命令来实现。于是我决定将基础业务模块做成通用的模块，根据不同的参数来初始化模块，或各具体产品自己适配模块。按照这个需求，我对核心资源进行分层设计。

总体上，核心资源分成三层，由低到高依次是：基础资源层，基础业务层，扩展业务层。基础资源层包括多进程框架，GUI 系统，系统 API 和驱动封装，虚拟通道模块等等。由于采集终端的操作系统是 LINUX，而且通讯口资源比较多，采用一个进程管理一个通讯口，单一管理便于维护，因此提供多进程框架，方便应用开发时的进程增加。对系统 API 和驱动进行封装，方便以后代码的移植。基础业务层主要包括用电信息采集系统的各个基础功能模块，有现场总线模块、表计规约模块、测量点采集模块、交流采样模块、负荷控制模块等等。扩展业务层主要对基础业务层中的各个模块进行参数化和适配，以适应本系统的需要。根据目前的情况，扩展业务层主要有计量营销一体化系统部件包和国网用电信息采集系统部件包。

其次对原有系统进行部件提取，做为核心资源的公共部件。计量营销一体化系统的采集

终端在研发时由于没有采用组件开发技术，各功能模块和应用层耦合较强，在提取公共部件时需要对应应用层解耦。各个具体的功能都有相应的控制参数，而控制参数可以由主站命令模块进行读写，将控制参数管理模块做成中介者模式，很好地实现了各功能模块的解耦。如 PPP 拨号模块，和应用层的拨号参数读写命令耦合在一起，通过参数管理模块将主站命令模块和 PPP 拨号模块解耦。

在对计量营销一体化系统的采集终端进行部件提取过程中，每完成一个部件的提取，则对原采集终端软件系统进行重构，并完成集成测试和确认测试。这样可以始终端保持原采集终端软件系统可行，成为第一个验证部件的产品。

最后加强对核心资源的管理，方便研发工程师查找部件及扩展部件。到了开发的后期，核心资源库的公共组件慢慢多起来了，同时由于在扩展业务层对很多基础部件进行了参数化和功能扩展，很多部件在标识和功能上都差异不大，出现了有点混淆的问题。为了更好地管理，我建立了 WIKI 服务器，采用 WIKI 服务器进行组件管理，在 WIKI 服务器上对组件的标识、功能、接口及与相关组件的差别等等进行了描述。研发工程师输入相关的关键字就能找到匹配的组件及每个组件详细的说明，方便研发工程师使用。

随着用电信息采集系统核心资源库的建立，国网用电信息采集系统项目的功能也逐渐完善起来。采集终端软件系统在今年 8 月份通过了国家电网电力科学研究院的全功能测试，这对全体项目组成员是一个振奋人心的好消息，说明我们的努力得到了认可。

二、论信息系统建模方法

1、真题原文：

论信息系统建模方法

系统模型在软件开发中扮演着重要的角色。可为已有的系统创建模型，以便更好地理解这些系统；也可以针对待开发的系统创建模型，作为记录业务需求或技术设计的方法。模型是建立信息系统的基础。恰当地运用信息系统建模方法，是成功地进行软件开发的一个关键环节。请围绕“论信息系统建模方法”论题，依次从以下三个方面进行论述。

1. 概要叙述你参与管理和开发的信息系统项目以及你在其中所承担的主要工作。

2. 论述常见的信息系统建模方法的主要内容(包括每种建模方法的核心思想以及所创建的模型)。

3. 具体阐述你参与管理和开发的项目中选择使用的信息系统建模方法以及选择该方法的原因，给出具体的实施过程和实施效果。

2、参考理论：

1. 结构化建模方法

结构化建模方法是以过程为中心的技术，可用于分析一个现有的系统以及定义新系统的业务需求。结构化建模方法所绘制的模型称为数据流图（DFD）。

2. 信息工程建模方法（或数据库建模方法）

信息工程建模方法是一种以数据为中心，但过程敏感的技术，它强调在分析和研究过程需求之前，首先研究和分析数据需求。信息工程建模方法所创建的模型被称为实体联系图（ERD）。

3. 面向对象建模方法

面向对象建模方法将“数据”和“过程”集成到被称为“对象”的结构中，消除了数据和过程的人为分离现象。面向对象建模方法所创建的模型被称为对象模型。随着面向对象技术的不断发展和应用，形成了面向对象的建模标准，即 UML（统一建模语言）。UML 定义了几种不同类型的模型图，这些模型图以对象的形式共建一个信息系统或应用系统。

3、参考范文：

摘要：

2020 年 3 月，本人就职的某互联网公司承担了“XXAPP 电子商务系统”的开发，该项目是集团为用户提供电子商务业务，拉新促活，提高交易效率的重点项目，主要实现消费者的网上购物和在线电子支付以及各种商务活动、交易活动。本人作为商城部门的核心成员有幸加入其中，担任架构师一职，主要负责该系统的需求分析和架构设计。该系统用户面向广大移动端和 PC 端互联网用户，主要用于提供用户网上订购、在线支付、广告宣传等服务，具有营销活动商品维护订单报表等服务能力。本文以该项目为例，最终确定本系统采用基于架构的软件设计方法，经过体系结构需求，体系结构设计，体系结构文档化，体系结构复审，体系结构演化等过程，为开发团队从结构上解决了业务难题，系统上线后得到了广大用户和集团对于系统的认可。

正文：

2020 年 3 月，我参与了公司电子商务系统”的开发，担任系统架构师职务，进行系统的架构工作。该系统的外部实体主要是分为 4 类组织，用户，商户，系统管理人员，金融支付机构。电商系统有几个特点，模型多且复杂度高，数据流链路长，业务变化快，应对突发流量性能要求高等特点。包括商品库存、订单支付、营销活动、售后配送、用户权益、商户管理等几大业务模型。随着业务不断迭代之后，每个模型的边界和管理会越来越复杂和逻辑冗余。

考虑到系统的复杂性，项目整体采用软件体系结构设计，指的是从宏观层次对需要开发的目标系统进行描述。用不同的视图从不同的视角对目标系统进行分析与设计的过程。具体的过程可以分为，体系结构需求，体系结构设计，体系结构文档化，体系结构复审，体系结构实现，体系结构的演化等重要过程。

1、体系结构需求

在体系结构需求阶段，本项目先对目标系统的质量目标进行分解并结合用户提出的需求，分别整理出，功能性需求和非功能性需求。架构必须同时满足功能性需求及非功能性需求。通过分析得到以下几个架构核心关注点：1. 数据的存储模型，由于每个商品可以分为普通商品和活动商品等不同类型，活动又分为预热期和活动期，以及周年大促、会员日、节日活动等类型，商户有自营商户和入驻商户，分别对应不同的后台信息管理类型等，订单分为实物、电子票券、酒店门票、充值等等，2. 由于面向广大互联网用户每次商品、订单等查询时间必须小于 2 秒，7*24 小服务，故障恢复时间不能大于 1 小时。3. 本系统是一个交易类系统必须有动态参数设置及监控界面。

2、标识构件

通过分析系统的核心需求，架构组决定采用构件化开发进行整体的架构设计。设计基于模型属性和模型能力的抽象开发方法，设计使用 event 事件作为一个原子，组装和重发事件驱动，设计无锁的队列存储优化操作相同资源的性能。使用消费事件的方式提升开发效率，对物理模型和业务逻辑进行解耦。

3、标识构件之间的关系

根据总体架构设计形成了高层的构件，包括对象及对象状态构件、外部实体行为构件、消息事件构件、服务框架管理构件。业务模型继承和实现这些构件，如此商品，库存，支付，营销，权益，配送等业务模型都统一的具备了状态，行为，事件的能力构件，骨架完成后再填充各自的业务逻辑。

外部实体行为构件：原子的业务逻辑完成后，通过策略选择，迭代模式来配置和组装原子逻辑成为完整的业务流程功能。例如可以通过业务的策略来选择支付方式，也可以配置多条迭代校验规则完成业务上对商品的校验，这些策略和迭代规则都是配置化可以随意拆分组装。

消息事件构件：事件体系也是交易系统内部抽象的模型，它定义唯一的事件名称和唯一的事件 ID，它支持同步或异步执行，支持不同线程上下文信息传递，支持写数据库设定，支持重试次数设定，支持自动的重试触发，支持无锁队列 ringbuffer 存放事件排队，支持多个事件组合成事件组，支持监控报警。以上是这个事件体系的特点。

服务框架管理构件：在下单支付等动作完成后，系统根据状态机映射触发对应的行为，行为中包括了系统服务调用和事件触发。在触发前把事件 id，上下文信息放入 redis 缓存或数据库，事件自动存入无锁队列，系统的事件 handle 自动从队列中拉取对象和逻辑处理。在处理发生异常时，事件根据配置自动存入数据库等待重试或者丢弃。对于处理失败有通用的重试机制和监控报警规则，让开发快速发现线上问题。

4，技术选择

由于主从架构已经在大数据领域成功应用，同时也有成熟的开源组件。通过与客户讨论决定采用 ZooKeeper 为核心注册及主从架构管理组件实现负载。数据存储采用两级存储，MySQL 作为核心数据存储，Redis 数据库作为数据缓存，减少核心数据库的读压力。RabbitMQ 作为消息核心组件。系统间通信采用基于 HTTP 协议的 Restful 接口形式。

5，架构文档化

根据设计意图，架构组采用 4+1 视图模型，对核心业务场景逻辑功能，开发架构，设计架构，部署架构，存储架构进行了描述并形成架构说明文档及需求质量设计说明书初版。

6，架构复审与细化

架构组与客户专家组对架构说明书，质量设计说明书进行了复审，并提出架构应考虑到数据的安全性，建议对数据库中的关键数据的加密，传输过程中的敏感数据采用加密和超时控制。架构组认为，数据库中的核心数据可以采用两种加密分别对不同数据加密。用户密码敏感信息采用 MD5 数字摘要算法进行加密，对于重要的金额数据采用对称加密。首次复审会议后架构组进行了架构的细化，确定了通信协议种类，主要接口，核心业务设计模式。并且对主要核心类进行的属性和方法的识别。再次经过与客户专家组讨论与评估，最终确定了 XXAPP 电商系统的架构。

此架构在后期的实现过程中经历一些调整，如下单改成多线程模式，消息通知失败的重试机制等部分，但总体结构稳定。系统上线后经客户专家组的验收反馈，本系统架构解决了需求中的核心关注点，并取得了很好的效果，关键是可以对系统运算能力进行水平扩展，能满足未来一定时期的业务需要。

结论：

经历 XXAPP 电商系统架构完整过程使我认识到架构不只是技术，而是要用技术解决客户的业务中的核心关注点，遵循一定架构过程和原则才能设计一个好的系统架构。也让我从中学到构件识别技术，比如构件的独立性，无可见状态，接口规约的版本一致性；构件关系组及架构评审相关知识。

三、论基于 REST 服务的 Web 应用系统设计

1、真题原文：

论基于 REST 服务的 Web 应用系统设计

REST (REpresentational State Transfer) 是指从几种基于网络的架构风格衍生出来的一种混合架构风格，它是目前互联网的核心架构风格。基于 REST 服务 (RESTful Service) 的 Web 应用系统设计任务主要包括：识别并设计 REST 风格的服务，采用面向服务的思想进行 REST 服务集成。采用这种方法设计的 Web 应用系统能够结合 REST 风格和面向服务思想的优点，近年来受到了广泛的关注。

请围绕“基于 REST 服务的 Web 应用系统设计”论题，依次从以下三个方面进行论述。

1. 概要叙述你参与实施的 Web 应用系统开发项目以及你所承担的主要工作。
2. 简要叙述与传统的 Web 服务相比，采用 REST 服务构建的 Web 应用具有哪些优势和不足。
3. 阐述你在设计基于 REST 服务的 Web 应用系统时遇到了哪些问题，如何解决。

2、参考理论：

REST (REpresentational State Transfer) 是指从几种基于网络的架构风格衍生出来的一种混合架构风格，目前 Web 的体系结构正是基于 REST 风格的。

REST 风格中的特点是客户端/服务器、无状态、缓存、统一接口、分层系统和按需代码。

REST 组件通过以一种数据格式转移资源的表述进行通信，可以基于接收者的能力和期待的内容，以及资源的性质动态地选择不同的表述。

与传统的 Web 服务相比，REST 服务主要有以下优势：

REST 服务基于 W3C/IETF 的标准与规范（包括 HTTP、XML、URI 和 MIME 等），其实现技术简单、成熟。

REST 服务基于 URI 和超链接技术，不需要通过集中式的服务信息仓库即可发现服务资源。

REST 服务支持缓存，具有无状态的特性，这些使得 REST 服务能够支持大量客户端，构建的应用系统具有较强的伸缩性。

REST 服务基于轻量级的 Web 框架，仅仅需要基本的开发工具支持，构建过程简单且成本较低。

REST 服务的测试相对简单，采用浏览器即可完成服务功能测试。

与传统的 Web 服务相比，REST 服务主要存在如下不足：

REST 服务倡导的 REST 风格与实际实现尚存在一定差距。例如高层 REST 服务倡导使用 GET、PUT、POST 和 DELETE 所有 4 个统一接口，在 REST 实现部分通常只能采用 GET 和 POST 接口，因为大多数的代理和防火墙会屏蔽其他接口；并且 XHTML 表单中只能使用 GET 和 POST 接口。

REST 服务要求所有的输入参数都必须在 URI 中传递，这样会产生对参数容量大小的限制（目前的大小是 4KB）。如果超出该数量，会导致 HTTP 协议错误（错误代码 414: Request-URI too long）。

在 URI 中表达复杂类型的参数比较困难，且目前对 URI 中的参数不存在一种公认的编组（marshalling）和解编（un-marshalling）方法。

3、参考范文：

参考范文 1：

摘要

2011 年上半年，我在上海中软资源软件有限公司，作为项目组长参与了公司人事管理（HR）系统开发，在系统开发前，公司再信息化建设汇总，也采用了请假流程，薪资管理，招聘等

系统,虽然较为成熟,但是批次之间互相独立,业务数据无法共享,且公司各个分公司间,对 HR 系统使用情况也截然不同,有的分公司由于各种原因,仍然采用手工管理本应信息系统化的业务流程,公司是以软件外包业务为主,所以人力资源管理系统在公司信息化建设中的地位至关重要,这次开发的 HR 系统,整合现有的业务系统,在整个公司内部推行使用,以解决信息孤岛带来的效率低下问题,为了以后的扩展需要,保证在业务和空间尽可能大的扩展性,因此,经过研讨,决定采用 REST web 服务方式实现系统应用层,本文将就 HR 系统开发过程,描述一下对 REST 服务的使用和认识的体会.

正文

上海中软 HR 系统整体此阿勇基于 B/S 的三层架构设计 我作为项目组长参与系统需求分析至测试和部署的整个过程,直接向 IT 部门总监汇报,负责沟通需求,建立项目组,确定系统架构风格和技术实现方案,预定开发周期为 120 天,系统部署后有两个月的试运行期,项目组人数在 2 到 10 人变动,由于项目开发资源(比如时间)紧张,公司 HR 系统业务逻辑复杂,旧系统改进个新需求交织,项目组对业务并不熟悉.

难以在一开始预估江所有业务移植到新系统的时间,因此在开发模型选择上,采用螺旋式增量开发,首先不必追求大而全,在开发完系统基本框架基础上,有限移植最期待改进的业务,经于领导和 HR 部门沟通研究,递交了系统准备实现的功能列表,按不同实现优先级排列,标记为 p1 的功能优先级最高,必须实现,标记为 p2/p3/p4 的功能优先级依次减少,必要时根据资源情况需求进行裁剪,在开发技术的选择上,由于本公司业务已微软外包为主,公司的开发人员大都熟悉一项或多项微软开发技术,作为微软公司合作伙伴可以低成本获取软件开发和管理工具,方便的获取技术支持,所以决定该系统采用微软技术:表示层基 ASP.NET4.0,中间业务层采用 rest 服务实现,基于 wcf 数据访问层,基于微软 orm 构件-arf,在构件的选择上,尽可能降低开发工作量,提高效率,力求避免吧主要精力放在通用的技术细节,而是放在业务逻辑的研究和实现上.

系统部署共有三台服务器,灵台 web 服务器 Windows server 2008 + IIS7.5 分别运行系统网站和 REST 服务,一台数据库服务器 Windows Server2008+Sqlserver2008,经过试运行于 7 月份投入使用,目前系统状况良好,经运行评估,实现了全部必须功能,性能,安全性等质量都达到了原定设计要求目前系统正在根据业务需要,由后续项目组做二次开发.采用 REST 服务方式实现系统业务逻辑层,完全符合项目开发时考虑的两个因素,简单和灵活,传统的 Internet web 服务一般基于 SOAP 协议,构造 soap 情况 xml 虽然目前.net framework 已经

实现了较好的封装,但不便非.net 语言调用,如果客户端页面中采用大量的 ajax 技术,使用 JavaScript 构造 soap 请求非常困难,在调用服务的 web 页面开发完成前,为了调试和测试服务,必须写单独的测试程序,十分不便.相比之下,而 REST 服务具有非常出色的灵活性,既能被服务器端面对对象语言调用,又可以直接被客户端的脚本语言调用,也很方便用浏览器和 Fiddler 工具进行测试,我们在项目中,并没有将 REST 服务单纯视为一串地址的响应,但是基于 HTTP 协议,可以最大的利用 HTTP 协议的语义特性,入数据的增删改查操作对应不同 Http Method.

用户可以用相同访问服务节点,根据需要,通过在请求头中设置不同的 accept-Type,获取不同形式的的结果,如 JSON 或 XML.

更好的性能和缓存支持,由于不需要构造 Soap 消息,请求 Rest 服务显然开销更小,REST 类 Web 服务可以利用高速缓存控制头,从而减少带宽的需求,从而 REST 可以改善响应时间和改进用户体验 可扩展性和无状态性,每个请求都是独立的,一旦被调用,服务器不保留任何会话,这样就可以更具响应性,通过减少时间后通讯状态的维护工作,提高了服务器的可扩展性.

在为系统开发 REST 服务的时候也遇到了一些问题.

一. 安全性方案,并不是指 REST 服务安全性不足,其本身没有内置的安全支持,但是所有 HTTP 支持安全模式和框架几乎都可以用于 REST 服务,真正潜在风险存在于 REST 灵活的使用方式中,既可以被服务器调用又能被客户端调用,所以一开始就要明确区分用户访问权限和系统访问权限,区分 Web 页面权限和 REST 服务权限,但是有时在开发中经常混为一谈,所以加强设计阶段这方面的文档和评估工作

二. 服务接口规范性,REST 服务基于 URI 地址访问,有非常强的语义性,服务接口的每个操作都基于一个 URI 模板,在实际业务中,功能类似的操作被做成多个重载,随着重载的增多,URI 模板如何约定,如何扩展就成为了一个规范性的问题,开始时,没有对这个予以足够重视,在多人开发服务,以至于一些服务操作予以产生了混乱,影响了理解和正确使用,后来又额外花费时间资源统一规定了操作 URI 的格式,这方面业界没有明确的标准,更重要的是,应该从设计时就考虑将来如果需要重载空能的扩展,URI 模板的语义扩展方式,还有一些其他的规范问题,诸如一些操作包括增删改查中一种以上的数据操作,HttpMethod 如何定义,也要一并考虑.

三. WCF REST 自身现在,WCF 从 3.0 到 4.0,已经是较为成熟,而 WCF 的 REST 构件则是全

新的计算,WCF 作为 .net 平台 web service 的代替者,无论在开发还是管理上,都有极大的灵活性,而 WCF REST 灵活体现在开发和使用事故那在管理维护情况下,WCF REST 服务接口操作没有提供入 WCF 一样的灵活的配置功能,URI 模板等元素必须在代码中设置 消息格式虽然可以根据客户端情况输出,但是不能予啊配置文件中设置.

总的来说,虽然 REST 仍在发展中,经验和技術还有很大的进步空间,但是毫无疑问,基于 REST 服务的 WEB 应用程序拥有很多优势,未来在 WEB 系统,将有更光明的应用前景。

四、软件可靠性及应用

1、真题原文：

软件可靠性及应用

目前在企业中,以软件为核心的产品得到了广泛的应用。随着系统中软件部分比例的不断增加,使得系统对软件的依赖性越来越强,对软件的可靠性要求也越来越高。软件可靠性与其他质量属性一样,是衡量软件架构的重要指标。

软件工程中已有很多比较成熟的设计技术,如结构化设计、模块化设计、自顶向下设计等,这些技术为保障软件的整体质量发挥了重要作用。在此基础上,为了进一步提高软件的可靠性,通常会采用一些特殊的设计技术,即软件可靠性设计技术。

在软件可靠性工程体系中,包含有可靠性模型与预测、可靠性设计和可靠性测试方法等。实践证明,保障软件可靠性最有效、最经济、最重要的手段是在软件设计阶段采取措施进行可靠性控制。

请围绕“软件可靠性设计与应用”论题,依次从以下三个方面进行论述。

1. 概要叙述你参与实施的软件开发项目以及你所承担的主要工作。
2. 简要叙述影响软件可靠性的因素有哪些。
3. 阐述常用的软件可靠性设计技术以及你如何应用到实际项目中,效果如何。

2、参考理论：

影响软件可靠性的主要因素有：运行环境（软件可靠性的定义是相对于运行环境的）：

软件规模；软件内部结构（内部结构越复杂，包含的缺陷数就可能越多）；软件的开发方法和开发环境；软件的可靠性投入等。

可靠性设计是在常规的软件设计中，应用各种方法和技术使程序设计在兼顾用户功能和性能需求的同时，全面满足软件的可靠性要求

软件可靠性设计技术就是以提高和保障软件的可靠性为目的，在软件设计阶段运用的一种特殊的设计技术。

1. 容错技术

对于软件失效后果特别严重的场合，例如宇航器控制系统、空中交通控制和核反应堆控制系统等，可采用容错设计方法。

常用的软件容错技术主要有恢复块设计、N 版本程序设计和冗余设计。

恢复块设计就是选择一组操作作为容错设计单元，从而把普通的程序块变为恢复块。一个恢复块中包含有若干功能相同、设计差异的程序块，每一时刻有一个程序块处于运行状态，一旦某程序块出现故障，则用备份程序块予以替换。

N 版本程序设计的核心是通过设计出多个模块或不同版本，对于相同初始条件和相同输入的操作结果进行多数表决（防止因其中某一软件模块/版本的故障而提供了错误的服务，以实现软件容错）。

冗余设计的思路来源于硬件系统，但有所不同。软件冗余设计技术是采用多种不同路径、不同算法或不同实现方法的模块或系统作为备份，在出现故障时进行替换，维持系统的正常运行。

2. 检测技术

在无需在线容错或不能采用冗余设计技术的部分，又有较高的可靠性要求时，一般采用检测性设计，在软件出现故障后能及时发现并报警。但其明显的缺点是不能自动解决故障，如果没有人工干预，最终将导致系统不能正常运行。

3. 降低复杂度设计

软件的复杂性与软件可靠性有密切关系。软件复杂性是产生软件缺陷的重要根源。降低复杂度设计的思想就是在保证实现软件功能基础上，简化软件结构。

3、参考范文：

范文 1：

一 摘要

2019 年 8 月，我司承接了某市医疗集团，智慧药房项目，该项目主要为集团下属 36 家社区卫生服务中心提供药品统一目录管理、药品集中采购、库存管理、处方合理用药审核、药品配发、自动化发药设备驱动提供软件支撑。在该项目中我担任了软件架构设计师一职，主要负责该项目的软件架构设计工作。

本文以智慧药房项目为例，主要论述了可靠性技术在项目中的具体应用，通过将平台服务层进行集群化部署，数据库分布式主从模式，以及服务器 RAID 磁盘阵列，对平台服务层、数据库以及硬件存储多维度的容错、容灾机制，提高了项目件硬件的可用性，安全性和可扩展性。通过可靠性技术的应用，提供了软件运行的质量，保障一软件能够 7*24 小时的稳定运行，最终项目成功上线，并获得了用户的一致好评

一 背景(所有论文同一个项目)

根据深化医药卫生体制改革规范药求，推进药品集中采购，增加药品的供应保障能力，严格监督管理，保障药品的用药安全，所有医疗机构开出的处方，必须通过处方审核后，方可进入划价计费环节，未经审核的处方，不得进行计费 and 配发。2019 年 8 月，我司承接了某医疗集团智慧药房项目，该项目为医疗集团下属 36 家社区卫生服务中心提供软件支持，主要分为药品供应模块、药房管理模块、处方用药审核模块、药品配发模块、设备协同模块。药品供应模块负责药品库存预警，供社区卫生服务中心药房对药品进行集中采购，并将药品发送到省采招平台；药房管理模块主要进行一些基础信息的维护，及关注药品的库存管理，药品批号跟踪；处方用药审核模块，负责对医生开出的药品进行合理用药审核，将不合理的处方审核结果返回给医生，提醒医生修改处方；药口中配发模块，负责将处方中的药品进行调配，打印用法用量标签，确认发药后扣减相应的库存；设备协同模块主要是驱动药房中的一些自动化发药设备，将要药品信息发给设备上位机，根据设备药品的效期进行排序，通过上位机程序调用下位机，驱动设备出药。在该项目中我担任软件架构设计师职务，主要负责软件的架构设计及中间件等技术的选型工作。

——中间正文主体 800~1000 字左右

软件可靠性设计主要通过冗余、心跳等技术对软件进行可靠性的设计，确保软件的稳定运行。冗余可以从硬件、软件两个方面着手，硬件可以使用多台服务器做容灾处理，大型项目会应用异地部署服务器的方式，防止因服务器故障、断网、断电或地震等不可抗拒的自然灾害造成服务不可用，给企业和用户造成损失，软件层面可以使用多应用部署群

集，确保在一个应用出现异常时，可以继续工作。同时应用负载均衡，CDN 等技术进行用户请求分流，保证用户访问性能的同时，对应用进行减压，数据库一般可以采用主从复制的方式部署。

一简介——中间过度作用

本文以智慧药房项目为例，将从集群化部署、数据库主从、软件冗余部署方面论述了可靠性架构设计在项目中的应用。

一介绍三种的应用

集群化部署：项目为了防止一台服务器出现故障，导致应用系统不可用，采用了多服务器冗余方式，以 N+1 的标准选了三台服务器做冗余，并且对服务器磁盘做了 RAID5 阵列，项目采用前后端分离的方式，前台采用 Vue.js，后端平台服务采用的 Java 语言开发，前端 Web 应用系统分别在服务器上部署了三个节点，并且通过使用 Nginx 做反向代理，使得应用不直接对外暴露，提高了安全性，同时配置了集群模式，将用户请求进行分流，提交了系统的可靠性同时也提高了性能，平台服务层以 Docker 的方式进行发布，并且通过 K8S 进行管理，同时引入了 SkyWalking 对接口链路进行监控，并且通过 Spring Admin 对服务节点进行监控，一旦链路应用节点发生故障，立马发送邮件给运维人员，让其及时干预，Java 服务采用 SpringCloud 做微服务架构，通过 Nacos 进行服务注册与发现，通过网关将服务暴露给前端 Web 层调用。

数据库主从：项目中数据层采用了关系型数据库 MySQL，以及 NoSQL 非关系型 MongoDB 同进为了减轻数据库的压力，引入了 Redis 缓存。MySQL 主要存放一些业务数据，如医生开出的处方数据，以及药品目录，ymtlwyth 以及采购订单，配送入库单等，这些数据的特点是使用频率不同，但极其重要，采用了主从复制的方式对 MySQL 进行了冗余，在 Redis 中使用了哨兵主从进行三个节点部署，当 Master 发生故障时，哨兵从新选出一个主节点，Redis 主要存放药品库存信息，在医生开处方时会检查药房药品库存信息，药房将药品发现时会扣除相应的库存，对同一个药品的数量变化频率相当高，而且要及时，否则医生将药开出来后，患者到药房取药时，取不到药，增加医患矛盾，，影响医生开处方的效率及数据一致性及库存及时性。因此引入了 Redis 缓存，在审方模块中 18 万种药品的相互作用关系数据量以迪卡尔积的量级存在，因此采用了适合处理大数据的 MongoDB，MongoDB 在配置文件中同样做了主从方式。

程序容错设计：程序设计及开发中需要考虑容错机制，否则出现错误后可能导致不可

用的情况发生，首选在获取对象属性时，要做异常判断，我们采用了阿里的 Java 契约插件，可以描述出一些可能出现异常的代码，实现方式，加入了 try catch 异常处理机制，对于数据访问层，通过 Spring Data 中的 Redis Template 以及 MongoDB 的 Template 进行主从连接配置，对于 MySQL 采用 MyCat 进行读写分离，在主节点发生故障时，能自动识别新的主节点。微服务开发新业务升级时，不影响现有项目的运行。

— 结尾

项目从 2019 年 8 月启动到 2020 年 10 月历时 14 个月，圆满完成，顺利完成验收，并取得了客户的一致好评，该项目运行一年多，也出现过一些小的问题，由于医院使用的是集团内部局域网，与外网隔离，给排查问题及维护增加了困难，一次发药设备上的上位机不小心被药房老师删除后，不会重新安装操作，上报我司后，我们联系医院信息科老师，在信息科的老师协助下，对软件重新进行了安装，此次事件，导致了设备停止工作了几个小时，影响了药房发药效率。后来我司安排了 1 名售后维护该项目，这一年内也新增了一些发药设备，由于选用了合适的微服务架构，使得设备的接入及服务的扩展变得非常容易，在售后同事的协助下，系统至今运行稳定。该项目的成功，让我意识到了使用了可靠性架构设计的作用和价值，坚定了我对可靠性架构设计应用的信心，合理选择合适的可靠性架构设计，能够大大的提高了软件设计的复用方法，加快开发的进程，在项目中起到事半功倍的作用。经过这次项目，我也看到了自己身上的不足之处，在未来还会不断地更新知识，完善本系统的设计，使系统能够适应国家医改的变化需要，这是作为软件从业的我为之努力的动力和方向。

范文 2:

【摘要】

2017 年 5 月，我参加了某省质量技术监督局“生产制造一体化监管平台”项目（以下简称一体化平台），担任架构师职务负责整体架构设计与中间件选型。该系统以监管企业生产制造为主线，旨在帮助各级监督部门解决监管企业生产、报审、送检、勘验、巡检和执法等问题。本文以一体化平台为例，论述了软件的可靠性设计在项目中的应用，并阐述了选择检错技术、容错技术和降低复杂度技术的原因、实施过程和效果。最后总结了在开发过程中遇到的问题及解决方式。实践证明通过合理的可靠性设计及相关技术的应用，使开发工作取得了成功，一体化平台于 2018 年 10 月验收上线，目前已稳定运行 1 年时间得到了各级监管部门和企业用户的一致认可和好评。

【背景】

质量技术监督在生产领域、市场流通领域等方面都起到至关重要的作用，关乎到企业生产和人民生活。某省质量技术监督局在信息化方面起步较早，先后建设了数套专项系统，但缺乏对业务的统一监管，为贯彻落实国家市场监督管理总局《关于加快推进质量体系监管信息化建设要求》，一体化平台的建设工作受到各级领导的关注。项目建设目标覆盖省、市、县区三级质量监管部门及全省三十多万家企业，为工业生产、报审、送检、勘验、巡检和执法工作提供支持。我所在公司长期从事相关行业的软件开发，对质量监督领域有着丰富经验，因此顺利中标承建了该项目，初期投入 15 名技术人员分成三个组，由我担任架构师职务，负责软件架构设计和中间件选型。

项目启动后通过对业务的梳理，初步将一体化平台划分为四大模块，其中政务服务模块针对的是通过互联网访问系统的企业用户，在功能上主要包括报审、送检、公示和政策法规宣传等功能；而其它三个模块针对的是通过电子政务外网访问系统的监管部门和事业单位用户，其中运行管理模块主要功能包括审批、勘验、颁证、建档；企业监管模块主要功能包括巡检、执法；电子监察模块主要功能包括实时监控、预警纠错和效能评估。

【论点】

在架构设计开始阶段，我意识到可靠性设计对架构设计的重要性，因此在基本需求确定后，我邀请了客户代表和领域专家，针对一体化平台可靠性设计进行了论证。系统建成后要承担起全省制造型企业从生产、送检、勘验到年审等各环节的监管工作，一旦出现故障将造成重大的经济损失和社会性事件，因此必须保证系统 7*24 小时稳定运行，常用 MTTF、MTBF 和 MTTR 指标进行衡量。根据经验保障大型复杂系统的可靠性并不能单纯靠一种或几种技术措施就能做好，而是需要从预防、检查和处理三方面来共同保障，对应架构设计则是降低复杂度技术、检错技术、容错技术，三者缺失任何一环都会造成可靠性的下降。

检错技术

检错是一种能够自动发现故障、确定故障性质和部位的技术，但必须配合容错技术才能排除故障。检错可用专门的程序实现属于程序设计的范围。常见的检错技术有判断返回结果、计算运行时间和状态检测等。相比航空科研等涉及大量计算的高精尖业务场景，一体化平台则更关注的是系统整体提供不间断服务的能力，通过风险分析活动识别出威胁系统可靠性主要的风险来自硬件设备和网络故障。对此在具体实现上我采取了两个措施，一是，针对数据

交换业务，开发了一个网络监控模块，采用 ping/echo 程序监控前置机到生产库的网络情况，并基于图形化界面实时展示网络状态。另外，为了防止交换程序假死对 Kettle (ETL 工具) 组件进行了集成，可实时检测服务状态，当故障发生时首先停止相关服务记录异常，然后通过网关发送报警短信，由人工处理；二是，针对双机部署的 web 程序我采用了 Keepalived 组件做心跳连接，当检测到 Master 宕机可迅速切换到 Backup 继续服务，整个过程自动完成，无需人工干预。通过以上检错技术的应用成功的提高了系统故障检测和应急处理能力。

容错技术

容错是保证系统在某些组成部分出现故障时仍能正常工作的技术。常见的容错技术有冗余技术、双机技术、集群技术。针对一体化平台用户量大、可用性要求高的业务特点，我选择了采用双机技术和集群技术来满足不同场景的容错要求。在逻辑上我将一体化平台可划分为面向公众服务和内部办公服务两部分。对于面向公众服务，作为办事入口和宣传平台必须提供 7*24 不间断服务。根据这个业务特点我在部署方式上采用了高可用集群技术，首先使用一台 Nginx 服务器 + 三台应用服务器构成负载均衡集群解决并发问题。然后为避免 Nginx 单点故障，实际部署两套 Nginx 作为 Master 节点和 Backup 节点同时指向应用服务器，并使用 Keepalived 组件连接两台 Nginx 服务器做心跳检测，构成高可用集群保障服务的连续运行；对于内部办公服务，因部署的节点较多，考虑到资源限制采用的是双机互备技术，将两个独立的模块在两台机器上同时运行，但彼此均设为 Backup，当某一台服务器心跳检测失败时，服务可用在短时间内进行切换，从而保证了业务的持续性。通过以上容错技术的应用成功的保障了系统的稳定运行。

降低复杂度设计

随着系统规模不断的扩大，架构复杂度也在不断的攀升，对于大型复杂系统有效的管理软件复杂度，已成为保障系统可靠性的一个重要手段。常见的降低复杂度方式有应用架构设计，使用构件和设计模式等。针对架构设计本次项目采用 J2EE 技术开发，其本身就具有 N 层架构的特点，结合系统业务我采用了三层 B/S 架构设计来构建系统，其层次划分为表现层、功能层和数据层。表现层，运行在客户机浏览器上，是应用的用户接口担负着用户与应用间对话功能；功能层，运行在 Web 服务器上，负责响表示层的用户请求、执行业务逻辑；数据层，就是数据库管理系统，它运行在数据库服务器上负责管理数据读写和执行事务。另外，在项目中我使用了多种构件来降低系统复杂度，例如使用 RabbitMQ 消息中间件，使系统以消息为中心进行交互有效降低各模块间耦合度；使用 SpringMVC 框架解耦视图和模型，

提高代码重用度；使用 Hibernate 框架解耦持久层和数据库，使持久层专注于面向对象编程，无需关心具体实现，降低了程序复杂度。最后在开发过程中还应用了抽象工厂、策略、责任链等多种模式来解耦代码。通过以上方式成功的降低了软件的复杂度，提高了系统可靠性。

【结尾】

由于在项目中多种可靠性技术的成功应用，有效的保障了系统的稳定性和可靠性，使开发工作取得了成功。但在项目过程中也遇到过问题，因客户单位部分用户派驻到办事大厅工作，窗口办公需要配合高拍仪、读卡器等硬件设备，而浏览器出于安全考虑隔离了 web 程序调取本地资源仅提供有限支持，给系统推广带来了困难。对此我采用了富客户端技术，使用 Electron 框架封装了一个客户端专门负责调取硬件资源及时解决了该问题，没有对项目造成实质影响。

通过项目组全体成员的共同努力，一体化平台于 2018 年 10 月完成验收正式上线，截至目前系统运行稳定运维工作量可控，得到了监管部门和企业用户的一致认可和好评。在今后的的工作中我将不断总结和提升专业技术能力，争取为我国信息化事业贡献一份力量。